

FACULTY OF SCIENCE

Software Engineering Teaching Unit

University of Kelaniya, Sri Lanka

SENG 31242 – System Design Project

Final Design Report

TeaRoutePay

Project	Tea Route Pay
GitHub Organisation	https://github.com/seng31242-tea-route-pay-2026
Repository	tea-collection-payment-system
Programme	B.Sc. (Hons.) in Software Engineering
Year / Semester	Year 3, Semester 1
Course Code	SENG 31242
Date	2026

Name	Student Number
SE/2022/009	Prabhath Harsha
SE/2022/024	Thisari Uresha
SE/2022/027	Shashini Wijesingha
SE/2022/032	Yehan Kanishka
SE/2022/044	Nihadh Nilabdeen

Acknowledgements

We would like to express our sincere gratitude to our project supervisor for their invaluable guidance and support throughout the design phase of this project. We extend our thanks to the tea estate owner who served as our client, generously sharing their time, domain expertise, and operational insights during the requirements elicitation process. We are also grateful to the Faculty of Science, Software Engineering Teaching Unit, University of Kelaniya, for providing the academic framework and resources that made this project possible. Finally, we thank our team members for their dedication, collaboration, and commitment to producing quality work throughout the project.

Abstract

This report presents the comprehensive analysis and design of TeaRoutePay, an offline-first tea collection and payment management system developed as part of SENG 31242 – System Design Project at the University of Kelaniya. The system addresses critical inefficiencies in the traditional tea leaf collection process used by small-scale tea estates in Sri Lanka, including manual data entry errors, paper-based record vulnerability, lack of mobile collection capability, and error-prone payment calculations. The requirements engineering phase employed structured interviews and observation with a real client to produce a Software Requirements Specification (SRS) documenting 20 use cases, 53 functional requirements, and 29 non-functional requirements. The system analysis includes use-case diagrams, activity diagrams, and system sequence diagrams for all critical business flows. The system design phase produced a Software Design Specification (SDS) defining a microservices architecture using Flutter for cross-platform mobile and desktop clients, Java Spring Boot for backend services, PostgreSQL for persistence, and Apache Kafka for event streaming. The design addresses security through JWT authentication, AES-256 encryption, and OWASP Top 10 mitigations. A dual synchronisation architecture supports both LAN and cloud modes to accommodate varying connectivity conditions in rural tea-growing regions.

Keywords: tea collection, payment management, offline-first, mobile application, microservices architecture, Software Requirements Specification, Software Design Specification

Table of Contents

Acknowledgements	2
Abstract	2
Table of Contents	3
List of Abbreviations	6
Chapter 1: Introduction	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Aims and Objectives	8
1.4 Scope	8
1.5 Organisation of Report	9
Chapter 2: System Analysis	9
2.1 Introduction	10
2.1.1 Purpose and Scope	10
2.1.2 Definitions, Acronyms, and Abbreviations	10
2.1.3 Overview of Document	11
2.2 Overall Description	12
2.2.1 Product Perspective (Context Diagram)	12
2.2.2 User Classes and Characteristics (Personas)	13
2.2.3 Operating Environment	13
2.2.4 Design and Implementation Constraints	13
2.2.5 Assumptions and Dependencies	14
2.3 System Analysis	15
2.3.1 Fact-Gathering Techniques Used	15
2.3.2 Existing System Analysis	15
2.3.2.1 Current System Overview	15
2.3.2.2 Pain Points Identified	16
2.3.3 Use-Case Diagram (System Boundary)	16
2.3.3.1 System Overview – All Actors	17
2.3.3.2 Tea Collector (Mobile App)	18
2.3.3.3 Owner / Office Staff (Desktop App)	18
2.3.3.3 Farmer (External / Physical Actor)	19
2.3.4 Use-Case Descriptions	20
2.3.4.1 UC - 01 – Log in to mobile app	20
2.3.4.2 UC - 02 – View assigned route	20
2.3.4.3 UC - 03 – Scan Farmer QR Code	21
2.3.4.4 UC - 04 – Record leaf weight	22
2.3.4.5 UC - 05 – Item distribution and record keeping	23
2.3.4.6 UC - 06 – Issue Advance Payment and Record Keeping	23
2.3.4.7 UC - 07 – Synchronize Collection Data	24
2.3.4.8 UC - 08 – Log in to desktop app	25
2.3.4.9 UC - 09 – Register Farmer	26

2.3.4.10 UC - 10 – Manage farmer profiles	27
2.3.4.11 UC - 11 – Manage Deductions	28
2.3.4.12 UC - 12 – Set monthly tea rate	29
2.3.4.13 UC - 13 – Calculate Monthly Payment	29
2.3.4.14 UC - 14 – View payment summary	30
2.3.4.15 UC - 15 – Approve Payment Run	31
2.3.4.16 UC - 16 – Process bank transfer	31
2.3.4.17 UC - 17 – Send SMS notification	32
2.3.4.18 UC - 18 – Generate reports	33
2.3.4.19 UC - 19 – Manage user accounts	33
2.3.4.20 UC - 20 – Manage collection routes	34
2.3.5 Activity Diagrams	34
2.3.5.1 Daily Tea Leaf Collection	34
2.3.5.2 Data Synchronization (Local and Cloud)	36
2.3.5.3 Monthly Payment Calculation and Processing	37
2.3.5.4 Deduction Management (Loans and Fertilizer)	38
2.3.5.5 Farmer Registration and Profile Management	39
2.3.5.6 Advance Payment	40
2.4. Functional Requirements	41
2.4.1 User Authentication and Authorization	41
2.4.2 Farmer Management	43
2.4.3 Route Management	45
2.4.4 Field Collection – Mobile Application	46
2.4.5 Data Synchronization	48
2.4.6 Tea Rate Management	50
2.4.7 Deduction Management	50
2.4.8 Monthly Payment Calculation	52
2.4.9 Payment Approval Workflow	53
2.4.10 Bank Transfer	54
2.4.11 SMS Notification	55
2.4.12 Reporting	56
2.5. Non-Functional Requirements	57
2.5.1 Performance	57
2.5.2 Security	57
2.5.3 Usability	58
2.5.4 Reliability	58
2.5.5 Scalability	58
2.5.6 Maintainability	59
2.6. Alternative Solutions & Feasibility Study	60
2.6.1 Alternative Solutions Analysed	60
Alternative 1 – Off-the-Shelf Agricultural Management Software	60
Alternative 2 – Spreadsheet-Based Enhancement (Enhanced Legacy Approach)	61
Alternative 3 – Recommended: Custom Multi-Platform System (TeaRoutePay)	61

2.6.2 Feasibility Study	62
2.6.2.1 Technical Feasibility	62
2.6.2.2 Economic Feasibility	64
2.6.2.3 Operational Feasibility	65
2.6.2.4 Schedule Feasibility	65
2.6.3 Justified Recommendation	66
Chapter 3: System Design	67
3.1. INTRODUCTION	67
3.1.1 Purpose	67
3.1.2 Scope	68
3.1.3 Overview of the Document	69
3.2. SYSTEM OVERVIEW	70
3.2.1 System Description and the Hybrid Local-First Architecture	70
3.2.2 Relationship to the SRS	71
3.3. ARCHITECTURAL DESIGN	73
3.3.1 Selected Architecture and Justification	73
3.3.2 High-Level Microservices Architecture	74
3.3.3 Service Responsibilities	75
3.3.4 Component Diagram	77
3.3.5 Deployment Diagram	78
3.3.6 Technology Stack	79
3.4. DETAILED DESIGN	81
3.4.1 Domain Class Diagram	81
3.4.2 Sequence Diagrams (Use-Case Realisations)	82
SD-01 — Log in to Mobile App (UC-01)	83
SD-02 — View Assigned Route (UC-02)	84
SD-03 — Scan Farmer QR Code (UC-03)	85
SD-04 — Record Leaf Weight (UC-04)	86
SD-05 — Item Distribution and Record Keeping (UC-05)	87
SD-06 — Issue Advance Payment and Record Keeping (UC-06)	88
SD-07 — Synchronise Collection Data (UC-07)	89
SD-08 — Log in to Desktop App (UC-08)	90
SD-09 — Register Farmer (UC-09)	91
SD-10 — Manage Farmer Profiles (UC-10)	92
SD-11 — Manage Deductions (UC-11)	92
SD-12 — Set Monthly Tea Rate (UC-12)	93
SD-13 — Calculate Monthly Payment (UC-13)	93
SD-14 — View Payment Summary (UC-14)	94
SD-15 — Approve Payment Run (UC-15)	95
SD-16 — Process Bank Transfer (UC-16)	96
SD-17 — Send SMS Notification (UC-17)	96
SD-18 — Generate Reports (UC-18)	97
SD-19 — Manage User Accounts (UC-19)	98

SD-20 — Manage Collection Routes (UC-20)	99
3.4.3 State Machine Diagrams	99
3.4.3.1 Farmer State Machine	100
3.4.3.2 Payment Run State Machine	100
3.4.3.3 Collection Record State Machine	101
3.4.4 Database Design	102
3.4.4.1 Central (Cloud / Office) Entity-Relationship Model	102
3.4.4.2 Mobile Local (SQLite) Schema	103
3.4.4.3 Data Dictionary	104
3.4.4.4 Architectural Data-Integrity Rules	106
3.4.4.5 Requirements Traceability Matrix (Functional)	106
3.5. USER INTERFACE DESIGN	110
3.5.1 UX Decisions for Low-Technical-Proficiency Users	110
3.5.1.1 Mobile Application Wireframes	111
3.5.1.2 Office Desktop Application Wireframes	113
3.5.2 Navigation Flow	117
3.5.3 UI Resources and Prototyping	118
3.6. INTERFACE DESIGN	119
3.6.1 External Interfaces	119
3.6.1.1 Bank Transfer API	119
3.6.1.2 SMS Gateway API	120
3.6.2 Internal Interfaces	121
3.6.2.1 Synchronisation Payload	121
3.7. SECURITY DESIGN	123
3.7.1 Authentication and Authorisation	123
3.7.2 Data Protection	124
3.7.3 Auditability	124
3.7.4 OWASP Top 10 Mitigations	125
3.8. NON-FUNCTIONAL DESIGN DECISIONS	126
APPENDIX A — ARCHITECTURAL DECISION RECORDS	129
ADR-001: Adopt a Microservices Architecture	129
ADR-002: Use PostgreSQL as the Central Database	130
ADR-003: Use RabbitMQ for Asynchronous Event-Driven Communication	130
ADR-004: Offline-First Mobile Architecture with Encrypted SQLite	131
Chapter 4: Conclusion	132
4.1 Summary of Objectives	132
4.2 Limitations	132
4.3 Future Work	132
References	133

List of Abbreviations

Abbreviation	Definition
ADR	Architectural Decision Record
AES	Advanced Encryption Standard
API	Application Programming Interface
AWS	Amazon Web Services
EKS	Elastic Kubernetes Service
ER	Entity-Relationship
FR	Functional Requirement
JWT	JSON Web Token
LAN	Local Area Network
LKR	Sri Lankan Rupee
NFR	Non-Functional Requirement
NIC	National Identity Card
OWASP	Open Web Application Security Project
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDS	Software Design Specification
SMS	Short Message Service
SQL	Structured Query Language
SRS	Software Requirements Specification
SSD	System Sequence Diagram
TCPS	Tea Collection and Payment System
TLS	Transport Layer Security
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Chapter 1: Introduction

1.1 Background

The tea industry in Sri Lanka remains a significant contributor to the national economy, with small-scale tea farmers supplying green leaf to collection centres and factories. The current process at many tea estates relies on manual record books and legacy Windows-based desktop applications. Tea collectors record daily leaf weights during lorry route collection, and office staff later enter the data manually into spreadsheets or desktop software. Monthly payments are subsequently calculated by applying a range of deductions including loans, fertilizer advances, and other tea-related charges.

This project, undertaken as part of SENG 31242 – System Design Project at the University of Kelaniya, addresses the inefficiencies of this manual process by designing a modern, offline-first Tea Collection and Payment System.

1.2 Problem Statement

The existing system suffers from several critical pain points:

Manual data entry leads to transcription errors and payment disputes with farmers.

Paper-based route records are easily lost or damaged.

The legacy desktop application does not support mobile collection and cannot operate without a stable LAN or internet connection.

Monthly payment calculation is time-consuming and prone to arithmetic errors.

There is no audit trail or reporting capability to support business decisions.

1.3 Aims and Objectives

The primary aim of this project is to design a comprehensive software system that digitises and streamlines tea collection and payment management. The specific objectives are:

Design a mobile application for real-time, offline-capable tea leaf weight entry during lorry routes.

Design an office desktop application for managing farmer records, deductions, payment calculation, and reporting.

Define a synchronisation architecture supporting both local (LAN) and cloud-based sync modes.

Specify integration points for SMS notification and bank transfer services.

1.4 Scope

This report covers the analysis and design phases of the project only. Implementation will be undertaken in SENG 34213 – System Development Project in the following semester. The scope includes:

- Collector Mobile Application (Android)
- Office Desktop Application (cross-platform)
- Backend API Server and Cloud Database
- Local synchronisation over LAN
- Cloud synchronisation over the Internet
- SMS notification integration
- Bank transfer interface

1.5 Organisation of Report

Chapter 2 presents the system analysis, including findings from fact-finding sessions, use cases, and the Software Requirements Specification.

Chapter 3 covers the system design, architecture decisions, detailed design artefacts, and the Software Design Specification.

Chapter 4 concludes the report with a summary of objectives met, known limitations, and proposed future work.

Chapter 2: System Analysis

This chapter presents the complete Software Requirements Specification (SRS) for the TeaRoutePay system. The SRS documents all functional and non-functional requirements, use cases, activity diagrams, and feasibility analysis.

2.1 Introduction

2.1.1 Purpose and Scope

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for TeaRoutePay – a multi-platform tea route collection and payment management system. This system replaces the current paper-based field process and legacy Windows desktop application. The system modernizes tea leaf collection recording, farmer profile management, deduction processing, monthly payment calculation, bank transfer disbursement, and SMS farmer notification.

This document governs design, implementation, system testing, acceptance testing, and future maintenance.

2.1.2 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
TCPS	Tea Collection & Payment System (alternate short form used in IDs)
SRS	Software Requirements Specification
UUID	Universally Unique Identifier
FR	Functional Requirement (numbered FR-01, FR-02 ...)
NFR	Non-Functional Requirement
UC	Use Case
SSD	System Sequence Diagram
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
LAN	Local Area Network
QR	Quick Response code – used for farmer identification
SMS	Short Message Service
LKR	Sri Lankan Rupee
SQLite	Embedded relational database for offline mobile / office local storage
TLS	Transport Layer Security
JWT	JSON Web Token – used for session authentication

Term / Acronym	Definition
ORM	Object-Relational Mapping framework
CI/CD	Continuous Integration / Continuous Delivery
BR	Business Rule

2.1.3 Overview of Document

Section 1 – Introduction: purpose, scope, definitions, references.

Section 2 – Overall Description: product perspective, user classes, operating environment, constraints, assumptions.

Section 3 – System Analysis: fact-gathering techniques, existing system analysis, use-case diagrams, use-case descriptions, activity diagrams, and system sequence diagrams.

Section 4 – Functional Requirements: numbered FR list (FR-01 ... FR-53) with ID, title, description, priority, source, and verification method.

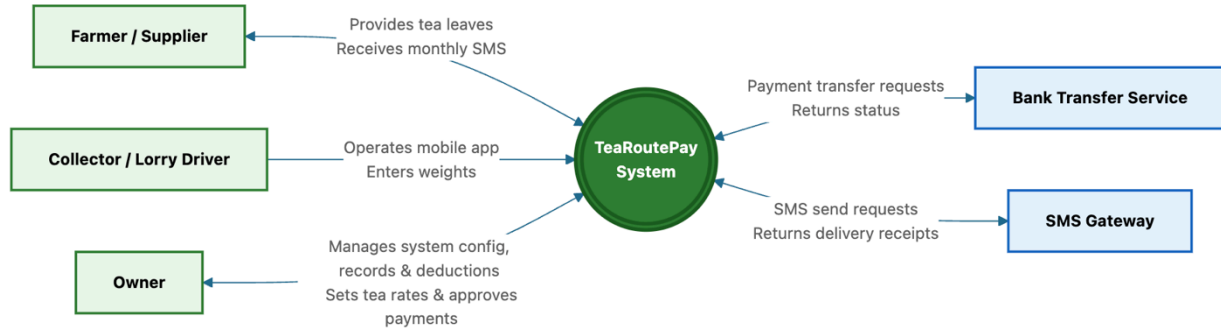
Section 5 – Non-Functional Requirements: performance, security, usability, reliability, scalability, maintainability.

Section 6 – Alternative Solutions & Feasibility Study: two alternatives analyzed, four-dimension feasibility, justified recommendation.

2.2 Overall Description

2.2.1 Product Perspective (Context Diagram)

TeaRoutePay is a greenfield replacement for a legacy Windows desktop application and manual paper-based route books. The diagram below illustrates the system boundary and all external entities.



Context boundary – external entities that interact with TeaRoutePay:

External Entity	Interaction	Direction
Farmer / Supplier	Provides tea leaves; receives monthly SMS payment notification	Two-way
Collector / Lorry Driver	Operates mobile app during field route; enters weights	Input
Office Staff / Owner	Manages farmer records, deductions, reports, payment calculation Sets tea rate, approves payment run, views dashboard and audit log Manages users, roles, sync mode, system configuration	Both
Bank Transfer Service (ext.)	Receives approved payment transfer request; returns status	Both
SMS Gateway (ext.)	Receives SMS send requests; returns delivery receipts	Both

2.2.2 User Classes and Characteristics (Personas)

User Class	Persona	Technical Level	Primary Interface	Key Responsibilities
Office Staff	Nimal, 52, business proprietor and office administrator. Comfortable with Windows software.	Medium	Desktop App	Set monthly rate, approve payment, view reports and audit log. Farmer registration, receipt printing
Collector	Suresh, 28, lorry driver. Uses a smartphone. Follows a route daily.	Low	Mobile App	QR scan, weight entry, data sync at end of route

2.2.3 Operating Environment

1. **Mobile Application:** Android 8.0 (API 26) + or iOS 13.0+. Min 2 GB RAM. Device camera required for QR scanning.
2. **Office Desktop Application:** Windows 10 (64-bit) or Windows 11. Min 4 GB RAM, 2 GHz CPU, 10 GB free disk.
3. **Backend / API Server:** Linux (Ubuntu 20.04 LTS+). Min 4 GB RAM, 2 vCPUs.
4. **Local Database (Mobile):** SQLite (embedded, no separate install).
5. **Local Database (Office):** PostgreSQL 13+.
6. **Cloud Database:** PostgreSQL 13+ on cloud host (when cloud sync mode is selected).
7. **Network:** No network required for field collection. LAN (802.11n Wi-Fi / 100 Mbps Ethernet) for local sync. Internet (min 2 Mbps) for cloud sync.

2.2.4 Design and Implementation Constraints

1. **Offline-First Resilience:** The mobile application must be fully functional offline during field collection with absolutely zero network dependency.
2. **Flexible Synchronization:** Both LAN sync (local network) and cloud sync (internet) modes must be supported. The active mode must be selectable via application configuration or UI toggle, requiring no code changes to switch.
3. **Financial Accuracy:** All monetary calculations and storage must use fixed-point arithmetic (or integer representation of cents/lowest denomination) to guarantee exact LKR accuracy and strictly prohibit the use of floating-point arithmetic to prevent rounding errors.
4. **Data Immutability & Auditing:** Approved payment records must be strictly immutable, no modifications or deletions are permitted after Factory Owner approval. Furthermore, the

system must maintain an immutable, append-only audit log tracking all changes to financial records and system configurations.

5. Data Security (In Transit & At Rest): Farmer personal and financial data must be encrypted in transit using TLS 1.2 or higher. Data stored locally on mobile devices or offline desktop clients must be encrypted at rest (e.g., utilizing SQLCipher for embedded SQLite databases).
6. Secrets Management: Bank API credentials, SMS API keys, database passwords, and other sensitive configurations must be managed via a secure secrets manager or environment variables. They must never be hardcoded into the source code repository.
7. Hardware Interoperability: QR codes generated by the system must use standard formats (e.g., plain text IDs or standard JSON) to ensure they are scannable by standard device cameras without the need for proprietary or third-party decoding software.

2.2.5 Assumptions and Dependencies

1. All farmers are pre-registered in the system before any collection route is executed.
2. Unique QR code cards distributed to all registered farmers.
3. The business operates on a monthly payment cycle with one consolidated payment per farmer per month.
4. A LAN is available at the office premises when local sync mode is used.
5. Internet connectivity (via mobile data in the field or Wi-Fi at the office) is available when cloud sync mode is utilized. The mobile application can sync directly to the cloud server from any location with an active internet connection.
6. External SMS and bank transfer service providers supply documented REST APIs and test environments.
7. Tea rates are updated by the Owner before payment calculation is run.
8. Device clocks are synchronized to within 1 minute to support accurate conflict resolution.

2.3 System Analysis

2.3.1 Fact-Gathering Techniques Used

The following methods were used to collect, understand, and confirm the requirements for the system:

Technique	Description	Artefacts Produced
Client Interviews	Structured interviews with the business owner to understand current workflows, pain points, and desired outcomes. Three interview sessions conducted.	Interview transcripts, pain-point list, initial requirement seeds
On-Site Observation	Direct observation of a lorry collection route and the office payment calculation process to understand the real-world operational context.	Process notes, route timing data, paper form samples
Document Analysis	Review of existing paper route books, the legacy desktop application screenshots, and current bank transfer forms.	Legacy system analysis, data entity list
Questionnaire	Questionnaire distributed to three collectors to capture mobile usage expectations and connectivity conditions in the field.	Usability constraints, NFR inputs

2.3.2 Existing System Analysis

2.3.2.1 Current System Overview

The current process relies on two manual components:

- (1) paper route books carried by collectors in the lorry
- (2) Legacy Windows desktop application used by office staff to re-enter field data and calculate monthly payments.

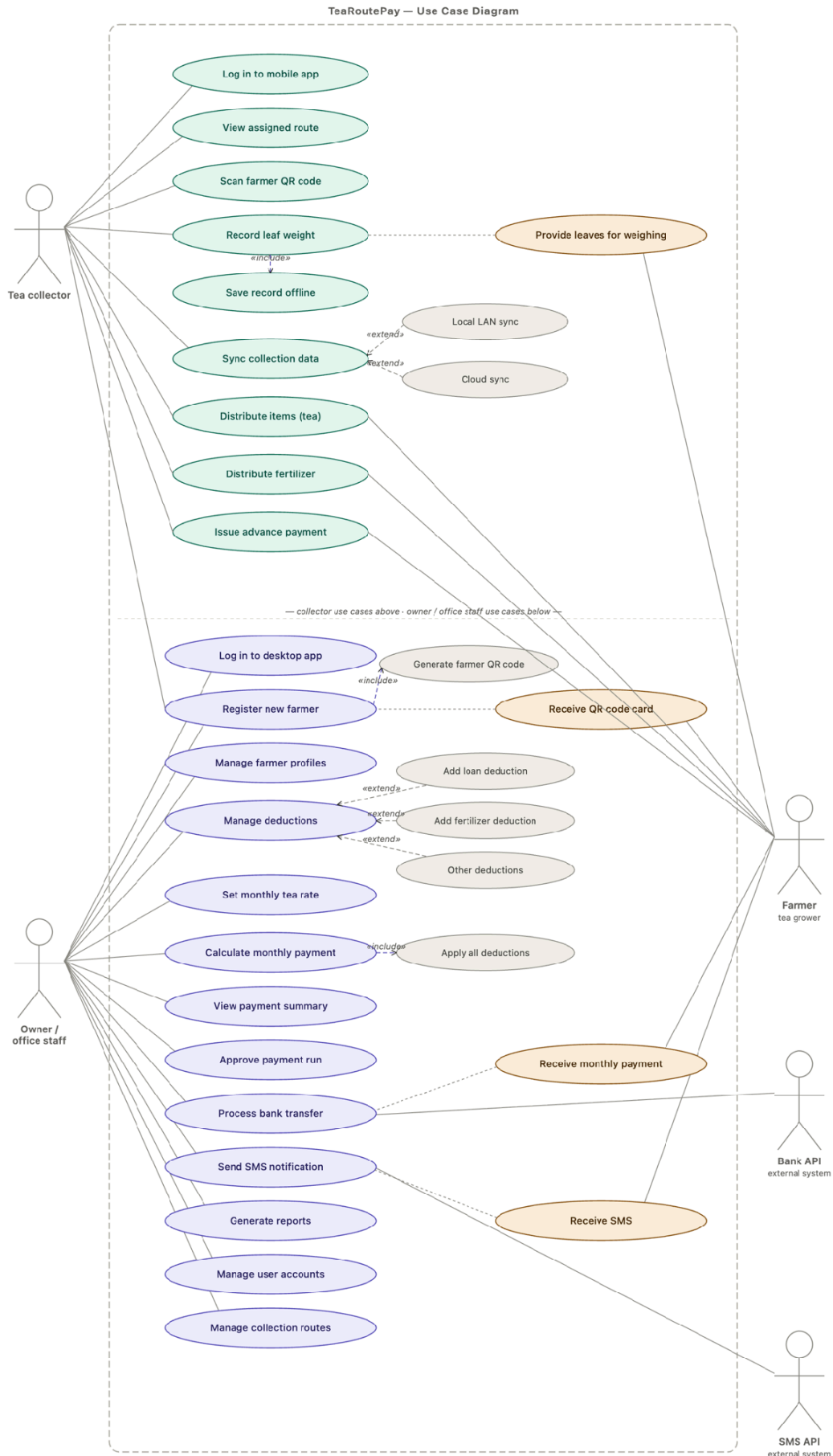
2.3.2.2 Pain Points Identified

#	Pain Point	Impact	Proposed Resolution
P-01	Field data recorded on paper, manually re-entered in office → transcription errors	Payment inaccuracies, farmer disputes	Offline mobile app with direct digital entry
P-02	Legacy desktop has no network capability → no real-time data sharing	Delays in payment processing	Backend API with LAN/cloud sync
P-03	Payment calculation is manual and slow (2–3 days per month)	Staff overtime, delayed farmer payments	Automated calculation engine
P-04	No audit trail for financial changes	Cannot investigate disputes	Immutable audit log for all financial records
P-05	Farmers do not receive notifications when the collected tea is entered into the office system and payments are made.	Farmers make unnecessary office trips	Automated SMS on payment confirmation
P-06	Deduction tracking is done manually	Errors in net payment	Integrated deduction management module
P-07	Paper QR-less identification	Collectors must write their name and supplier number with each collection round.	QR code-based farmer identification

2.3.3 Use-Case Diagram (System Boundary)

Use-case diagrams are presented per actor below. A combined system overview diagram should also be created showing all actors, all use cases, and include/extend relationships within the system boundary of TeaRoutePay.

2.3.3.1 System Overview – All Actors



2.3.3.2 Tea Collector (Mobile App)

- UC01: Log in to mobile app
- UC02: View assigned route
- UC03: Scan farmer QR code
 - UC03a: Record leaf weight
- UC04: Record leaf weight
 - UC04a: Save record offline
- UC05: Item distribution and record keeping
 - UC05a: Save record offline
- UC06: Issue advance payment and record keeping
 - UC06a: Save record offline
- UC07: Sync collection data
 - UC07a: Local LAN sync
 - UC07b: Cloud sync
- UC09: Register new farmer (*Actor: Tea Collector*)
 - UC09a: Generate or add QR code

2.3.3.3 Owner / Office Staff (Desktop App)

- UC08: Log in to desktop app
- UC09: Register new farmer (*Actor: Owner / Office Staff*)
 - UC09a: Generate or add QR code
- UC10: Manage farmer profiles
- UC11: Manage deductions
 - UC11a: Add loan deduction
 - UC11b: Add fertilizer deduction
 - UC11c: Other deductions
- UC12: Set monthly tea rate
- UC13: Calculate monthly payment
 - UC13a: Apply all deductions
- UC14: View payment summary
- UC15: Approve payment run
- UC16: Process bank transfer – (*interacts with Bank API*)

- UC17: Send SMS notification – (*interacts with SMS API*)
- UC18: Generate reports
- UC19: Manage user accounts
- UC20: Manage collection routes

2.3.3.3 Farmer (*External / Physical Actor*)

- UC21: Receive QR code card
- UC22: Provide leaves for weighing
- UC23: Receive monthly payment – (*via Bank API*)
- UC24: Receive fertilizer and other items
- UC25: Receive SMS – (*via SMS API*)

2.3.4 Use-Case Descriptions

2.3.4.1 UC - 01 – Log in to mobile app

Use Case ID	UC-01
Use Case Name	Log in to mobile app
Actor(s)	Collector
Preconditions	The mobile app is installed, and the Collector has an active account.
Postconditions – Success	Collector is authenticated and the main dashboard is displayed.
Postconditions – Failure	Access is denied; error message displayed.
Main Flow	1. Collector opens the app. 2. Collector enters Username/Phone and Password/PIN. 3. Collector taps "Log In". 4. System validates credentials against the database (local cache if offline). 5. System grants access and loads the dashboard.
Alternative Flow 1	At step 4: Invalid credentials. System displays "Incorrect username or password". Collector retries.
Business Rules	BR-01: If logging in offline, the system must use a securely cached token from the last successful online login.

2.3.4.2 UC - 02 – View assigned route

Use Case ID	UC-02
Use Case Name	View assigned route
Actor(s)	Collector
Preconditions	Collector is logged in and data has been synced.
Postconditions – Success	The route details and list of farmers for the day are displayed.
Postconditions – Failure	No route is displayed; error or "No Route Assigned" message shown.

Main Flow	1. Collector selects "My Route" from the dashboard. 2. System retrieves the daily route assigned to the Collector. 3. System displays the route name, geographical area, and the list of farmers to visit in order.
Alternative Flow 1	At step 2: No route. System finds no route assigned for the current date. Collector must contact Office Staff to assign a route and re-sync or need to check your internet connection
Business Rules	BR-02: Routes must default to displaying only farmers actively assigned to the logged-in Collector for the current calendar day.

2.3.4.3 UC - 03 – Scan Farmer QR Code

Use Case ID	UC-03
Use Case Name	Scan Farmer QR Code
Actor(s)	Collector
Preconditions	Collector is logged in and has selected a collection route. Farmer has a printed QR code. Device camera is functional.
Postconditions – Success	Farmer is identified and the weight entry screen is displayed.
Postconditions – Failure	Farmer is not identified; error is displayed. Collector may attempt manual selection.
Main Flow	1. Collector taps 'Scan QR' on the collection screen. 2. System activates the device camera in QR scan mode. 3. Collector positions the device over the farmer's QR code. 4. System decodes the QR code and retrieves the farmer ID. 5. System looks up the farmer record in the local database. 6. System displays the farmer's name and the weight entry form. 7. Collector visually confirms the farmer's name shown on screen.
Alternative Flow 1	At step 4: QR code is unreadable (damaged or poor lighting). System shows 'QR unreadable' message. Collector retries or selects farmer manually by name/ID.
Alternative Flow 2	At step 5: Farmer ID not found in local database. System shows 'Farmer not found'. Collector contacts Office Staff to verify registration and re-sync data.
Business Rules	BR-03: The QR code shall be generated using the farmer's NIC and a unique system identifier, with no personal data stored in plain text.

BR-04: Manual fallback requires confirmation tap before proceeding to weight entry.

2.3.4.4 UC - 04 – Record leaf weight

Use Case ID	UC-04
Use Case Name	Record leaf weight
Actor(s)	Collector
Preconditions	Farmer has been identified via UC-03 or manual selection. A collection route session is active.
Postconditions – Success	Weight entry is saved to mobile local database with status 'Pending Sync'. Running session total is updated.
Postconditions – Failure	No record is saved. Collector is shown a validation error and must re-enter a valid weight.
Main Flow	1. System displays the weight entry form showing farmer name, route, date, and current time. 2. Collector enters the tea leaf weight in kilograms using the numeric keypad. 3. System automatically applies standard bag / water deductions (if configured) to calculate net weight. 4. Collector taps 'Save Entry'. 5. System validates the entry (ensures it is not empty or an impossible number). 6. [UC-04a] System writes the collection record to the mobile local database with status 'Pending Sync'. 7. System displays a success confirmation and returns to the scanner/route screen. 8. System returns to the collection home screen ready for the next farmer.
Alternative Flow 1	At step 5: Weight is zero, negative, or non-numeric. System highlights the field in red and displays a validation error message. Collector must correct the value before saving.
Alternative Flow 2	At Step 6: Storage Error – If device storage is full, the system displays "Device storage full." The collector must free device storage or sync pending data before continuing.
Business Rules	BR-05: Maximum weight per entry is configurable by the Administrator (default 500 kg). BR-06: A duplicate entry for the same farmer on the same date requires explicit Collector confirmation.

2.3.4.5 UC - 05 – Item distribution and record keeping

Use Case ID	UC-05
Use Case Name	Item distribution and record keeping (Includes UC-05a: Save record offline)
Actor(s)	Collector
Preconditions	Collector is logged in. Farmer is identified.
Postconditions – Success	Distributed item (e.g., fertilizer) is recorded against the farmer and saved locally.
Postconditions – Failure	Record is not saved.
Main Flow	1. Collector selects "Distribute Item". 2. System displays a list of available inventory (e.g., Fertilizer Bags). 3. Collector enters the quantity given to the farmer. 4. System calculates total cost (to be deducted later). 5. Collector taps "Confirm". 6. [UC-05a] System saves the distribution record locally.
Alternative Flow 1	At step 3: Exceeds limits. If the requested quantity exceeds the farmer's limit or truck inventory, system alerts the Collector.
Business Rules	BR-07: Item cost is fixed based on current inventory rates and cannot be overridden by the Collector.

2.3.4.6 UC - 06 – Issue Advance Payment and Record Keeping

Use Case ID	UC-06
Use Case Name	Issue Advance Payment and Record Keeping (Includes UC-06a: Save record offline)
Actor(s)	Collector
Preconditions	Collector is logged in and has a cash float. Farmer is successfully identified (via QR or manual). System has access to the farmer's accumulated collection data for the current month.
Postconditions – Success	Advance payment is successfully handed out, recorded against the farmer's profile, securely saved offline, and flagged for future deduction.
Postconditions – Failure	Advance payment is denied (e.g., exceeds allowable limit) or canceled; no record is saved.

Main Flow	- Collector selects the "Issue Advance" option for the identified farmer. - System retrieves and displays the farmer's accumulated leaf weight for the current period. - System calculates and displays the maximum allowable advance based on the recorded weight. - Collector enters the requested advance cash amount. - System validates that the requested amount is less than or equal to the maximum allowable limit. - Collector physically hands the cash to the farmer and taps "Confirm Payment". [UC-06a] System saves the advance transaction locally to the device's offline database with a timestamp. - System displays a success confirmation and updates the farmer's local ledger.
Alternative Flow 1	At step 5: Amount exceeds limit. If the entered amount exceeds the calculated allowable limit, the system blocks the entry and displays an error message. The Collector must enter a lower amount or obtain a special managerial override code to proceed.
Alternative Flow 2	At step 2: Insufficient collection history. If the farmer has zero or near-zero recorded leaf weight for the current period, the system displays "Insufficient collection for advance." The Collector must cancel the transaction.
Business Rules	BR-08: The maximum allowable advance must be calculated as a set ratio/percentage of the estimated value of the farmer's currently accumulated but unpaid tea leaves (e.g., 50% of current accumulated value). BR-09: Offline advance records act as financial receipts; they cannot be modified locally once confirmed and must be strictly synced to the desktop app.

2.3.4.7 UC - 07 – Synchronize Collection Data

Use Case ID	UC-07
Use Case Name	Sync Collection Data (Extends UC-07a: LAN Sync, UC-07b: Cloud Sync)
Actor(s)	Collector
Preconditions	Collector has one or more 'Pending Sync' records. Device is connected to the office LAN (local mode) or Internet (cloud mode).
Postconditions – Success	All pending records transferred and marked 'Synced'. Sync count and timestamp displayed.

Postconditions – Failure	Sync fails/aborts. Local data remains safely stored on the device for a future attempt.
Main Flow	1. Collector taps "Sync Data" on the mobile app. 2. System detects network environment (Office LAN or External Cloud). System authenticates the collector with the central server. 3. System pushes all unsynced data batches to the server. 4. Server validates and ingests the data, returning a "Success" token for those records. 5. System pulls updated data (new farmers registered by office, new monthly rates, etc.) from the server. 6. System marks uploaded local records as "Synced" and updates the "Last Sync" timestamp.
Alternative Flow 1	At step 4/5: Network drops. If the connection drops mid-sync, the system aborts. Unconfirmed records remain marked as "Unsynced" to prevent data loss.
Alternative Flow 2	At step 5: Data conflict. If a record conflicts with server data, the server accepts the record but flags it for manual review by Office Staff.
Business Rules	BR-10: Sync operations are idempotent – repeating a sync must not create duplicate records. BR-11: System retries failed sync up to 3 times with exponential back-off before notifying the user.

2.3.4.8 UC - 08 – Log in to desktop app

Use Case ID	UC-09
Use Case Name	Log in to desktop app
Actor(s)	Owner / Office Staff
Preconditions	Application is installed/accessible; user has credentials.
Postconditions – Success	User accesses the admin dashboard.
Postconditions – Failure	Access denied.
Main Flow	1. User enters Email and Password on the login screen. 2. User clicks "Login". 3. System authenticates credentials with the master database. 4. System loads specific dashboard.
Alternative Flow 1	At step 3: Account locked. If too many failed attempts occur, system temporarily locks the account.

Alternative Flow 2	At step 3: Missing Fields. If required fields are empty, the system disables the "Save" button and highlights the missing fields in red.
Business Rules	BR-12: The system shall restrict access to authorized users only through secure authentication.

2.3.4.9 UC - 09 – Register Farmer

Use Case ID	UC-09
Use Case Name	Register New Farmer (Includes UC-09a: Generate or add QR code)
Actor(s)	Tea Collector (Mobile), Owner / Office Staff (Desktop)
Preconditions	Actor is logged into the system and has "Registration" permissions.
Postconditions – Success	Farmer profile is created in the database and a QR code is generated/assigned.
Postconditions – Failure	Farmer is not registered (e.g., due to local duplicate details or missing required fields).
Main Flow	1. Actor selects "Register New Farmer" from the main menu. 2. Actor enters farmer details (Name, Address, Contact Number, NIC, Route ID, Bank Details). 3. System validates that required fields are filled and checks for duplicates (against the local database if offline, or full database if online). 4. System creates a new farmer profile with a unique Farmer ID (UUID). 5. [UC-09a] System automatically generates a printable QR code mapping to the UUID, OR Actor scans an existing pre-printed generic QR card to link it. 6. Actor taps "Save". 7. System saves the profile (syncs immediately if online, saves locally if offline).
Alternative Flow 1	At step 3: Local Duplicate found. If a farmer with the same phone number or exact name already exists in the local database/cache, the system flags a warning. Actor can choose to cancel or "Proceed anyway" (creates a new profile).
Alternative Flow 2	At step 3: Missing Fields. If required fields are empty, the system disables the "Save" button and highlights the missing fields in red.

Business Rules	BR-13: A QR code/UUID must be strictly 1-to-1 with a farmer profile. BR-14: Mobile registrations done offline must be flagged as "Pending Sync" and pushed to the master database during the next connection. BR-15: Offline Sync Conflict Resolution: If a mobile registration is synced later and the central server detects a global duplicate (e.g., phone number already exists on another route), the server accepts the record provisionally but flags it and triggers an immediate dashboard alert for the Office Staff to manually review and merge/resolve.
-----------------------	--

2.3.4.10 UC - 10 – Manage farmer profiles

Use Case ID	UC-10
Use Case Name	Manage farmer profiles
Actor(s)	Owner / Office Staff
Preconditions	User is logged in with Admin/Staff permissions.
Postconditions – Success	Farmer profile is updated, deactivated, or viewed.
Postconditions – Failure	Updates are not saved.
Main Flow	1. User searches for a farmer by ID or Name. 2. System retrieves and displays the profile. 3. User edits details (e.g., address, bank account info). 4. User clicks "Save Updates". 5. System validates data and updates the master database.
Alternative Flow 1	At step 5: Invalid Bank Info. If bank format is incorrect, system highlights the field and prevents saving.
Business Rules	BR-16: Deactivating a farmer does not delete their historical records; it only prevents them from appearing in active daily routes.

2.3.4.11 UC - 11 – Manage Deductions

Use Case ID	UC-11
Use Case Name	Manage deductions (Extends Add loan, add manual deduction, Edit synced mobile deduction)
Actor(s)	Owner / Office Staff
Preconditions	User is logged in. Mobile data (advances/items) has been synced to the database, OR staff needs to schedule a new manual deduction.
Postconditions – Success	A new deduction is scheduled, or an existing synced deduction is successfully corrected and saved to the farmer's ledger.
Postconditions – Failure	Deduction updates are not saved (e.g., invalid amounts).
Main Flow	1. User navigates to the Deductions management screen for a selected farmer. 2. System displays all current deductions, including auto-synced records from the mobile app (Advances, Distributed Items) and existing manual records. 3. User chooses to either edit a synced record OR add a new manual deduction. 4. If Editing: User modifies the amount or details (e.g., fixing a Collector's entry error) and inputs a mandatory revision note. 5. If Adding: User selects the deduction type (Loan, Special Deduction), enters the amount, and sets the installment plan. 6. User clicks "Save". 7. System validates the entry and updates the farmer's ledger in the master database.
Alternative Flow 1	In step 5: The limit is exceeded. If a manually added loan amount is suspiciously high, the system will flag it and require recheck before saving it.
Alternative Flow 2	At step 4: Attempting to edit an approved month. If the user tries to edit a deduction that belongs to a month where payments have already been approved (UC-15), the system blocks the edit and displays a "Record Locked" error.
Business Rules	BR-17: Scheduled and verified deductions are automatically pulled by UC-13 during monthly calculations. BR-18: Any modification made by Office Staff to a record originally synced from the mobile app (e.g., changing an Advance amount) must permanently log an audit trail showing the original amount, the new amount, the timestamp of the change, and the Staff ID.

2.3.4.12 UC - 12 – Set monthly tea rate

Use Case ID	UC-12
Use Case Name	Set monthly tea rate
Actor(s)	Owner / Office Staff
Preconditions	User has Owner/Admin privileges.
Postconditions – Success	The official price per KG is saved for the specified month.
Postconditions – Failure	Rate is not saved.
Main Flow	1. User selects "Set Tea Rate". 2. User selects the applicable Month/Year. 3. User inputs the final per-KG rate. 4. User confirms and saves. 5. System applies this rate globally for the selected month's calculation.
Alternative Flow 1	At step 4: Overwriting existing rate. If a rate already exists and payments are processed, the system prevents modifications unless a full recalculation is authorized.
Business Rules	BR-19: Only one active standard rate can exist per calendar month unless specific farmer-level tiering is enabled.

2.3.4.13 UC - 13 – Calculate Monthly Payment

Use Case ID	UC-13
Use Case Name	Calculate Monthly Payment (Includes UC-13a: Apply all deductions)
Actor(s)	Owner / Office Staff
Preconditions	The monthly tea rate (UC-12) has been set. All mobile devices have synced their end-of-month data.
Postconditions – Success	Net payable amounts are accurately calculated for all farmers, and the payment summary is ready for approval.
Postconditions – Failure	Calculation cannot proceed (e.g., missing tea rate or pending syncs alert).
Main Flow	1. Actor selects the target month and clicks "Run Calculations". 2. System verifies that the monthly tea rate is configured.

	- System aggregates the total accepted leaf weight for each individual farmer. - System calculates Gross Income (Total Weight × Monthly Rate). [UC-13a] System gathers all deductions for the month (Advances given, fertilizer costs, loan installments). - System subtracts total deductions from Gross Income to find the Net Payable. - System generates a preliminary Payment Summary Sheet for review. - System highlights any anomalies (e.g., negative balances).
Alternative Flow 1	At step 6: Negative Balance. If deductions exceed the gross income, the Net Payable becomes 0. The system automatically creates a carry-forward deduction record for the next month.
Alternative Flow 2	At step 2: Missing Rate. If the monthly rate is not set, system halts the process and prompts the actor to set the rate (UC-12) before continuing.
Business Rules	BR-20: Calculations must only include leaf records and advance records explicitly dated within the selected calendar month.

2.3.4.14 UC - 14 – View payment summary

Use Case ID	UC-14
Use Case Name	View payment summary
Actor(s)	Owner / Office Staff
Preconditions	UC-13 (Calculations) has been run for the month.
Postconditions – Success	System displays a comprehensive, filterable list of all farmer payments, gross incomes, deductions, and net pay.
Postconditions – Failure	Summary does not load.
Main Flow	- User navigates to "Payment Summaries". - System retrieves calculation results for the current period. - System displays data in a tabulated grid. - User filters by Route or Collector to verify accuracy.
Business Rules	BR-21: Payment summary is read-only. Any required changes must be done via adjusting weights (UC-04) or deductions (UC-11) and re-running calculations (UC-13).

2.3.4.15 UC - 15 – Approve Payment Run

Use Case ID	UC-15
Use Case Name	Approve Payment Run
Actor(s)	Owner
Preconditions	UC-14 is reviewed and user has Owner privileges.
Postconditions – Success	Payment records locked as 'Approved'. Audit log updated. Bank transfer process (UC-12) triggered automatically.
Postconditions – Failure	Approval fails.
Main Flow	1. Owner clicks "Approve Payments" on the summary screen. 2. System prompts for secondary confirmation or admin PIN. 3. Owner enters confirmation. 4. System locks the month's records to prevent further edits.
Business Rules	BR-22: Once approved, no historical data (weights, rates, deductions) for that specific month can be altered.

2.3.4.16 UC - 16 – Process bank transfer

Use Case ID	UC-16
Use Case Name	Process bank transfer (Interacts with Bank API)
Actor(s)	Owner
Preconditions	Payment run is Approved (UC-15).
Postconditions – Success	API call to the bank is successful, and funds are scheduled for dispersal.
Postconditions – Failure	Bank API rejects the file/request.
Main Flow	1. Owner clicks "Process Bank Transfers". 2. System compiles net pay data and farmer bank details into the required Bank API format (or generates a bulk upload CSV/XML). 3. System transmits the payload via the secure Bank API. 4. System receives confirmation tokens from the bank. 5. System updates payment status to "Processing".
Alternative Flow 1	At step 4: Invalid accounts. Bank API returns errors for specific accounts (e.g., closed account). System flags these specific farmers for manual cash payment.
Business Rules	BR-23: All payloads must be encrypted and logged in an audit trail.

2.3.4.17 UC - 17 – Send SMS notification

Use Case ID	UC-17
Use Case Name	Send SMS notification (Interacts with SMS API)
Actor(s)	Owner / Office Staff
Preconditions	Payments are processed, or important alerts need sending.
Postconditions – Success	SMS messages are delivered via third-party API.
Postconditions – Failure	SMS gateway fails to send.
Main Flow	1. System detects a completed payment transfer OR User drafts a manual broadcast. 2. System parses farmer phone numbers. 3. System sends payloads to the SMS Gateway API. 4. System logs delivery statuses.
Alternative Flow 1	At step 3: API Timeout. System retries up to 3 times before logging a failure.
Business Rules	BR-24: Automated payment SMS must include Total Weight, Rate, Total Deductions, and Net Paid. BR-25: SMS notifications shall be sent only after a bank transfer is confirmed successful.

2.3.4.18 UC - 18 – Generate reports

Use Case ID	UC-18
Use Case Name	Generate reports
Actor(s)	Owner / Office Staff
Preconditions	System contains collection and payment data.
Postconditions – Success	Report (PDF/Excel) is generated and downloaded.
Postconditions – Failure	Report fails to compile.
Main Flow	1. User navigates to "Reports". 2. User selects report type (e.g., Monthly Collection by Route, Deduction Totals). 3. User sets date range parameters. 4. System queries database and compiles the visual/tabular report. 5. User exports or prints.
Business Rules	BR-26: All generated reports must contain a timestamp of generation for auditing.

2.3.4.19 UC - 19 – Manage user accounts

Use Case ID	UC-19
Use Case Name	Manage user accounts
Actor(s)	Owner
Preconditions	User is logged in as Owner.
Postconditions – Success	Collector or Staff account is created/modified/disabled.
Postconditions – Failure	Account is not modified.
Main Flow	1. Owner views user list. 2. Owner selects "Add User". 3. Owner assigns Role (Collector vs Staff), Username, and default password. 4. System saves the new credentials to the authentication server.
Alternative Flow 1	At step 4: Weak password. System rejects passwords under 8 characters.
Business Rules	BR-27: Usernames must be unique across the entire system.

2.3.4.20 UC - 20 – Manage collection routes

Use Case ID	UC-20
Use Case Name	Manage collection routes
Actor(s)	Owner / Office Staff
Preconditions	User is logged in.
Postconditions – Success	Routes are created or modified, and farmers are reassigned.
Postconditions – Failure	Route is not saved.
Main Flow	1. User selects "Route Management" from the dashboard. 2. User clicks "Create New Route" or selects an existing route to edit. 3. User enters or updates the Route Name (e.g., "North Valley"). 4. User assigns an active Tea Collector to be responsible for this route. 5. System saves the route details to the master database.
Alternative Flow 1	At step 4: Collector reassignment. If the selected Collector is currently assigned to another active route, the system displays a warning prompt. The user can confirm to reassign the Collector to the new route or cancel.
Business Rules	BR-28: A farmer can only belong to one active collection route at a time. BR-29: A route must be successfully created here before it can populate the dropdown menu used during "Register New Farmer" (UC-09). BR-30: If a route is marked as "Deactivated", the system should mark all farmers assigned to that route so that they can be reassigned.

2.3.5 Activity Diagrams

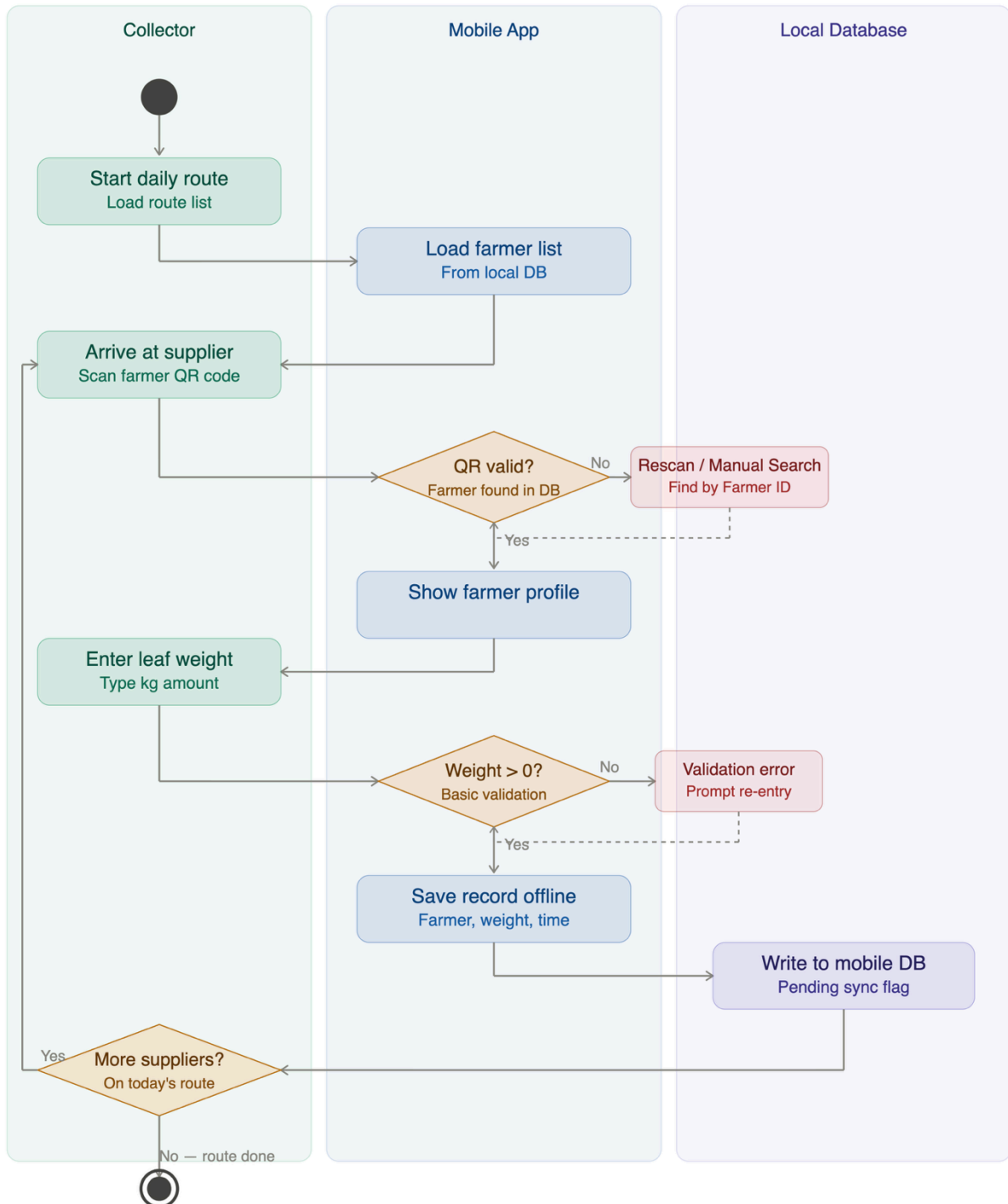
Activity diagrams are provided for the five most critical use cases.

2.3.5.1 Daily Tea Leaf Collection

This diagram maps out the core offline-first field operations performed by the Tea Collector. It illustrates the sequence of starting a daily route, scanning a farmer's QR code, validating and entering the leaf weight, and securely saving the transaction to the mobile device's local database without requiring an active internet connection.

Use Cases Covered:

- **UC-02:** View assigned route
- **UC-03:** Scan farmer QR code
- **UC-04:** Record leaf weight (Includes **UC-04a:** Save record offline)

Activity Diagram 1 — Daily tea leaf collection

2.3.5.2 Data Synchronization (Local and Cloud)

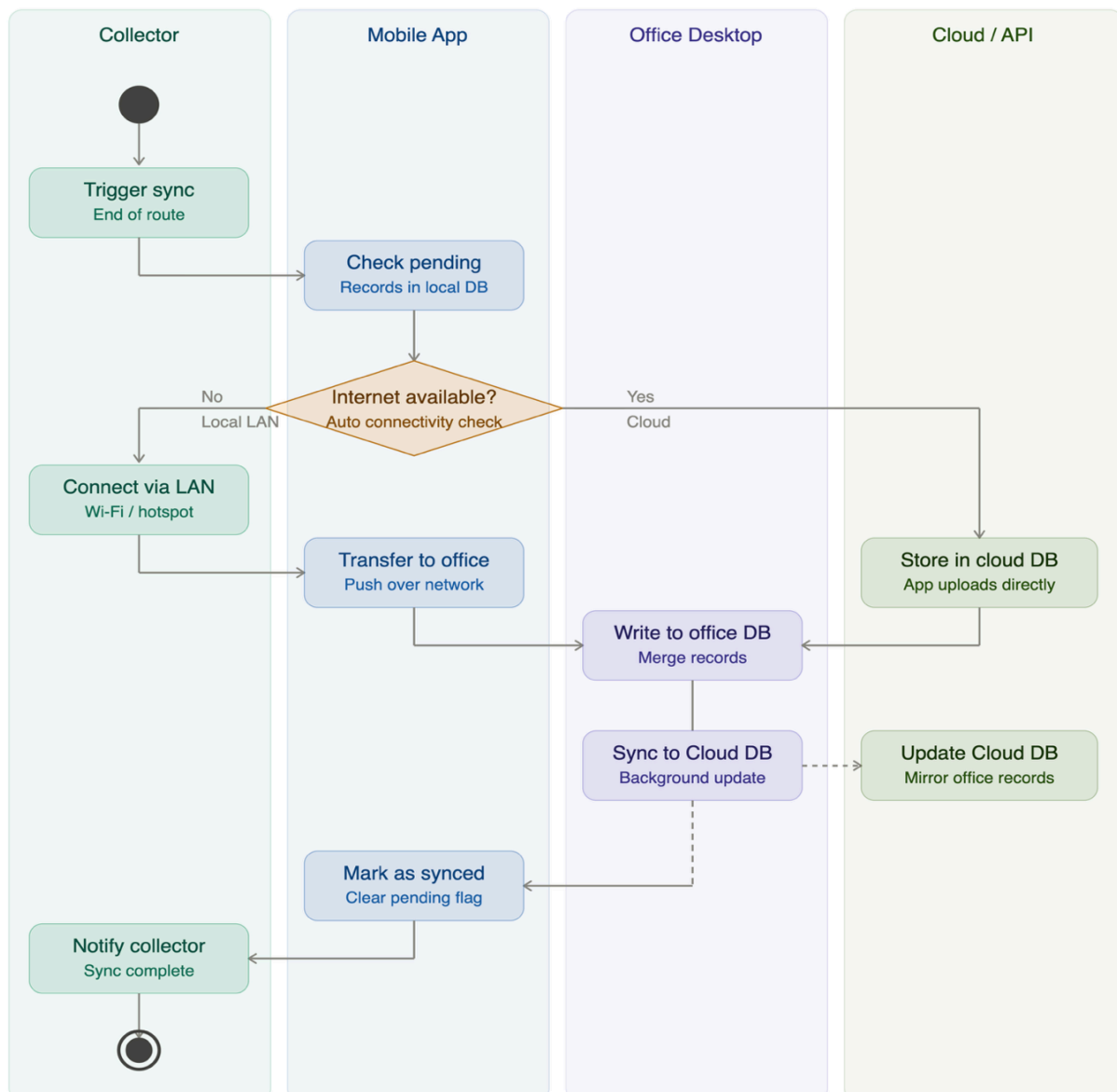
This diagram details the distributed data architecture of the system, showcasing the decision-making process for syncing mobile data. It demonstrates how the system detects connectivity and dynamically routes unsynced local data either

directly to the Cloud database (via mobile internet) or through the local office network (LAN) to ensure data integrity.

Use Cases Covered:

- **UC-07:** Sync collection data
- **UC-07a:** Local LAN sync (Extended)
- **UC-07b:** Cloud sync (Extended)

Activity Diagram 2 – Data synchronisation (local and cloud)



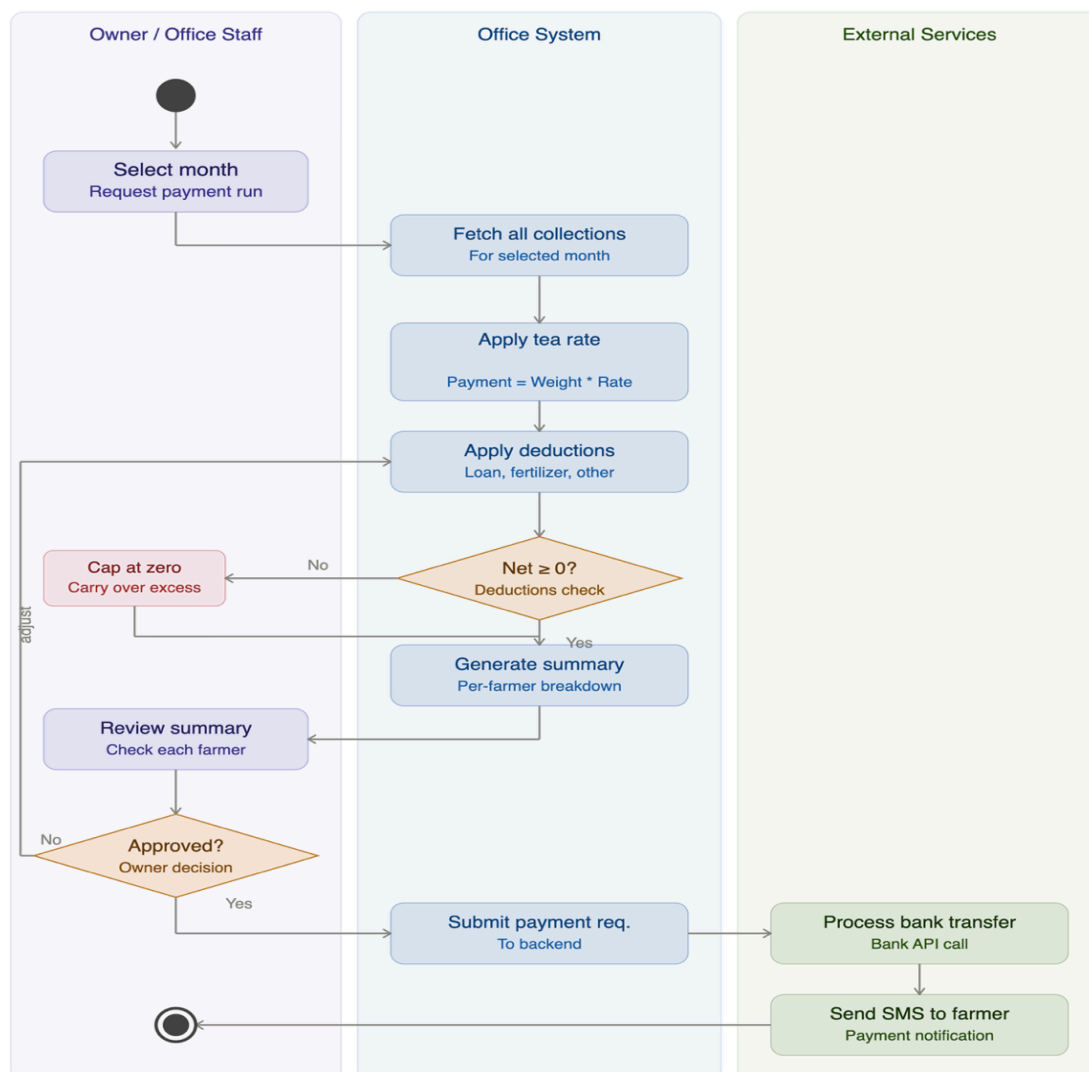
2.3.5.3 Monthly Payment Calculation and Processing

This diagram visualizes the heavy financial "engine" of the desktop application. It tracks the end-of-month workflow from aggregating leaf weights and applying the standard tea rate, to deducting advances/loans, handling negative net balances, securing owner approval, and finally communicating with external Bank and SMS APIs to execute the payments.

Use Cases Covered:

- **UC-13:** Calculate monthly payment (Includes **UC-13a:** Apply all deductions)
- **UC-14:** View payment summary
- **UC-15:** Approve payment run
- **UC-16:** Process bank transfer (API)
- **UC-17:** Send SMS notification (API)

Activity Diagram 3 — Monthly payment calculation and processing



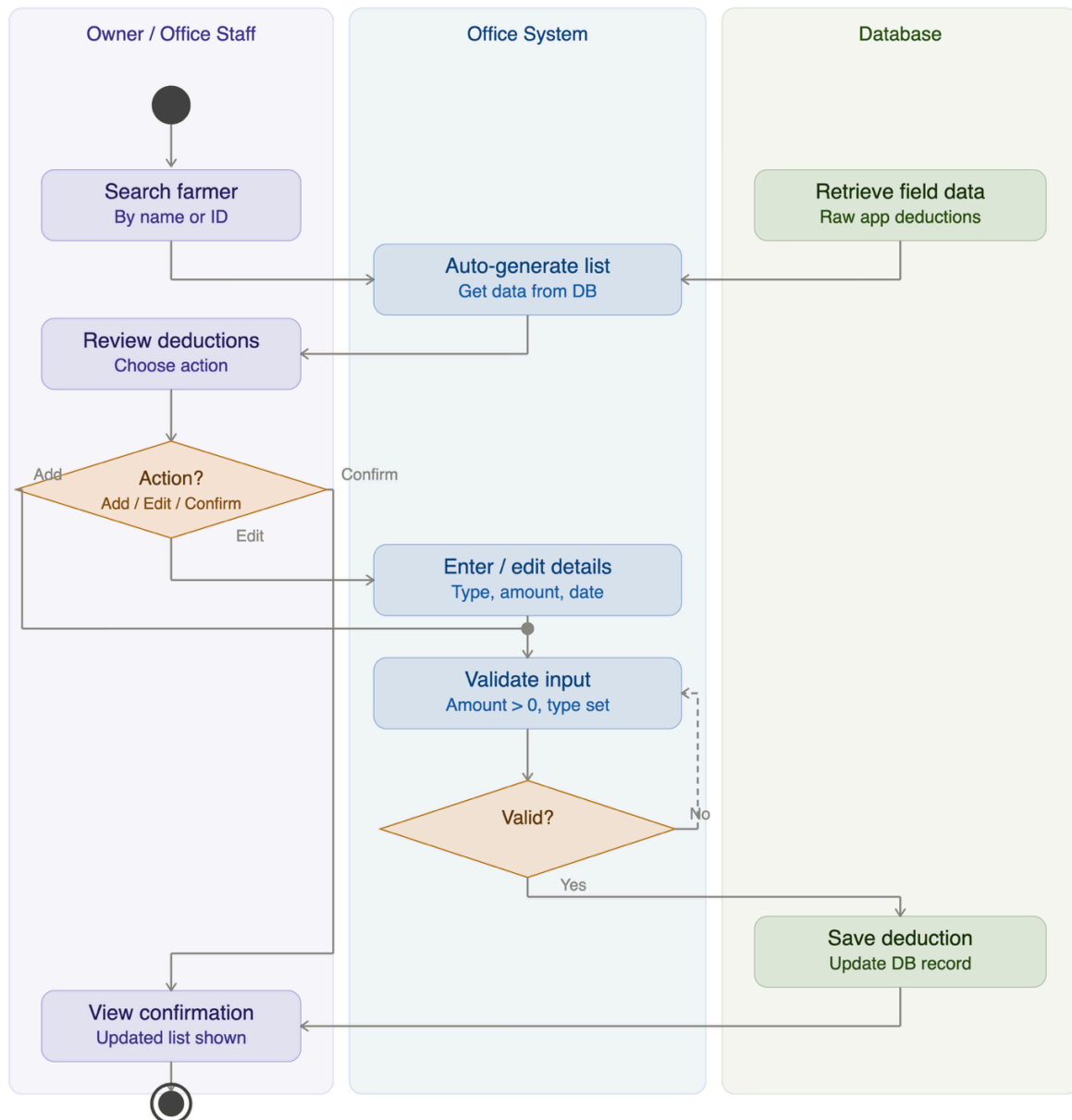
2.3.5.4 Deduction Management (Loans and Fertilizer)

This diagram explains the administrative workflow for managing farmer liabilities. It shows how the office system aggregates raw deduction data from the field (like distributed fertilizer or cash advances) and allows the office staff to review, manually add, edit, or confirm these deductions against a farmer's profile before the final monthly calculations are run.

Use Cases Covered:

- **UC-11:** Manage deductions
- **UC-11a / UC-11b / UC-11c:** Add loan, fertilizer, and other deductions (Extended)

Activity Diagram 4 — Deduction management (loans and fertilizer)



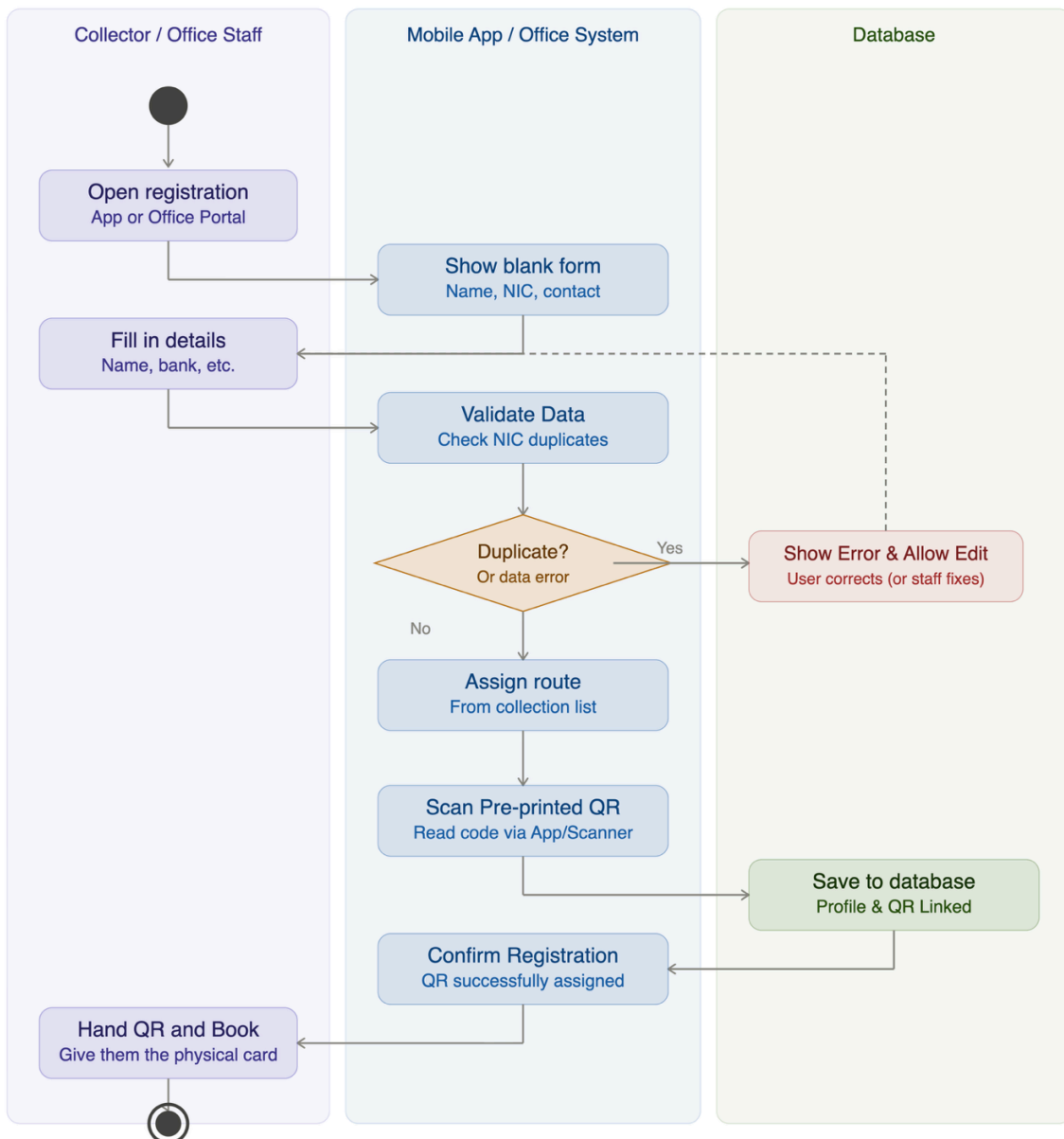
2.3.5.5 Farmer Registration and Profile Management

This diagram illustrates the onboarding process for new suppliers, highlighting a flow that can be executed by either the Collector (via mobile) or Office Staff (via desktop). It outlines the entry of biographical data, validation against duplicate records (like NIC checks), route assignment, and the physical linking of a pre-printed QR code card to the newly created digital profile.

Use Cases Covered:

- **UC-09:** Register new farmer
- **UC-09a:** Generate or add QR code (Included)
- **UC-21:** Receive QR code card (Physical external actor action)

Activity Diagram 5 — Farmer registration and profile management



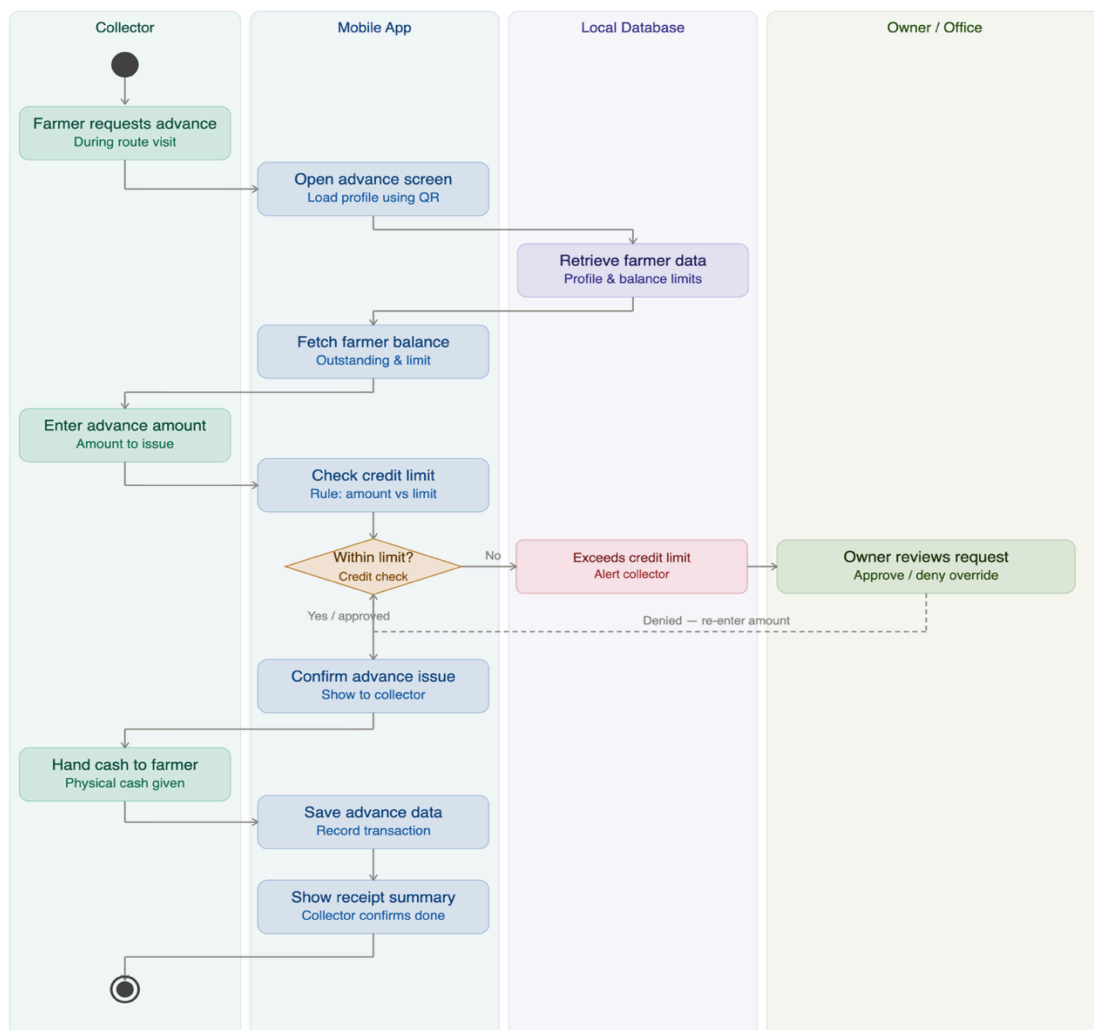
2.3.5.6 Advance Payment

This diagram illustrates the workflow for handing out assets—whether issuing cash advances or distributing physical items like fertilizer—to farmers in the field. It highlights the mobile app's ability to query the local database to verify a farmer's accumulated balance and credit limits before approving the transaction. It also maps out the exception-handling process for when a request exceeds the limit (requiring an override review by the Owner) and concludes with the physical handover and secure offline saving of the transaction.

Use Cases Covered:

- **UC-05:** Item distribution and record keeping
- **UC-05a:** Save record offline (Included)
- **UC-06:** Issue advance payment and record keeping

Activity Diagram 6 — Advance payment



2.4. Functional Requirements

Priority scale:

P1 – Critical (system cannot function without this).
 P2 – High (important for full operation).
 P3 – Medium (valuable enhancement).
 P4 – Low (deferred to later release).

2.4.1 User Authentication and Authorization

ID	FR-01
Title	User Login
Description	The system shall authenticate users with a username and password before granting access to any function.
Priority	P1 – Critical
Source	Client Interview #1; UC-01, UC-08
Verification	Test Case TC-FR-01 (functional test)

ID	FR-02
Title	Role-Based Access Control
Description	The system shall enforce role-based access control with four roles: Owner, Office Staff, Collector, and System Administrator. Each role shall have a distinct, non-overlapping permission set.
Priority	P1 – Critical
Source	Client Interview #1; UC-01, UC-08
Verification	Test Case TC-FR-02 (security test)

ID	FR-03
Title	Account Lockout
Description	The system shall lock a user account after five consecutive failed login attempts. A System Administrator must manually unlock the account.
Priority	P2 – High
Source	Security Review; NFR-30
Verification	Test Case TC-FR-03 (security test)

ID	FR-04
Title	Session Timeout
Description	The system shall automatically expire sessions after a configurable idle timeout period. The default idle timeout shall be 30 minutes.
Priority	P2 – High
Source	Security Review
Verification	Test Case TC-FR-04 (functional test)

ID	FR-05
Title	User Account Management
Description	Owner shall be able to create, update, deactivate, and reset passwords for user accounts.
Priority	P2 – High
Source	Client Interview #1; UC-19
Verification	Test Case TC-FR-05 (functional test)

ID	FR-06
Title	Audit Logging of Authentication Events
Description	All authentication events (login success, login failure, logout, account lock) shall be written to an immutable security audit log with user ID and timestamp.
Priority	P2 – High
Source	Security Review
Verification	Inspection TC-FR-06 (log review)

2.4.2 Farmer Management

ID	FR-07
Title	Farmer Registration
Description	The system shall allow Office Staff and collector to create a farmer profile with mandatory fields: full name, NIC number, address, primary phone number, route assignment, and bank account number.
Priority	P1 – Critical
Source	Client Interview #1; UC-09
Verification	Test Case TC-FR-07 (functional test)

ID	FR-08
Title	Unique NIC Validation
Description	The system shall validate that the NIC number is unique across all farmer records (including deactivated records) and reject duplicate entries.
Priority	P1 – Critical
Source	Document Analysis
Verification	Test Case TC-FR-08 (validation test)

ID	FR-09
Title	QR Code Generation
Description	The system shall automatically generate(add) and display a unique QR code for each registered farmer upon profile creation. The QR code shall encode the farmer UUID only.
Priority	P1 – Critical
Source	On-Site Observation; UC-03, UC-21
Verification	Test Case TC-FR-09 (functional test)

ID	FR-10
Title	QR Card Printing
Description	The system shall allow Office Staff to print individual farmer QR code cards from the farmer profile screen.
Priority	P2 – High
Source	On-Site Observation
Verification	Test Case TC-FR-10 (functional test)

ID	FR-11
Title	Farmer Profile Update
Description	The system shall allow Office Staff to update any editable farmer profile field. All changes shall be logged in the audit trail with the performing user's ID and timestamp.
Priority	P2 – High
Source	Client Interview #2; UC-10
Verification	Test Case TC-FR-11 (functional test)

ID	FR-12
Title	Farmer Deactivation
Description	The system shall allow soft-deletion (deactivation) of farmer profiles. Deactivated farmers shall not appear in active collection lists. A farmer with collection or payment history cannot be hard-deleted.
Priority	P2 – High
Source	Client Interview #2; UC-10
Verification	Test Case TC-FR-12 (functional test)

ID	FR-13
Title	Farmer Search and Filter
Description	The system shall provide search and filter of farmers by name, NIC, route assignment, or active status.
Priority	P3 – Medium
Source	Client Interview #2
Verification	Test Case TC-FR-13 (functional test)

2.4.3 Route Management

ID	FR-14
Title	Route Creation
Description	The system shall allow the Owner to create collection routes with a route name, route code, and an assigned Collector.
Priority	P1 – Critical
Source	On-Site Observation; UC-20
Verification	Test Case TC-FR-14 (functional test)

ID	FR-15
Title	Farmer-to-Route Assignment
Description	The system shall allow multiple farmers to be assigned to a single route. Farmers may be re-assigned between routes; the full assignment history shall be retained.
Priority	P1 – Critical
Source	On-Site Observation
Verification	Test Case TC-FR-15 (functional test)

ID	FR-16
Title	Route Schedule
Description	The system shall support daily route scheduling – specifying which route runs on which days of the week.
Priority	P3 – Medium
Source	Questionnaire

Verification	Test Case TC-FR-16 (functional test)
---------------------	--------------------------------------

2.4.4 Field Collection – Mobile Application

ID	FR-17
Title	Route Selection at Session Start
Description	The mobile app shall allow a Collector to select their assigned route before beginning a collection session.
Priority	P1 – Critical
Source	On-Site Observation; UC-02
Verification	Test Case TC-FR-17 (functional test)

ID	FR-18
Title	QR Code Scanning
Description	The mobile app shall scan a farmer QR code using the device camera to identify the farmer for a collection entry.
Priority	P1 – Critical
Source	On-Site Observation; UC-03
Verification	Test Case TC-FR-18 (functional test)

ID	FR-19
Title	Manual Farmer Fallback Selection
Description	The mobile app shall allow manual farmer selection by name or ID as a fallback when QR scanning is not possible.
Priority	P1 – Critical
Source	Questionnaire
Verification	Test Case TC-FR-19 (functional test)

ID	FR-20
Title	Offline Weight Entry and Local Storage
Description	The mobile app shall allow the Collector to enter tea leaf weight and store the record locally, regardless of network availability.
Priority	P1 – Critical

Source	On-Site Observation; UC-04
Verification	Test Case TC-FR-20 (offline test)

ID	FR-21
Title	Weight Entry Correction (Pre-Sync)
Description	The mobile app shall allow editing of a collection weight entry before synchronisation. The original value and the correcting user shall be retained in the record.
Priority	P2 – High
Source	Client Interview #2;
Verification	Test Case TC-FR-21 (functional test)

ID	FR-22
Title	Duplicate Entry Prevention
Description	The mobile app shall detect a potential duplicate entry for the same farmer on the same date and session, and require explicit Collector confirmation before saving.
Priority	P2 – High
Source	Document Analysis; BR-06
Verification	Test Case TC-FR-22 (validation test)

ID	FR-23
Title	Session Summary View
Description	The mobile app shall display a running summary of total kilograms collected per route for the current session, updated after each entry.
Priority	P3 – Medium
Source	Questionnaire
Verification	Test Case TC-FR-23 (functional test)

ID	FR-24
Title	Connectivity Status Indicator
Description	The mobile app shall display a visual indicator showing the current connectivity status: Offline, LAN Connected, or Internet Connected.
Priority	P3 – Medium
Source	Questionnaire
Verification	Test Case TC-FR-24 (functional test)

2.4.5 Data Synchronization

ID	FR-25
Title	Dual Sync Mode Support
Description	The system shall support two synchronisation modes: Local LAN Sync and Cloud Sync. The active mode shall be configurable by the Owner without requiring a code change.
Priority	P1 – Critical
Source	Client Interview #1; UC-07
Verification	Test Case TC-FR-25 (configuration test)

ID	FR-26
Title	Local LAN Synchronisation
Description	In Local LAN Sync mode, the mobile app shall transfer all pending collection records to the Office Local Database when connected to the office LAN.
Priority	P1 – Critical
Source	Client Interview #1; UC-07a
Verification	Test Case TC-FR-26 (integration test)

ID	FR-27
Title	Cloud Synchronisation
Description	In Cloud Sync mode, the mobile app shall upload all pending collection records to the Backend API Server over the Internet.
Priority	P1 – Critical
Source	Client Interview #1; UC-07b
Verification	Test Case TC-FR-27 (integration test)

ID	FR-28
Title	Sync Conflict Resolution
Description	The system shall detect data conflicts (same farmer, same date, different weight from two devices) and resolve using a last-write-wins strategy based on device timestamp. All conflicts shall be logged for owner review.
Priority	P2 – High
Source	Document Analysis; BR-06, BR-10
Verification	Inspection TC-FR-28 (conflict log review)

ID	FR-29
Title	Sync Retry with Back-Off
Description	The system shall retry failed synchronisation attempts up to three times using exponential back-off before notifying the user of failure.
Priority	P2 – High
Source	Technical Constraint; BR-11
Verification	Test Case TC-FR-29 (resilience test)

2.4.6 Tea Rate Management

ID	FR-30
Title	Monthly Tea Rate Setting
Description	The system shall allow the Owner or Office Staff to set the monthly tea purchase rate (LKR per kg) for each calendar month.
Priority	P1 – Critical
Source	Client Interview #1; UC-12
Verification	Test Case TC-FR-30 (functional test)

ID	FR-31
Title	Tea Rate History
Description	The system shall maintain a history of all monthly tea rates. Past rates for months with completed payment calculations shall not be editable.
Priority	P2 – High
Source	Document Analysis
Verification	Inspection TC-FR-31 (data integrity test)

2.4.7 Deduction Management

ID	FR-32
Title	Loan Deduction Recording
Description	The system shall allow Office Staff to record a loan issued to a farmer with: total loan amount, monthly installment amount, start month, and description.
Priority	P2 – High
Source	Client Interview #2; UC-11
Verification	Test Case TC-FR-32 (functional test)

ID	FR-33
Title	Automatic Loan Installment Deduction

Description	The system shall automatically apply the monthly loan installment deduction during payment calculation until the loan balance reaches zero.
Priority	P2 – High
Source	Client Interview #2 / UC-11a
Verification	Test Case TC-FR-33 (calculation test)

ID	FR-34
Title	Fertilizer Deduction Recording
Description	The system shall allow Office Staff to record fertilizer or input material supplies issued to a farmer as a one-time or recurring deduction.
Priority	P2 – High
Source	Client Interview #2 / UC-11b, UC05
Verification	Test Case TC-FR-34 (functional test)

ID	FR-35
Title	Custom Deduction Types
Description	The System Administrator shall be able to define additional custom deduction types beyond the built-in Loan and Fertilizer types.
Priority	P3 – Medium
Source	Client Interview #2; UC-11
Verification	Test Case TC-FR-35 (functional test)

ID	FR-36
Title	Negative Payment Alert
Description	The system shall alert Office Staff when a farmer's net payment would be negative after applying deductions, and prevent the payment run from proceeding to the approval stage until the issue is resolved.
Priority	P1 – Critical
Source	Document Analysis; UC-13 Alternative Flow 1
Verification	Test Case TC-FR-36 (validation test)

2.4.8 Monthly Payment Calculation

ID	FR-37
Title	Gross Payment Calculation
Description	The system shall calculate the monthly gross payment per farmer as: $\text{Gross Payment} = \text{Total kg Collected (for the month)} \times \text{Monthly Tea Rate}$, using fixed-point arithmetic.
Priority	P1 – Critical
Source	Client Interview #1/ UC-13
Verification	Test Case TC-FR-37 (calculation test)

ID	FR-38
Title	Net Payment Calculation
Description	The system shall calculate the monthly net payment per farmer as: $\text{Net Payment} = \text{Gross Payment} - \text{Sum of all applicable deductions for the month}$.
Priority	P1 – Critical
Source	Client Interview #1/ UC-13a
Verification	Test Case TC-FR-38 (calculation test)

ID	FR-39
Title	Payment Preview
Description	The system shall display a payment preview table showing for each farmer: total kg, gross payment (LKR), itemised deductions (LKR), and net payment (LKR), before the run is sent for approval.
Priority	P1 – Critical
Source	Client Interview #1/ UC-14
Verification	Test Case TC-FR-39 (functional test)

2.4.9 Payment Approval Workflow

ID	FR-40
Title	Owner Payment Approval
Description	The Owner shall be able to review the monthly payment preview and approve or reject the payment run. A rejected run must include a rejection reason.
Priority	P1 – Critical
Source	Client Interview #1; UC-15
Verification	Test Case TC-FR-40 (functional test)

ID	FR-41
Title	Payment Record Immutability on Approval
Description	Once a payment run is approved by the Owner, all associated payment records shall be locked and immutable. No further edits shall be permitted by any user.
Priority	P1 – Critical
Source	Document Analysis; BR-22
Verification	Inspection TC-FR-41 (security test)

ID	FR-42
Title	Approval Audit Record
Description	The system shall record the approving Owner's identity, approval timestamp, month, total approved amount, and any remarks against each approved payment run in the audit log.
Priority	P2 – High
Source	Document Analysis; BR-22
Verification	Inspection TC-FR-42 (log review)

2.4.10 Bank Transfer

ID	FR-43
Title	Bank Transfer Submission
Description	The system shall submit approved payments to the Bank Transfer API within 5 minutes of Owner approval.
Priority	P1 – Critical
Source	Client Interview #1; UC-16, BR-23
Verification	Test Case TC-FR-43 (integration test)

ID	FR-44
Title	Transfer Status Retrieval
Description	The system shall retrieve and display the bank transfer status response (Success / Pending / Failed) from the Bank Transfer API for each payment.
Priority	P1 – Critical
Source	Client Interview #1;
Verification	Test Case TC-FR-44 (integration test)

ID	FR-45
Title	Transfer Audit Log
Description	All bank transfer API requests, responses, HTTP status codes, and timestamps shall be stored in the audit log.
Priority	P1 – Critical
Source	Security Review; BR-23
Verification	Inspection TC-FR-45 (log review)

ID	FR-46
Title	Manual Transfer Re-submission
Description	The system shall allow the Owner or Office Staff to manually re-submit a failed bank transfer after the failure cause has been resolved.
Priority	P2 – High
Source	Client Interview #2

Verification	Test Case TC-FR-46 (functional test)
---------------------	--------------------------------------

2.4.11 SMS Notification

ID	FR-47
Title	Farmer SMS on Successful Payment
Description	The system shall send an SMS to each farmer upon confirmation of a successful bank transfer, stating the net payment amount in LKR and the payment month.
Priority	P2 – High
Source	Client Interview #1; UC-17
Verification	Test Case TC-FR-47 (integration test)

ID	FR-48
Title	SMS Sent Only After Transfer Confirmation
Description	SMS notifications shall be sent only after a bank transfer is confirmed successful – not at the payment calculation or approval stage.
Priority	P2 – High
Source	Business Rule BR-25
Verification	Test Case TC-FR-48 (functional test)

ID	FR-49
Title	Multi-Language SMS
Description	The system shall support SMS message delivery in Sinhala, Tamil, and English. The farmer's preferred language shall be set on their profile and used for SMS content.
Priority	P3 – Medium
Source	Client Interview #3; Questionnaire
Verification	Test Case TC-FR-49 (localisation test)

2.4.12 Reporting

ID	FR-50
Title	Daily Collection Report
Description	The system shall generate a Daily Collection Report showing total kg per farmer and per route for a selected date.
Priority	P2 – High
Source	Client Interview #2; UC-18
Verification	Test Case TC-FR-50 (functional test)

ID	FR-51
Title	Monthly Payment Report
Description	The system shall generate a Monthly Payment Report showing gross payment, itemised deductions, and net payment per farmer for a selected month.
Priority	P2 – High
Source	Client Interview #2; UC-18
Verification	Test Case TC-FR-51 (functional test)

ID	FR-52
Title	Report Export
Description	All system reports shall be exportable in PDF and CSV formats.
Priority	P3 – Medium
Source	Client Interview #2
Verification	Test Case TC-FR-52 (functional test)

ID	FR-53
Title	Farmer Payment Receipt
Description	The system shall generate and allow printing of an individual payment receipt for each farmer upon successful payment processing.
Priority	P3 – Medium
Source	Client Interview #2

Verification

Test Case TC-FR-53 (functional test)

2.5. Non-Functional Requirements

2.5.1 Performance

ID	Requirement	Measure
NFR-01	The mobile app shall respond to QR scan and save a weight entry within 1 second under normal conditions.	< 1 second response time
NFR-02	The office desktop app shall load a monthly payment report for up to 500 farmers within 5 seconds.	< 5 seconds load time
NFR-03	A full-day route sync (up to 500 records) shall complete within 30 seconds on a 100 Mbps LAN.	< 30 seconds transfer
NFR-04	Monthly payment calculation for up to 1,000 farmers shall complete within 10 seconds.	< 10 seconds compute
NFR-05	The system shall support a minimum of 10 concurrent office desktop sessions without degradation.	10 concurrent users

2.5.2 Security

ID	Requirement	Verification
NFR-06	All network communication shall use TLS 1.2 or higher.	Security audit
NFR-07	All farmer personal and financial data shall be encrypted at rest using AES-256 or equivalent.	Security audit
NFR-08	Authentication tokens (JWT) shall be signed with RS256 and expire within the configured session timeout.	Security test
NFR-09	Bank API credentials and SMS API keys shall be stored in a secrets manager and never committed to source control.	Code review / inspection
NFR-10	The system shall maintain an immutable audit log of all actions affecting farmer profiles, deductions, and payment records.	Inspection
NFR-11	Approved payment records shall be cryptographically locked to detect unauthorised modification.	Security test

2.5.3 Usability

ID	Requirement	Verification
NFR-1 2	The mobile app shall use minimum touch targets of 48×48 dp (Android) / 44×44 pt (iOS) for all interactive elements.	UI inspection
NFR-1 3	A newly onboarded Collector shall be able to complete a full collection session with no assistance after a maximum of 30 minutes of training.	Usability test
NFR-1 4	The mobile app shall support display in Sinhala, Tamil, and English.	Localisation test
NFR-1 5	All error messages shall be written in plain language with clear corrective action guidance.	Heuristic review
NFR-1 6	The office desktop app shall follow standard Windows UI conventions (keyboard shortcuts, tab order, context menus).	Heuristic review

2.5.4 Reliability

ID	Requirement	Verification
NFR-1 7	The mobile app shall guarantee zero data loss for locally stored collection records, even on app crash or device shutdown.	Failure injection test
NFR-1 8	All database writes shall use ACID-compliant transactions to prevent partial data corruption.	Code inspection
NFR-1 9	Synchronisation operations shall be idempotent – repeating a sync shall not create duplicate records.	Idempotency test
NFR-2 0	Payment calculation results shall be deterministic – identical inputs shall always produce identical outputs.	Unit test
NFR-2 1	External API calls (bank, SMS) shall implement automatic retry with exponential back-off (max 3 retries).	Integration test

2.5.5 Scalability

ID	Requirement	Verification
NFR-2 2	The system shall support a minimum of 2,000 registered farmer records without performance degradation.	Load test

ID	Requirement	Verification
NFR-2 3	The system shall support a minimum of 5 concurrent collector devices synchronising simultaneously.	Load test
NFR-2 4	The backend API server architecture shall allow horizontal scaling (additional instances behind a load balancer) without code changes.	Architecture review

2.5.6 Maintainability

ID	Requirement	Verification
NFR-2 5	The codebase shall follow documented coding standards with inline comments for all business logic functions.	Code review
NFR-2 6	The system shall support database schema migrations with rollback capability using a migration framework.	Migration test
NFR-2 7	All configurable parameters (rate limits, session timeout, SMS templates) shall be manageable via admin UI without a code deployment.	Functional test
NFR-2 8	Unit test coverage for payment calculation and deduction logic shall be a minimum of 90%.	Coverage report
NFR-2 9	The system shall produce structured application logs in JSON format compatible with standard log aggregation tools.	Log inspection

2.6. Alternative Solutions & Feasibility Study

2.6.1 Alternative Solutions Analysed

Alternative 1 – Off-the-Shelf Agricultural Management Software

Several commercial agricultural management platforms (e.g., FarmERP, AgriSoft) are available that cover crop collection and payment features.

Aspect	Assessment
Description	Purchase or subscribe to an existing multi-purpose agricultural ERP system and configure it for tea collection.
Advantages	Rapid deployment. Vendor-supported. Known reliability. Reduced development risk.
Disadvantages	• Generic platforms lack Sri Lankan tea industry-specific workflows (e.g., route-based lorry collection, LKR payment integration, local bank API). • High licensing cost (USD 200–500/month). • Offline-first mobile collection is not a standard feature. • Customization is expensive and constrained by vendor APIs. • Sinhala/Tamil localization is typically not included.
Cost Estimate	LKR 600,000–1,200,000 per year (licensing). Additional customization: LKR 400,000–800,000 one-time.
Verdict	Not recommended. The system cannot be adequately configured for the specific operational model without prohibitive customisation cost.

Alternative 2 – Spreadsheet-Based Enhancement (Enhanced Legacy Approach)

Enhance the current process by replacing the legacy desktop application with a shared Microsoft Excel workbook (OneDrive or SharePoint) while keeping paper-based field collection.

Aspect	Assessment
Description	Office staff maintain a shared Excel workbook with payment formula templates. Collectors continue using paper route books; data is entered in the office.
Advantages	Very low cost (uses existing Microsoft 365 subscription). No development required. Immediately available.
Disadvantages	- Does not resolve the root cause pain points: manual transcription errors persist (P-01). - No mobile field data entry (P-01). - No QR identification (P-07). - No automated bank transfer (P-03). - No SMS notification (P-05). - Concurrent editing conflicts in shared spreadsheets. - No audit trail (P-04). - Not scalable beyond ~100 farmers without significant Excel complexity. - Poor data security.
Cost Estimate	- LKR 0 development cost. - Ongoing manual labor cost.
Verdict	Not recommended. Fails to address the core operational and accuracy problems identified in Section 3.2.2.

Alternative 3 – Recommended: Custom Multi-Platform System (TeaRoutePay)

Develop a purpose-built system comprising an offline-first mobile application, an office desktop application, and a backend API server tailored exactly to the tea collection business workflow.

Aspect	Assessment
Description	Custom cross-platform mobile app (Flutter), Windows desktop app (Flutter), and a Linux-hosted REST API backend.
Advantages	- Precisely addresses all seven pain points (P-01 to P-07). - Offline-first mobile design. - QR-based farmer identification. - Automated payment calculation and bank transfer. - SMS notification. Sinhala/Tamil localisation. - Full audit trail. - LKR fixed-point arithmetic.

Aspect	Assessment
Disadvantages	- Higher upfront development effort and cost compared to alternatives - Requires internal or contracted maintenance post-delivery.
Cost Estimate	- LKR 1,500,000–2,200,000 development (one-time). - LKR 120,000–180,000/year maintenance.
Verdict	Recommended. The only alternative that fully resolves all identified pain points within the operational and technical constraints.

2.6.2 Feasibility Study

2.6.2.1 Technical Feasibility

The system is designed as a microservices architecture handling financial transactions of approximately LKR 10 million per month. All required technologies are mature, well-documented, and appropriate for this scale:

1. Cross-platform development (Flutter) — A single Flutter codebase covers both the collector mobile app (Android, offline-first) and the office desktop application (Windows), reducing development effort while maintaining a consistent user experience across platforms.
2. Backend microservices (Java Spring Boot) — Spring Boot is the industry-standard framework for enterprise-grade, security-critical business APIs. Each microservice (Collection, Farmer, Payment, Deduction, Notification, Auth) is an independent Spring Boot application. Spring Boot's strong typing, transactional integrity via `@Transactional`, and mature ecosystem make it the correct choice for a system processing financial data at this scale.
3. API Gateway and service discovery (Spring Cloud Gateway + Eureka) — Spring Cloud Gateway handles all inbound routing, rate limiting, and authentication enforcement at the network edge. Eureka provides service discovery so microservices locate each other dynamically, consistent with the Spring Cloud ecosystem.
4. Event streaming and audit trail (Apache Kafka) — All financial events — payment calculations, deduction changes, transfer approvals — are published to Kafka topics. Kafka retains an immutable, ordered log of all events, providing a full audit trail for dispute resolution and compliance. This is a critical requirement for a system handling LKR 10 million per month.

5. Mobile local storage (SQLite via Drift + Hive) — SQLite managed through the Drift package provides typed, compile-safe offline SQL queries on the collector's device. Hive provides lightweight key-value storage for app state. Both support the offline-first collection workflow and pending-sync flagging.
6. Cloud database (PostgreSQL + Flyway) — PostgreSQL serves as the production-grade relational database, with each microservice maintaining its own schema. Flyway manages all schema migrations with version control, ensuring safe, repeatable database changes across environments.
7. Caching and session management (Redis) — Redis supports JWT token blacklisting (for secure logout), idempotency key storage (preventing duplicate payment processing on network retry), and inter-service rate limiting.
8. Containerization and orchestration (Docker + Kubernetes on AWS EKS) — Each microservice is containerized with Docker. Kubernetes manages deployment, scaling, and recovery in production. AWS Elastic Kubernetes Service (EKS) in the Singapore region (ap-southeast-1) provides the lowest latency to Sri Lanka with high availability guarantees.
9. QR code generation and scanning — ZXing handles server-side QR code generation for farmer identity cards. The `mobile_scanner` Flutter package handles scanning on the collector's device using the native camera API on Android.
10. Security (Spring Security + JWT + RBAC + AES-256 + TLS) — Spring Security provides enterprise-grade authentication and role-based access control across all services. JWT tokens with short expiry and Redis-backed blacklisting handle session management. AES-256 encrypts sensitive data at rest. TLS 1.3 encrypts all data in transit. AWS Secrets Manager stores all credentials and API keys — none are stored in source code or environment files. Spring Data Envers automatically maintains a tamper-evident audit log of every change to payment, deduction, and farmer records, recording the user, timestamp, and before/after values.
11. Reporting (JasperReports) — JasperReports is a Java-native reporting engine that integrates directly with Spring Boot, providing structured PDF generation for monthly payment reports, collection summaries, and deduction statements.

12. SMS notifications (Dialog Axiata SMS Gateway) — Dialog Axiata provides a documented REST API for programmatic SMS delivery to Sri Lankan mobile numbers, used for payment notifications to farmers.
13. Bank transfer (LankaPay / Bank Transfer API) — LankaPay is Sri Lanka's national payment infrastructure, providing a documented API for inter-bank transfers. This handles monthly payment disbursements directly to farmer bank accounts.
14. Monitoring and observability (Spring Actuator + Prometheus + Grafana) — Spring Boot Actuator exposes health and metrics endpoints for all services. Prometheus collects metrics and Grafana provides real-time dashboards. Alerts are configured for payment service downtime and failed bank transfer events.

Conclusion: Technically feasible. All selected technologies are proven in production financial systems, actively maintained, and well-documented. The stack is internally consistent — Spring Cloud, Spring Security, Spring Data, and Spring Boot integrate natively, reducing integration risk. No unproven or experimental technologies are required.

2.6.2.2 Economic Feasibility

Cost Item	Estimate (LKR)	Notes
Software development	1,500,000 – 2,200,000	One-time, based on team size and timeline
Server hosting (cloud, 1 year)	60,000 – 120,000	Optional; not needed in LAN-only mode
SMS Gateway setup + first year	30,000 – 60,000	Depends on message volume
Bank API integration	0 – 50,000	Typically no fee; developer effort only
Annual maintenance	120,000 – 180,000	Post-delivery bug fixes and updates
Savings: staff time (manual re-entry)	~900,000/year	Estimated 3 staff hours/day × 300 days
Savings: payment error corrections	~200,000/year	Estimated dispute resolution time
Break-even point	~2.5 years	Development cost ÷ annual savings

Conclusion: Economically feasible. Break-even within approximately 2.5 years with ongoing annual savings thereafter.

2.6.2.3 Operational Feasibility

The system is designed for low-technical-proficiency users, ensuring that field and office operations can be executed with minimal training:

1. **Collector Mobile App:** Built with large touch targets, a localized Sinhala/Tamil user interface, and a 30-minute training target. Core tasks are streamlined into simple, offline-capable modules to minimize typing:
 - **Weight Collection:** A fast, 3-step flow (scan → weigh → save).
 - **Field Registration:** Guided, on-the-spot profile creation for new farmers directly from the field.
 - **Advances & Distribution:** Quick-select menus to instantly log financial deductions (e.g., cash advances) and record item distributions (e.g., fertilizer or tea packets) to a farmer's account.
2. **Owner / Office Staff Desktop Dashboard:** A Windows-style desktop application for managing farmers, inventory, deductions, and payments. It automatically syncs field data and provides a simple dashboard with one-click approvals, requiring only basic computer skills.

Conclusion: Operationally feasible. The user interface design, automated offline syncing, and localized training plan specifically address the low-technical-proficiency user base, ensuring that advanced tasks like field registration and inventory distribution remain intuitive.

2.6.2.4 Schedule Feasibility

Phase	Duration	Key Deliverables
Phase 1 – Requirements & Design	8 weeks	This SRS, system architecture, UI wireframes, database schema
Phase 2 – Mobile App Development (Parallel)	8 weeks	Offline collection, QR scan, sync engine (Android + iOS)
Phase 3 – Desktop App Development (Parallel)	8 weeks	Farmer mgmt, deductions, payment calculation, reports
Phase 4 – Backend & Integrations (Parallel)	10 weeks	REST API, bank API, SMS gateway, cloud sync
Phase 5 – System Testing & UAT	4 weeks	Integration tests, performance tests, user acceptance
Phase 6 – Deployment & Training	2 weeks	Production deployment, staff training, pilot route

Phase	Duration	Key Deliverables
Total	24 weeks (6 months)	Full system go-live

2.6.3 Justified Recommendation

RECOMMENDATION: Alternative – Custom Multi-Platform System (TeaRoutePay) is the recommended solution. It is the only alternative that fully and directly addresses all seven identified pain points within the operational, technical, economic, and schedule constraints of the business. The higher upfront development cost is justified by a break-even point of approximately 2.5 years, elimination of manual transcription errors, accurate automated payments, real-time farmer notifications, and a full audit trail that protects the business from financial disputes.

Chapter 3: System Design

This chapter presents the complete Software Design Specification (SDS) for the TeaRoutePay system. The SDS defines the system architecture, detailed class and sequence designs, database model, UI wireframes, security design, and interface specifications.

3.1. INTRODUCTION

3.1.1 Purpose

The purpose of this Software Design Specification (SDS) is to provide a complete, unambiguous, and technically rigorous description of the design of the TeaRoutePay system, derived directly from the approved Software Requirements Specification (SRS, document TCPS-SRS-2026-v1.0). Where the SRS establishes what the system is required to do, the present document establishes how those requirements are to be realised in software.

This SDS is intended to serve as the authoritative engineering reference that bridges the analysis phase and the implementation phase of the project. It is written to be sufficient for the following purposes:

- to allow the development team to implement each subsystem without re-deriving design intent from the requirements;
- to allow reviewers, supervisors, and panel members to verify that every functional requirement (FR-01 to FR-53) and every non-functional requirement (NFR-01 to NFR-29) of the SRS is satisfied by a concrete, traceable design decision;
- to allow future maintainers, including the SENG 34213 development team, to understand the rationale behind each architectural and detailed-design choice; and
- to establish the verification baseline against which the constructed system will be tested.

A central design principle adopted throughout this document is explicit traceability. Each significant design decision is mapped back to the specific SRS requirement or requirements that motivate it. This traceability is expressed both inline — through Requirements addressed annotations — and consolidated in the requirements-traceability matrices presented in Sections 4 and 8.

3.1.2 Scope

TeaRoutePay is a distributed, offline-first software platform that digitises the complete operational workflow of a tea collection business, extending from field-level leaf collection through to monthly farmer payment disbursement and confirmation. The design described in this document encompasses the following system boundary:

- **Collector Mobile Application** — a Flutter-based Android application operated by tea collectors in the field. It captures collection data, performs QR-based farmer identification, records field deductions and advances, and operates with complete functional independence from any network connection. It is the primary realisation of the offline-first constraint (NFR-17, FR-20).
- **Office Desktop / Web Application** — a React-based client operated by the Owner and Office Staff for farmer administration, deduction management, tea-rate configuration, payment calculation, payment approval, and reporting.
- **Backend Services** — a suite of independently deployable Spring Boot microservices (Authentication, Farmer, Route, Collection, Deduction, Payment, Bank Integration, SMS, Reporting, Audit, and Synchronisation) fronted by a Spring Cloud API Gateway. These services constitute the central source of truth and the integration surface for external systems.
- **Data Stores** — an embedded SQLite database on each mobile device (the local-first store), a PostgreSQL instance backing the office environment, and a central PostgreSQL cluster in the cloud environment.
- **External Integrations** — the Bank Transfer API (for disbursement of approved payments) and the SMS Gateway (for payment confirmation to farmers).

The following are explicitly outside the scope of this design: the internal implementation of third-party Bank and SMS providers; physical procurement of mobile and office hardware; and business-process change management. These are treated only at their integration boundaries.

3.1.3 Overview of the Document

The remainder of this document is organised as follows. Section 2 (System Overview) describes the system at a conceptual level and establishes its relationship to the SRS, with particular attention to the hybrid local-first architecture that underpins rural offline operation. Section 3 (Architectural Design) justifies the selected microservices architecture, presents the high-level architecture, component, and deployment views, and specifies the technology stack. Section 4 (Detailed Design) presents the domain class model, the realisation of all twenty principal use cases through sequence diagrams, the state models for the system's stateful entities, and the complete database design including the data dictionary, data-integrity rules, and a requirements-traceability matrix. Section 5 (UI Design) documents the user-experience decisions and the navigation model. Section 6 (Interface Design) specifies the external and internal interfaces. Section 7 (Security Design) describes the authentication, authorisation, data-protection, auditability, and OWASP-aligned mitigation strategies. Section 8 (Non-Functional Design Decisions) consolidates the mapping of every non-functional requirement to the specific design mechanism that satisfies it. Architectural Decision Records are provided as an appendix.

Section	Title
1	Introduction
2	System Overview
3	Architectural Design
4	Detailed Design
5	User Interface Design
6	Interface Design
7	Security Design
8	Non-Functional Design Decisions
Appendix A	Architectural Decision Records

3.2. SYSTEM OVERVIEW

3.2.1 System Description and the Hybrid Local-First Architecture

TeaRoutePay is conceived as an offline-first, hybrid local-first distributed system. This characterisation is the single most consequential design decision in the document, and it is a direct response to the operating environment established by the SRS: collection is performed by lorry-borne collectors along rural routes where network connectivity is intermittent or absent, yet the business simultaneously requires a single, authoritative financial record of truth at the office and in the cloud.

The hybrid local-first model resolves this tension by maintaining authority at two tiers and reconciling them through controlled synchronisation:

- **The local tier** is realised by an embedded SQLite database on each collector's device. Every field transaction — a weight entry, a fertilizer distribution, a cash advance, or a field farmer registration — is committed first and unconditionally to local storage, with no dependency on connectivity. This guarantees that field operations never block on the network and that no field datum can be lost to a connectivity failure (addressing FR-05, FR-20, and NFR-17). At this tier, the device's local store is the temporary authority for records that have not yet been reconciled.
- **The central tier** is realised by a PostgreSQL database governed by the backend services. Once a record is synchronised — whether over the office LAN or over the Internet — the central tier becomes the permanent authority. Reconciliation is idempotent and conflict-aware (NFR-19), so that the same record synchronised twice does not create a duplicate, and competing edits to the same logical record are resolved deterministically by a last-write-wins policy keyed on device timestamp (FR-28).

The functional decomposition of the system into its principal modules, each of which is elaborated in the architectural and detailed designs that follow, is shown in Figure 1. The data flow from field collection to SMS confirmation — spanning the local tier, the office tier, and the cloud tier — is depicted in Figure 2, which is reproduced exactly from the approved project workflow model.

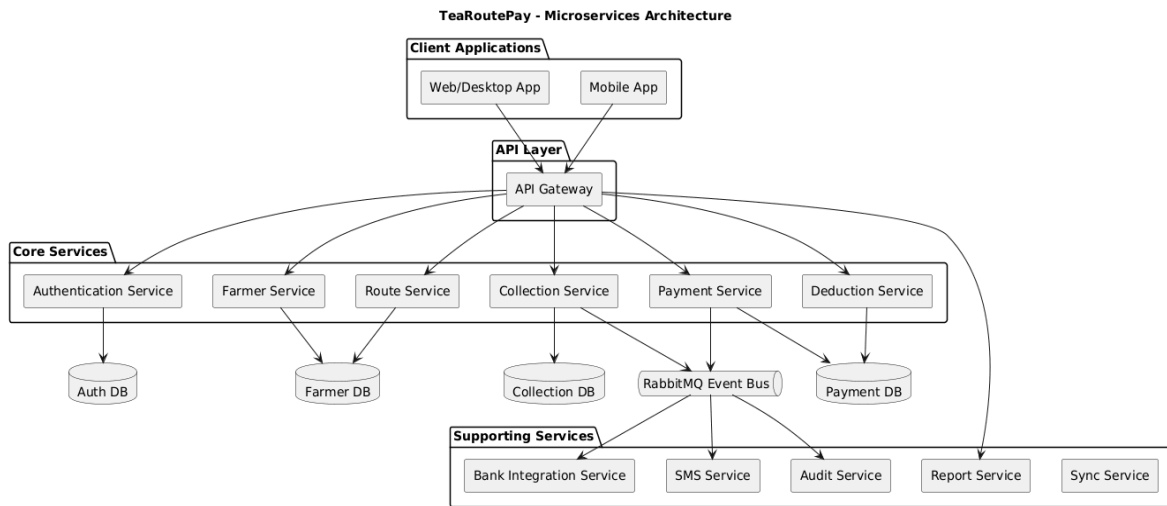


Figure 1: TeaRoutePay — High-Level System Workflow (Local, Cloud, and Payment Integration Pathways)

3.2.2 Relationship to the SRS

The design presented in this document implements the entirety of the SRS requirement baseline. Every functional requirement from FR-01 to FR-53 is realised by a service, a class, a use-case realisation, or a database structure described in Section 4, and every non-functional requirement from NFR-01 to NFR-29 is satisfied by a specific architectural or detailed-design mechanism consolidated in Section 8. The mapping is summarised at the level of quality attributes in the table below.

Requirement Category	Governing NFRs	Design Goal	Principal Design Mechanism
Performance	NFR-01 ... NFR-05	Sub-second field interaction; fast payment computation and sync	Local SQLite reads; indexed queries; batch synchronisation; horizontal scaling
Security	NFR-06 ... NFR-11	Encrypted communication and storage; immutable financial records	TLS 1.3; AES-256 at rest; RS256 JWT; immutable audit log; record locking
Usability	NFR-12 ... NFR-16	Field usability for low-proficiency users; localisation	Large touch targets; offline indicators; Sinhala/Tamil/English UI

Requirement Category	Governing NFRs	Design Goal	Principal Design Mechanism
Reliability	NFR-17 ... NFR-21	Zero field data loss; deterministic, idempotent processing	Write-ahead local persistence; ACID transactions; idempotent sync; retry with back-off
Scalability	NFR-22 ... NFR-24	Growth in farmers, devices, and offices	Stateless services; load-balanced gateway; per-service databases
Maintainability	NFR-25 ... NFR-29	Independent evolution and operability	Microservice isolation; schema migrations; externalised configuration; structured logging

3.3. ARCHITECTURAL DESIGN

3.3.1 Selected Architecture and Justification

The architecture selected for TeaRoutePay is a microservices architecture for the backend, deployed in concert with an offline-first mobile architecture on the collector device. This pairing is a deliberate response to the system's two dominant and partially opposing forces: the need for a robust, independently scalable, fault-isolated central platform that processes financial transactions of material value, and the need for a self-sufficient field client that continues to function with zero network dependency.

The microservices style decomposes the backend into a set of independently deployable services, each owning a bounded business capability and its own data store. The justification, expressed against the quality attributes prioritised by the SRS, is as follows.

Quality Attribute	Architectural Benefit	Requirements Addressed
Scalability	Each service scales independently behind the gateway; the computationally intensive Payment Service can be scaled at month-end without scaling unrelated services. The backend permits horizontal scaling without code change.	NFR-22, NFR-23, NFR-24
Fault isolation / Availability	A failure in a non-critical service (e.g., Reporting) cannot cascade into the collection or payment path. Combined with offline-first operation, the system tolerates partial outages gracefully.	NFR-17, NFR-21
Maintainability	Services can be modified, tested, and redeployed in isolation, supporting independent evolution of the codebase.	NFR-25, NFR-26, NFR-27
Security	Each service enforces capability-specific access control; the gateway provides a single, hardened security perimeter for authentication and rate-limiting.	NFR-06, NFR-08, FR-02
Deployability	Containerised services are deployed and orchestrated independently, enabling continuous delivery.	NFR-25

The offline-first mobile architecture treats local persistence as the primary write path and treats synchronisation as a deferred, asynchronous concern. This makes the field client immune to connectivity loss and is the direct architectural realisation of FR-20 and NFR-17. Encryption of the local store at rest (SQLCipher) and idempotent reconciliation complete the design (NFR-07, NFR-19).

Alternatives considered and rejected: A monolithic architecture was rejected because its tight coupling impedes the independent scaling of the payment-calculation workload and complicates the integration of the offline synchronisation subsystem. A layered (n-tier) architecture was rejected because it does not naturally accommodate the distributed, independently deployable topology required to host field, office, and cloud tiers with differing availability and scaling profiles. These decisions are recorded formally in Appendix A.

3.3.2 High-Level Microservices Architecture

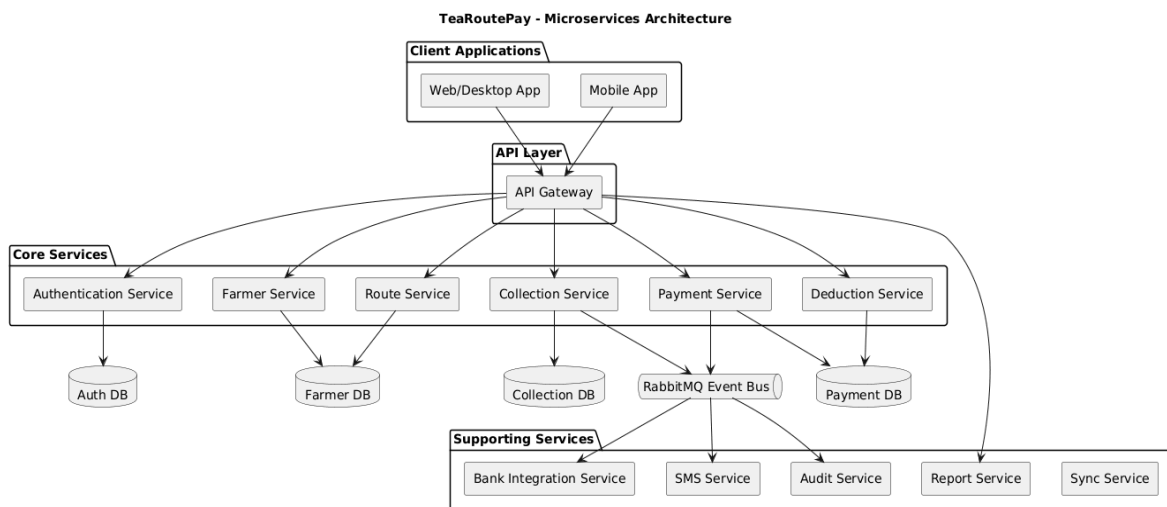


Figure 2: TeaRoutePay Microservices Architecture — Client, API, Core Service, and Supporting Service tiers

The high-level architecture organises the system into four horizontal tiers. At the top, the Client Applications tier comprises the Web/Desktop application (Office Staff and Owner) and the Mobile application (Collector). Neither client communicates with a service directly; all traffic is mediated by the API Layer, whose single ingress point is the API Gateway. The gateway is the architectural

locus of cross-cutting concerns — request routing, authentication enforcement, and rate limiting — and it is the only component exposed to client networks. Concentrating these responsibilities at the gateway keeps the downstream services free of duplicated security logic and provides a single, auditable security perimeter, directly supporting the access-control and session requirements (FR-01, FR-02, NFR-08).

The Core Services tier contains the six services that own the system's primary transactional capabilities: Authentication, Farmer, Route, Collection, Payment, and Deduction. The diagram shows the database-per-service pattern in operation: the Authentication Service owns Auth DB; the Farmer and Route services are backed by Farmer DB; the Collection Service owns Collection DB; and the Payment and Deduction services are backed by Payment DB. This per-service data ownership permits each service to be scaled, migrated, and secured independently, and is the structural basis for the scalability requirements (NFR-22 to NFR-24) and the schema-migration requirement (NFR-26).

The RabbitMQ Event Bus and the Supporting Services tier — Bank Integration, SMS, Audit, Report, and Sync — occupy the lower section of the diagram. The event bus decouples the synchronous payment path from the asynchronous side-effects that must follow a financial event. When the Payment Service completes and a payment run is approved, it publishes events to the bus rather than invoking downstream services in line; the Bank Integration, SMS, and Audit services consume those events independently. This decoupling allows external-integration latency and transient failures to be absorbed through retry and back-off without blocking the core financial transaction (NFR-21, FR-43 to FR-48). The Audit Service captures an immutable record of every financially significant event, which is the architectural foundation for NFR-10 and the auditability design of Section 7.

3.3.3 Service Responsibilities

Each backend service owns a single bounded capability. The responsibilities and data ownership of each are specified below.

Service	Responsibilities	Owns	Principal Requirements
Authentication Service	User login, JWT issuance and validation, user-account management, role-based access control	Auth DB (Users, Roles, Permissions, RefreshTokens)	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06
Farmer Service	Farmer registration, QR-code generation, profile update and deactivation, farmer search	Farmer DB (Farmers, QRMappings, FarmerAudit)	FR-07 to FR-13
Route Service	Route creation, farmer-to-route assignment, collector allocation, scheduling	Route DB	FR-14, FR-15, FR-16
Collection Service	Weight-record ingestion, offline synchronisation reconciliation, duplicate detection	Collection DB (Collections, SyncRecords)	FR-17 to FR-29
Deduction Service	Loan, fertilizer, and custom deduction recording; installment scheduling	Deduction DB	FR-32 to FR-36
Payment Service	Tea-rate management, monthly gross/net calculation, payment preview, approval, record locking	Payment DB	FR-30, FR-31, FR-37 to FR-42
Bank Integration Service	Bank Transfer API communication, retry logic, transfer-status retrieval, re-submission	— (integrates via gateway)	FR-43 to FR-46
SMS Service	SMS composition, localisation, delivery tracking	—	FR-47, FR-48, FR-49
Reporting Service	PDF and CSV report generation and export	— (reads from owning services)	FR-50, FR-51, FR-52, FR-53
Audit Service	Immutable, append-only audit logging; compliance tracking	Audit store	FR-06, FR-42, FR-45, NFR-10
Sync Service	Coordination of LAN and cloud synchronisation sessions; conflict logging	—	FR-25 to FR-29

3.3.4 Component Diagram

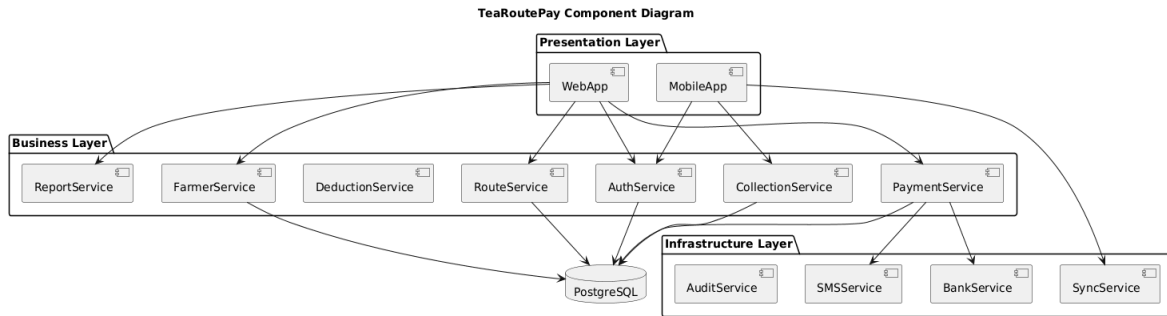


Figure 3: TeaRoutePay Component Diagram — Presentation, Business, and Infrastructure layers with dependency direction

The component diagram presents the system organised by logical layer, making the dependency direction explicit. The Presentation Layer contains the WebApp and MobileApp components. The Business Layer contains the service components — ReportService, FarmerService, DeductionService, RouteService, AuthService, CollectionService, and PaymentService — each a self-contained unit of business logic. The Infrastructure Layer contains the cross-cutting and integration components: AuditService, SMSService, BankService, and SyncService.

The dependency arrows encode an important design rule: presentation components depend on business components, and business components depend downward on shared infrastructure, but no business component depends on another business component for its core transaction. This acyclic, downward-only dependency structure makes the services independently testable and deployable, and prevents the emergence of the tangled coupling that motivated the rejection of the monolithic alternative. The convergence of multiple services on the PostgreSQL component in this view is a logical convergence on the relational technology; at deployment time each service is bound to its own schema, preserving the database-per-service property.

3.3.5 Deployment Diagram

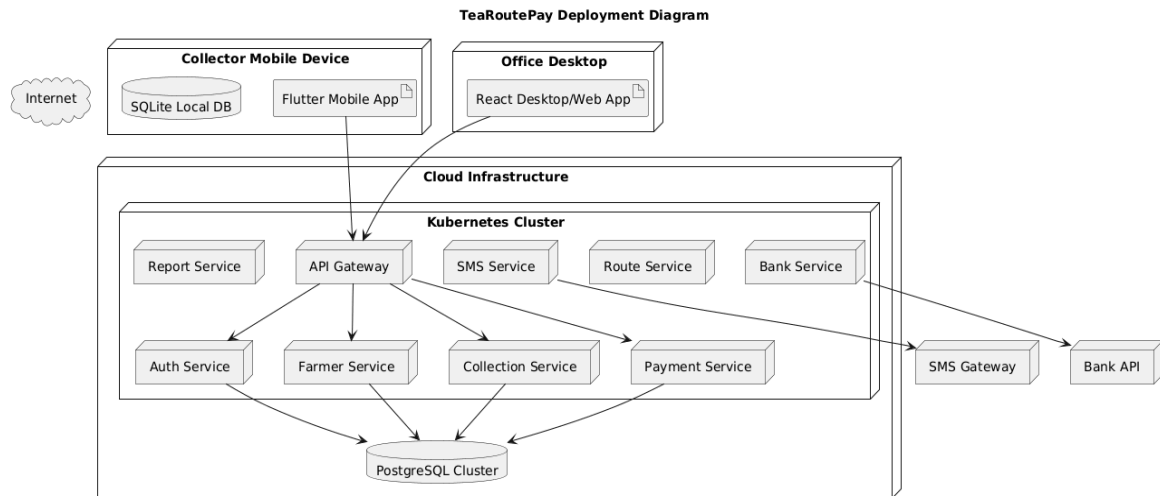


Figure 4: TeaRoutePay Deployment Diagram — Mobile Device, Office Desktop, and Kubernetes Cloud Infrastructure nodes

The deployment diagram fixes the physical placement of every executable component. Three deployment nodes are shown. The Collector Mobile Device node hosts the Flutter Mobile App and its co-resident SQLite Local DB; their co-residence is the physical embodiment of the offline-first principle — the field client reads and writes its authoritative working set entirely on-device, delivering the sub-second interaction of NFR-01 and the zero-data-loss guarantee of NFR-17. The Office Desktop node hosts the React Desktop/Web App used by Office Staff and the Owner.

The Cloud Infrastructure node contains a Kubernetes Cluster that orchestrates the containerised services — API Gateway, Auth Service, Farmer Service, Collection Service, Payment Service, Route Service, SMS Service, Bank Service, and Report Service — together with the backing PostgreSQL Cluster. Kubernetes provides the auto-scaling and self-healing that satisfy the horizontal-scaling requirement (NFR-24). The diagram also shows the two external integration endpoints, SMS Gateway and Bank API, reached outbound from the cluster. The combination of per-service schema ownership (described in Section 3.3) and a single managed database platform achieves a pragmatic balance between operational simplicity and architectural independence.

3.3.6 Technology Stack

The technology stack is specified below. Each selection is justified against the requirement or quality attribute it serves; collectively these choices constitute the implementation contract that the development phase must honour.

Component	Technology	Technical Justification	Requirements Addressed
Mobile App	Flutter	A single Dart codebase compiles to native Android, giving native-grade camera access for QR scanning and a high-performance offline UI. Flutter's reactive widget model supports the large touch targets and offline indicators required for low-proficiency field users.	FR-17 to FR-24, NFR-12, NFR-13
Office Client	React.js	A component-based, browser-deployable client that runs as both a desktop-hosted and web application, supporting tabular, dashboard-oriented administrative workflows.	FR-30 to FR-53, NFR-16
API Gateway	Spring Cloud Gateway	Provides a single, hardened ingress with built-in routing, authentication enforcement, and rate limiting, concentrating the security perimeter at one auditable point.	FR-01, FR-02, NFR-06, NFR-08
Backend Services	Spring Boot	The industry standard for enterprise, security-critical financial APIs; strong typing and declarative @Transactional boundaries provide the ACID guarantees required for monetary processing.	FR-37, FR-38, NFR-18, NFR-20
Central Database	PostgreSQL	A production-grade relational engine providing full ACID compliance, the correctness foundation for financial records, and per-service schema isolation.	NFR-18, NFR-22
Mobile Local Store	SQLite (SQLCIPHER)	An embedded, zero-administration, transactional store that delivers	FR-05, FR-20, NFR-07, NFR-17

Component	Technology	Technical Justification	Requirements Addressed
		on-device durability and, through SQLCipher, AES-256 encryption of data at rest on the device.	
Message Broker	RabbitMQ	Decouples the synchronous payment path from asynchronous side-effects (bank transfer, SMS, audit), enabling reliable, retryable, event-driven processing.	FR-43 to FR-48, NFR-10, NFR-21
Authentication	JWT (RS256) + RBAC	Stateless, signed tokens permit scalable authentication across services without server-side session affinity; RS256 asymmetric signing allows services to verify tokens without holding the signing key.	FR-01 to FR-04, NFR-08
Containerisation	Docker	Produces immutable, reproducible service images, guaranteeing parity between environments.	NFR-25
Orchestration	Kubernetes	Provides declarative deployment, horizontal auto-scaling, and self-healing for the service fleet.	NFR-24
Reporting	JasperReports	A Java-native engine integrating directly with Spring Boot to produce structured PDF reports and receipts.	FR-50 to FR-53
CI/CD	GitHub Actions	Automates build, test, and deployment, enforcing the coverage and quality gates of the maintainability requirements.	NFR-25, NFR-28

3.4. DETAILED DESIGN

3.4.1 Domain Class Diagram

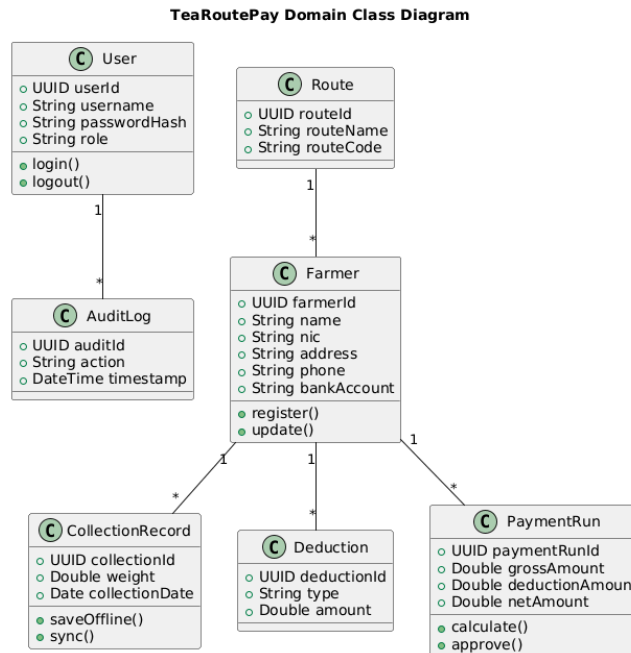


Figure 5: TeaRoutePay Domain Class Diagram — core entities, attributes, methods, and associations

The domain class diagram defines the core entities of the TeaRoutePay business domain, their attributes, their behaviour, and the associations and multiplicities that bind them. The model is intentionally aligned with the database design of Section 4.4, so that each persistent class corresponds to a table and each association corresponds to a foreign-key relationship.

User. This class represents any authenticated principal and carries `userId : UUID`, `username : String`, `passwordHash : String`, and `role : String`. Its behaviour — `login()` and `logout()` — encapsulates the authentication lifecycle. The `role` attribute is the discriminator on which all role-based access control decisions are made (FR-02). The one-to-many association from `User` to `AuditLog` means every audit entry is attributable to exactly one acting user, providing the structural guarantee behind the auditability requirement (NFR-10).

Farmer. The Farmer class is the pivot of the domain. It carries `farmerId` : UUID, `name`, `nic`, `address`, `phone`, and `bankAccount`, and exposes `register()` and `update()` behaviour. A Route aggregates many Farmers (1-to-*), each Farmer is the subject of many CollectionRecords (1-to-*), many Deductions (1-to-*), and many PaymentRun outcomes (1-to-*). The uniqueness of `nic` (enforced per FR-08) and the immutability of a farmer with payment history (FR-12) are realised against this class.

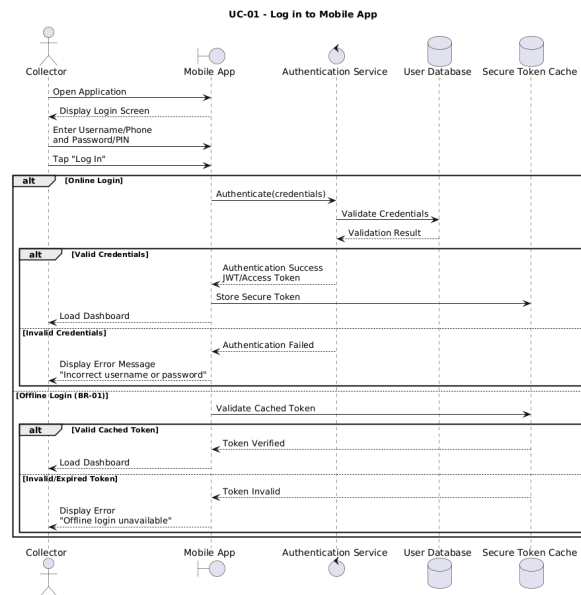
CollectionRecord. This class models a single field weight entry. It carries `collectionId` : UUID, `weight` : Double, and `collectionDate` : Date, and exposes the offline-first behaviour `saveOffline()` and `sync()`. The lifecycle of this class — from local creation, through the pending-sync and synced states, to its final immutable locking once it has contributed to an approved payment — is modelled explicitly by the Collection Record state machine in Section 4.3.

PaymentRun. This class represents one month's payment computation for the business. It carries `paymentRunId` : UUID, `grossAmount`, `deductionAmount`, and `netAmount`, and exposes `calculate()` and `approve()`. The transition of a PaymentRun from a mutable draft to a locked, immutable, approved artefact (FR-41) is the most safety-critical state transition in the system and is modelled by the Payment Run state machine in Section 4.3.

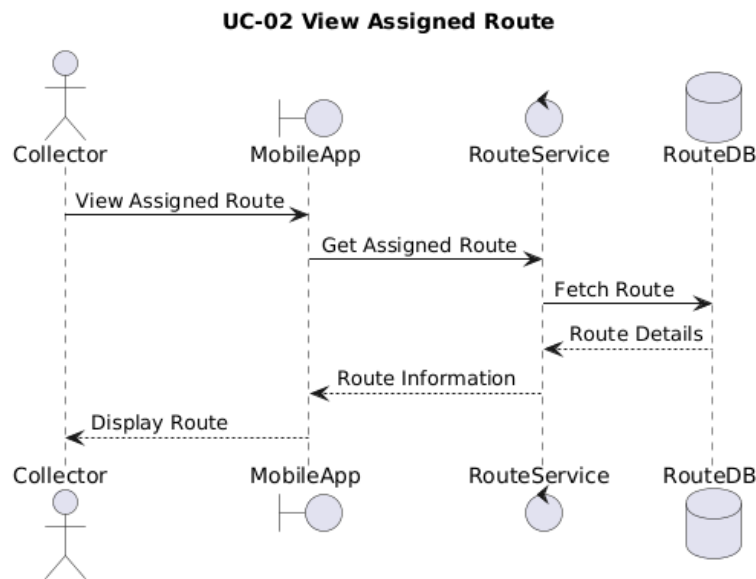
All monetary attributes shown as Double in the class diagram are, per the financial-accuracy constraint of the SRS, implemented as fixed-point (integer-cent) representations in storage and computation; floating-point arithmetic is prohibited for monetary values (FR-37, NFR-20). The class diagram captures the logical attribute; Section 4.4 specifies its precise persistent type.

3.4.2 Sequence Diagrams (Use-Case Realisations)

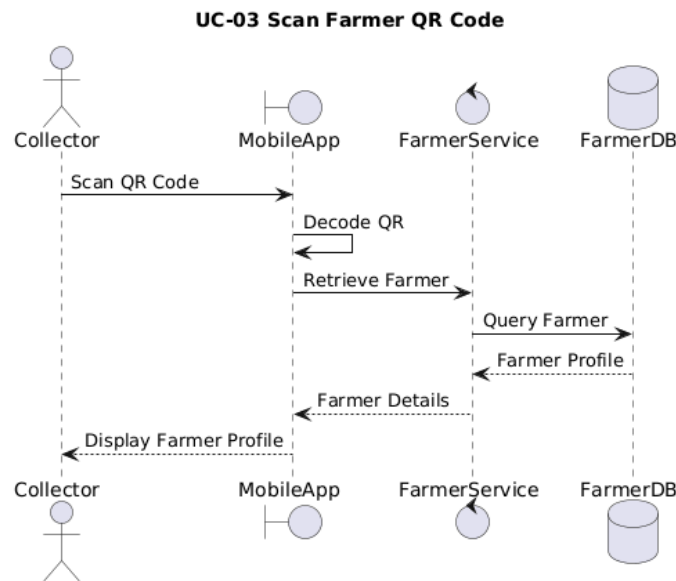
Each of the twenty principal use cases of the SRS is realised below by a sequence diagram reproduced exactly from the approved design. For each, the step-by-step technical execution is described with particular attention to data validation and to the state changes that the interaction effects. The diagrams share a consistent participant convention: an actor initiates an interaction with a client application (Mobile or Web), which dispatches through the API Gateway to the owning service, which reads or writes its database, a cache, or an external API.

SD-01 — Log in to Mobile App (UC-01)Figure 6: *SD-01 — Log in to Mobile App (UC-01)*

The Collector opens the application and is presented with the login screen. In the online branch, the Mobile App forwards credentials to the Authentication Service, which validates them against the User Database. On success, a JWT access token is returned, stored in the Secure Token Cache, and the dashboard is loaded. On invalid credentials, a neutral error message is displayed without disclosing which field was wrong. In the offline branch (BR-01), a previously cached token is validated locally: a verified token loads the dashboard, while an invalid or expired token yields an 'Offline login unavailable' message. The state change is the establishment of an authenticated, role-bound session on which all subsequent authorisation depends (FR-01, FR-02, NFR-08).

SD-02 — View Assigned Route (UC-02)*Figure 7: SD-02 — View Assigned Route (UC-02)*

Following authentication, the Collector requests their assigned route for the session. The Mobile App resolves the request against the locally synchronised route and farmer working set, returning the ordered list of farmers assigned to the collector's active route. Because the route working set is pre-synchronised to the device, this interaction completes entirely on-device with no connectivity dependency, allowing route selection to begin a collection session in the field (FR-17). No persistent state change occurs.

SD-03 — Scan Farmer QR Code (UC-03)*Figure 8: SD-03 — Scan Farmer QR Code (UC-03)*

The Collector invokes the device camera to scan the farmer's QR card. The Mobile App decodes the encoded farmer UUID and resolves it against the local farmer cache to load and display the farmer's identifying details for confirmation. Validation ensures the decoded UUID corresponds to an active farmer on the current route; an unresolved or out-of-route code is rejected with corrective guidance, and the manual-fallback path is offered (FR-19). The QR payload encodes only the farmer UUID — never personal or financial data — bounding the consequences of a lost card (FR-09, FR-18).

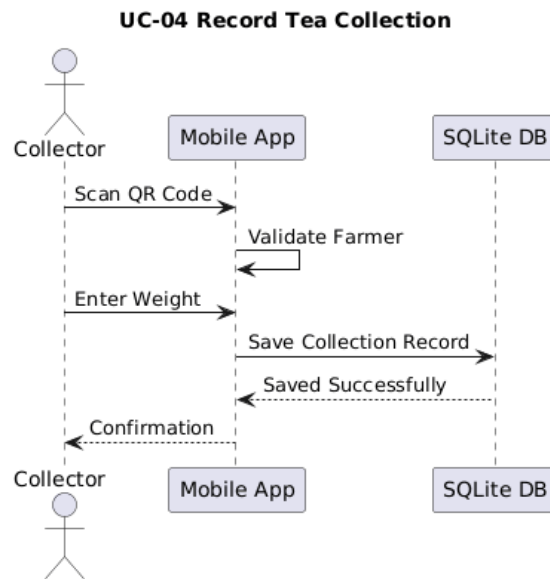
SD-04 — Record Leaf Weight (UC-04)

Figure 9: SD-04 — Record Leaf Weight (UC-04)

With a farmer identified, the Collector enters the leaf weight. The Mobile App validates the entry and commits the record to the local SQLite store immediately with a PendingSync status, independent of any network state. This unconditional local write is the defining moment of the offline-first design: the record is durable the instant it is saved, satisfying FR-20 and the zero-loss guarantee of NFR-17, and the running session total is updated (FR-23). The duplicate-prevention rule (FR-22) is evaluated here: a second entry for the same farmer in the same session prompts explicit confirmation. The state change is the creation of a new CollectionRecord in the PendingSync state.

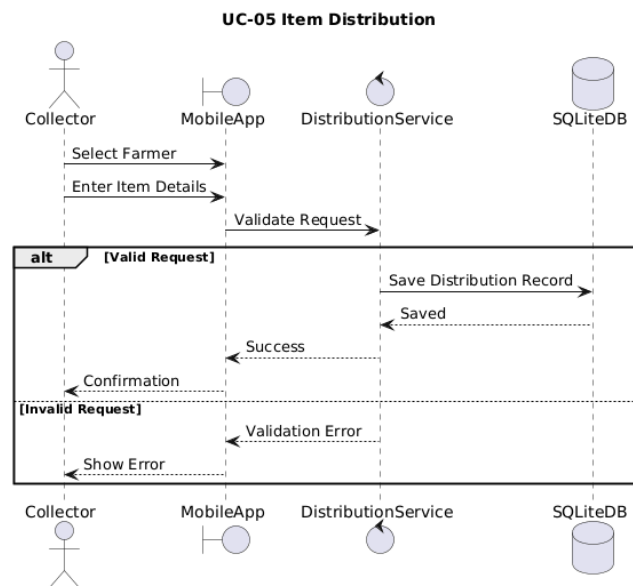
SD-05 — Item Distribution and Record Keeping (UC-05)

Figure 10: SD-05 — Item Distribution and Record Keeping (UC-05)

When fertilizer or other input materials are distributed to a farmer in the field, the Collector records the distribution through a quick-select flow. The Mobile App writes the distribution as a field-originated deduction record to local storage with PendingSync status, mirroring the weight-entry persistence pattern. On synchronisation, the Deduction Service ingests the record and schedules it as a deduction against the farmer's account (FR-34), to be applied during the monthly calculation. The interaction is fully offline-capable, and the state change is the creation of a pending field-deduction record.

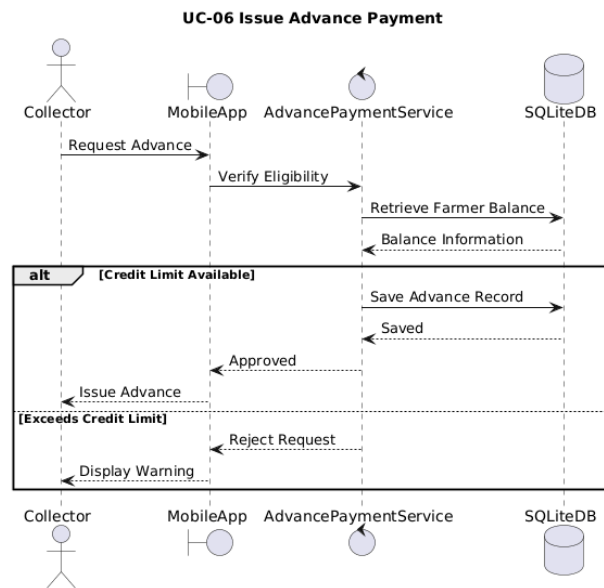
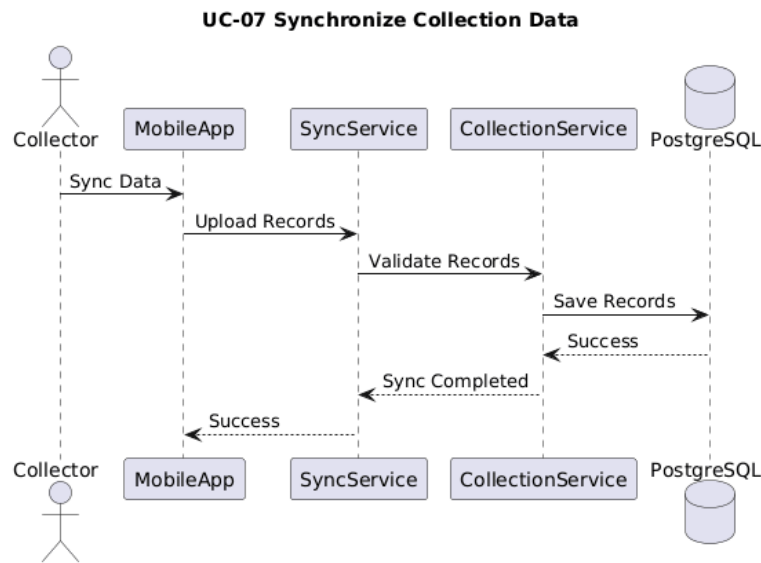
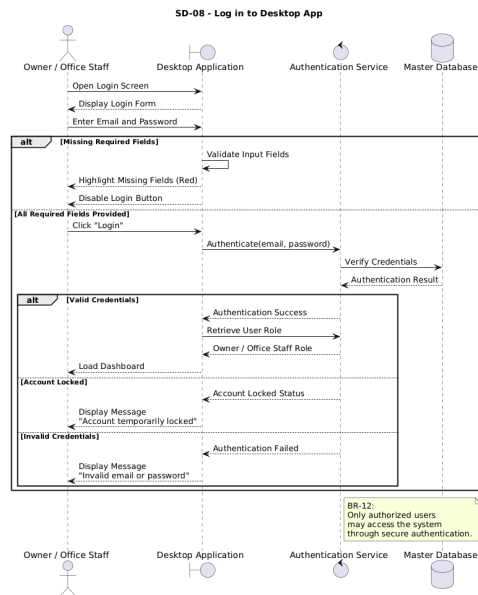
SD-06 — Issue Advance Payment and Record Keeping (UC-06)

Figure 11: SD-06 — Issue Advance Payment and Record Keeping (UC-06)

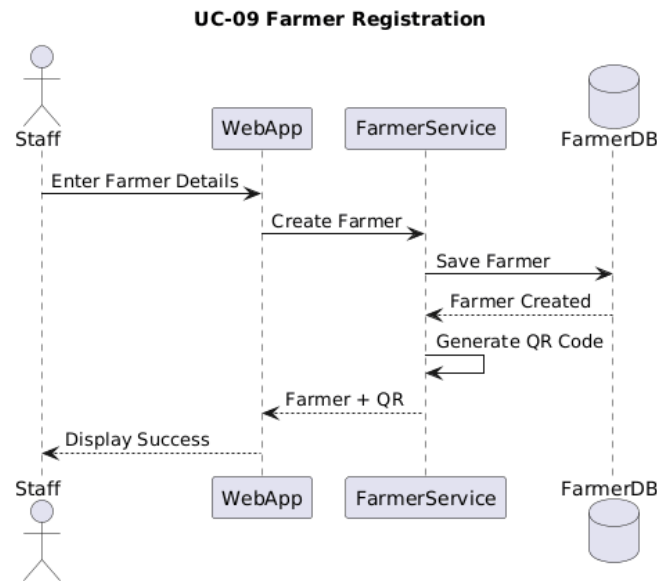
The Collector issues a cash advance to a farmer. Before approving the transaction, the Mobile App queries the local database to verify the farmer's accumulated balance and any applicable credit limit. A request within the limit is recorded locally as a pending advance and the physical handover proceeds; a request exceeding the limit follows the exception path, which requires an Owner override review before completion. The advance is reconciled to the Deduction Service on sync, where it joins the set of liabilities applied at month-end. This realisation enforces a business-rule validation at the point of field action while preserving the offline-first persistence guarantee.

SD-07 — Synchronise Collection Data (UC-07)*Figure 12: SD-07 — Synchronise Collection Data (UC-07)*

At the end of a route, the Collector triggers synchronisation. The system selects the configured pathway (Local LAN, Cloud, or Local-First-Cloud-Backup per FR-25) and transmits all PendingSync records in a batch. The receiving tier validates and persists the records, returns a confirmation, and the Mobile App marks the corresponding local records as Synced. The reconciliation is idempotent — a record already present is acknowledged but not re-inserted (NFR-19) — and conflicts are resolved by last-write-wins on device timestamp and logged for Owner review (FR-28). Failed transmissions are retried up to three times with exponential back-off before the user is notified (FR-29, NFR-21). The state change is the transition of each CollectionRecord from PendingSync to Synced.

SD-08 — Log in to Desktop App (UC-08)*Figure 13: SD-08 — Log in to Desktop App (UC-08)*

The Office Staff or Owner authenticates through the Web/Desktop client. Credentials are dispatched through the gateway to the Authentication Service and validated; on success a signed JWT is returned and held for the session, and the role-appropriate dashboard is loaded. Unlike the mobile flow, the desktop flow has no offline branch, as office operation presupposes connectivity to the central tier. Account lockout after five consecutive failures (FR-03) and the configurable idle session timeout (FR-04) are enforced on this path.

SD-09 — Register Farmer (UC-09)Figure 14: *SD-09 — Register Farmer (UC-09)*

Office Staff (or a Collector in the field) enters the mandatory farmer attributes — name, NIC, address, phone, route assignment, and bank account. The Farmer Service validates the submission, with the critical validation being the uniqueness of the NIC across all records, including deactivated ones (FR-08); a duplicate is rejected. On success, the farmer is persisted and a unique QR code encoding the new farmer UUID is generated (FR-09). The creation is written to the farmer audit trail with the acting user and timestamp (FR-11). The state change is the creation of a Farmer in the Registered state and the minting of its QR identity.

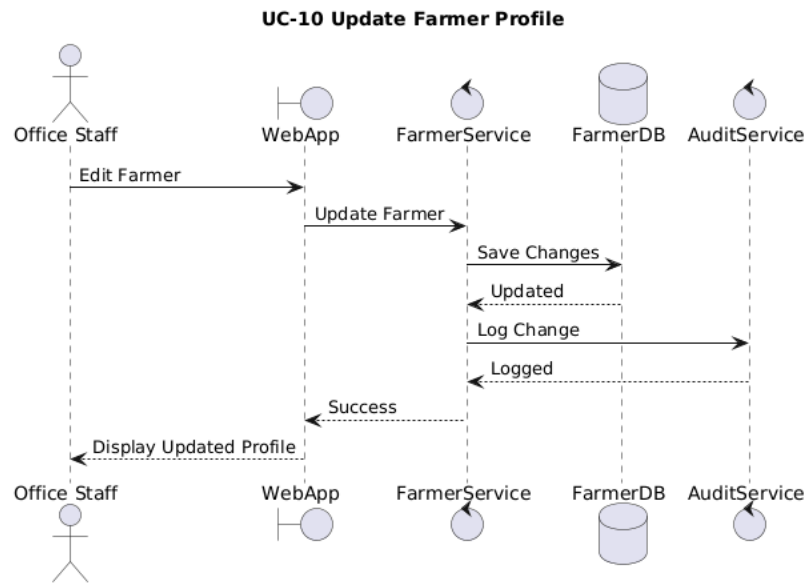
SD-10 — Manage Farmer Profiles (UC-10)

Figure 15: SD-10 — Manage Farmer Profiles (UC-10)

Office Staff retrieves a farmer profile and updates editable fields, or deactivates the farmer. The Farmer Service applies the change transactionally and appends an audit entry capturing the performing user's identity, timestamp, and before/after values (FR-11). Deactivation is a soft delete: a farmer carrying collection or payment history cannot be hard-deleted (FR-12), so the operation transitions the Farmer to the Deactivated state rather than removing the row, preserving the historical integrity on which past payment runs depend.

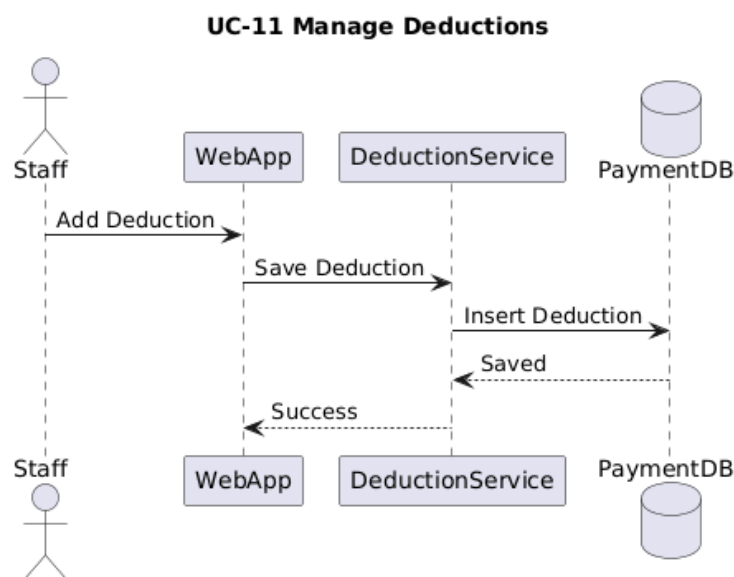
SD-11 — Manage Deductions (UC-11)

Figure 16: SD-11 — Manage Deductions (UC-11)

Office Staff records a liability against a farmer — a loan (with total amount, monthly installment, start month, and description), a fertilizer deduction, or a custom deduction type. The Deduction Service validates the parameters and persists the deduction, scheduling recurring installments where applicable (FR-32, FR-33, FR-34, FR-35). Field-originated deductions synced from the mobile app are surfaced here for review and confirmation before they participate in a calculation. The state change is the creation or amendment of scheduled liabilities that the Payment Service will consume at month-end.

SD-12 — Set Monthly Tea Rate (UC-12)

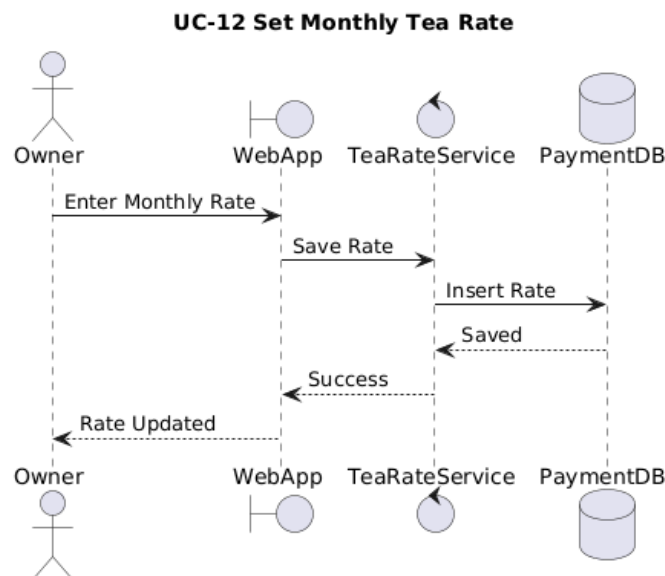


Figure 17: SD-12 — Set Monthly Tea Rate (UC-12)

The Owner or Office Staff sets the purchase rate (LKR per kilogram) for a given calendar month. The Payment Service validates that the target month does not already have a completed payment calculation — a rate for such a month is immutable (FR-31) — and persists the new rate to the rate history. The state change is the establishment of the multiplier that the gross-payment calculation (FR-37) will apply for that month; because rates are historised, recomputation of any past month uses that month's rate, never the current one.

SD-13 — Calculate Monthly Payment (UC-13)

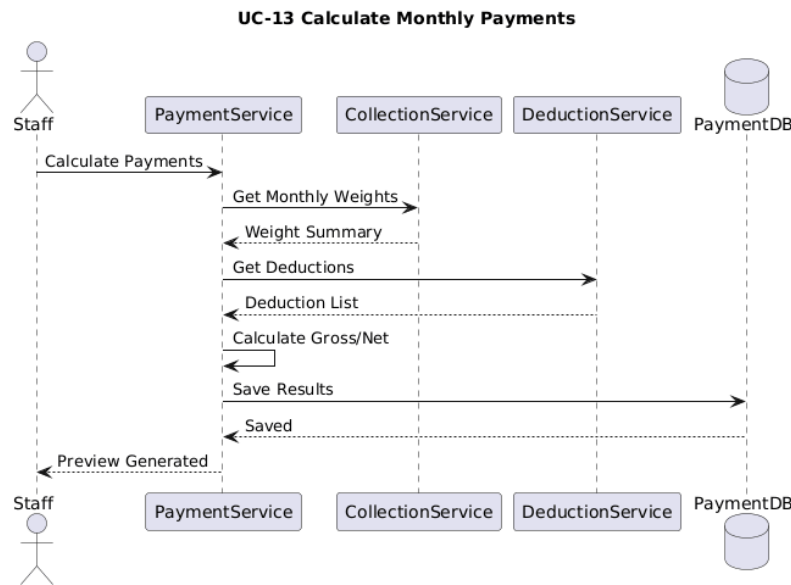


Figure 18: SD-13 — Calculate Monthly Payment (UC-13)

Office Staff initiates the monthly calculation. The Payment Service orchestrates the computation: it requests the monthly weight summary per farmer from the Collection Service, requests the applicable deduction list from the Deduction Service, and then computes, per farmer, the gross payment (total kilograms times the month's tea rate, in fixed-point arithmetic per FR-37) and the net payment (gross minus the sum of applicable deductions, per FR-38). Results are persisted and a preview is returned. A farmer whose net would be negative triggers the negative-payment alert and blocks the run from advancing to approval until resolved (FR-36). The calculation is deterministic (NFR-20) and completes within the performance envelope of NFR-04. The state change is the transition of the PaymentRun from Draft to Calculated.

SD-14 — View Payment Summary (UC-14)

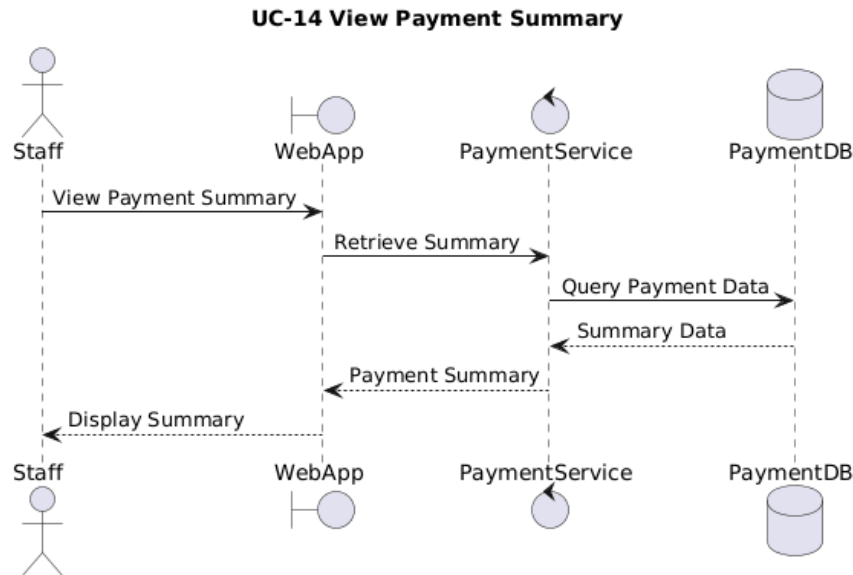


Figure 19: SD-14 — View Payment Summary (UC-14)

The Owner or Office Staff retrieves the computed preview, which presents for each farmer: total kilograms, gross payment, itemised deductions, and net payment in LKR (FR-39). The Payment Service serves the persisted results; the desktop client renders the tabular preview, loading within the performance bound of NFR-02. No state change occurs — this is a read interaction whose purpose is to furnish the Owner with the complete basis for the approval decision that follows.

SD-15 — Approve Payment Run (UC-15)

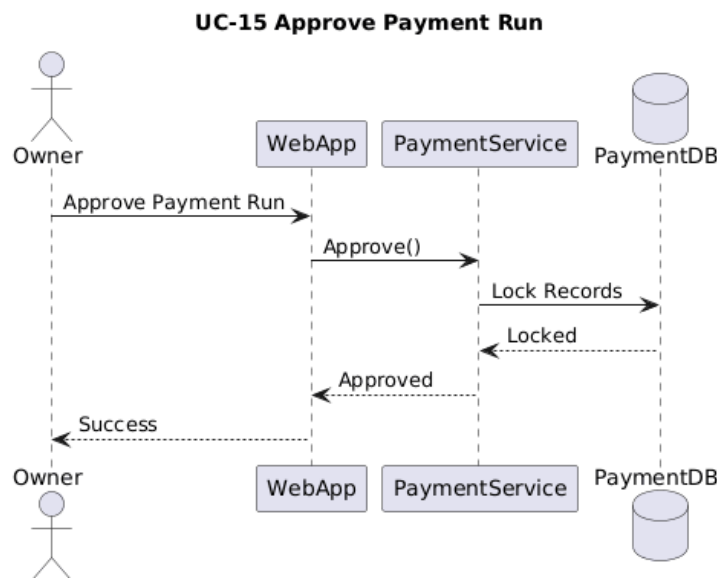


Figure 20: SD-15 — Approve Payment Run (UC-15)

The Owner reviews the preview and approves or rejects the run; a rejection must carry a reason (FR-40). On approval, the Payment Service performs the most safety-critical state transition in the system: every payment record in the run is locked and rendered immutable, after which no user may edit it (FR-41). The approving Owner's identity, approval timestamp, month, total approved amount, and any remarks are written to the immutable audit log (FR-42). Approval transitions the PaymentRun from PendingApproval to Approved and emits the event that initiates downstream disbursement. This separation of calculation (Staff) from approval (Owner) is the maker-checker control.

SD-16 — Process Bank Transfer (UC-16)

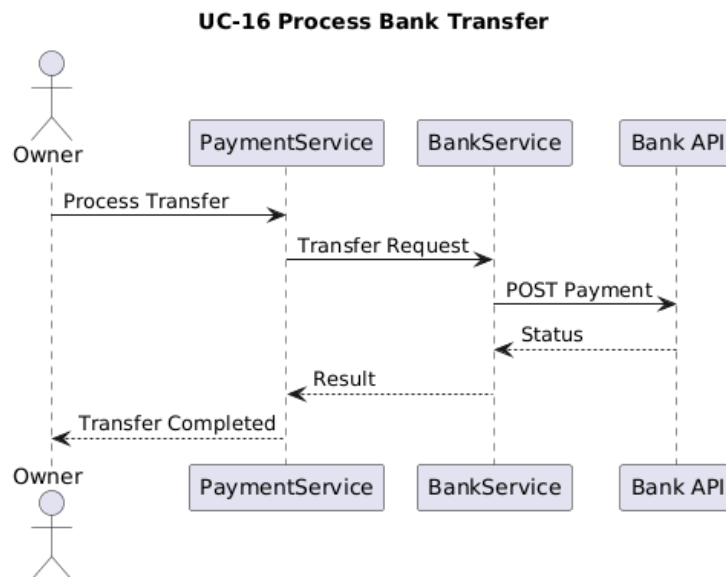


Figure 21: *SD-16 — Process Bank Transfer (UC-16)*

Upon approval, the Bank Integration Service submits the approved payments to the external Bank Transfer API, targeting submission within five minutes of approval (FR-43). For each payment it retrieves and records the transfer status — Success, Pending, or Failed (FR-44) — and writes the full request, response, HTTP status, and timestamp to the audit log (FR-45). Transient failures are retried with exponential back-off (max three attempts, NFR-21); a persistently failed transfer may be manually re-submitted once the cause is resolved (FR-46). The state change is the transition of the PaymentRun into BankProcessing and, on confirmation, the recording of a definitive per-payment transfer outcome.

SD-17 — Send SMS Notification (UC-17)

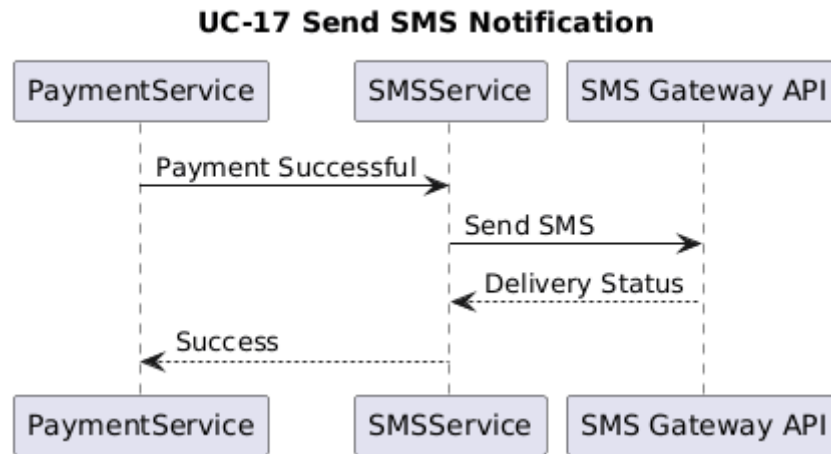


Figure 22: SD-17 — Send SMS Notification (UC-17)

Strictly after a bank transfer is confirmed successful — never at calculation or approval (FR-48, BR-25) — the SMS Service composes and dispatches a confirmation to the farmer stating the net amount in LKR and the payment month (FR-47). The message is rendered in the farmer's preferred language among Sinhala, Tamil, and English (FR-49). Delivery is tracked, and the loop with the farmer — the final node of the high-level workflow — is closed. The state change is the recording of a delivered (or failed-and-retryable) notification against the payment.

SD-18 — Generate Reports (UC-18)

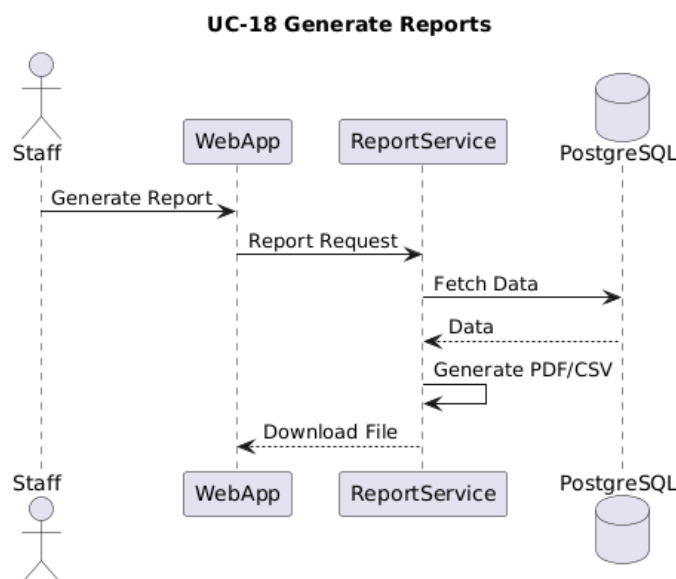


Figure 23: SD-18 — Generate Reports (UC-18)

An Owner or Office Staff requests a report — the daily collection report (FR-50) or the monthly payment report (FR-51). The Reporting Service aggregates the relevant data from the owning services and renders the output through JasperReports, offering export in both PDF and CSV (FR-52). Because reporting reads from rather than writes to the transactional services, report generation cannot perturb financial state. No state change occurs.

SD-19 — Manage User Accounts (UC-19)

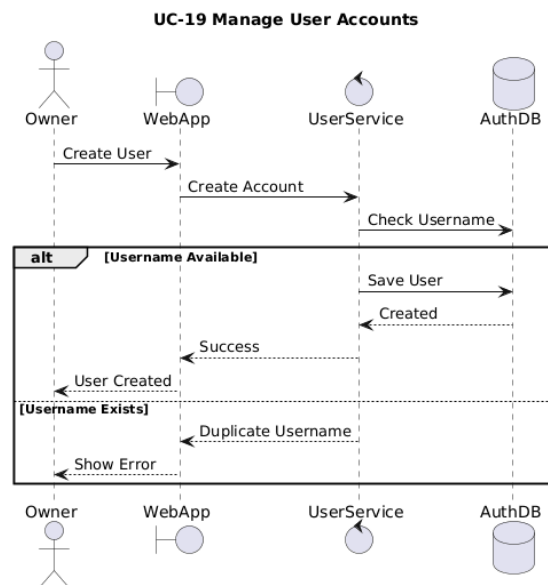


Figure 24: *SD-19 — Manage User Accounts (UC-19)*

The Owner manages system users: creating a Collector or Staff account, assigning a role and a default password, or deactivating an account (FR-05). The Authentication Service validates the submission — rejecting usernames that are not unique (BR-27) and passwords under eight characters — and persists the credential set. Every such administrative action is audit-logged (FR-06). The state change is the creation, modification, or deactivation of a principal and its associated role binding, which immediately governs that principal's subsequent access.

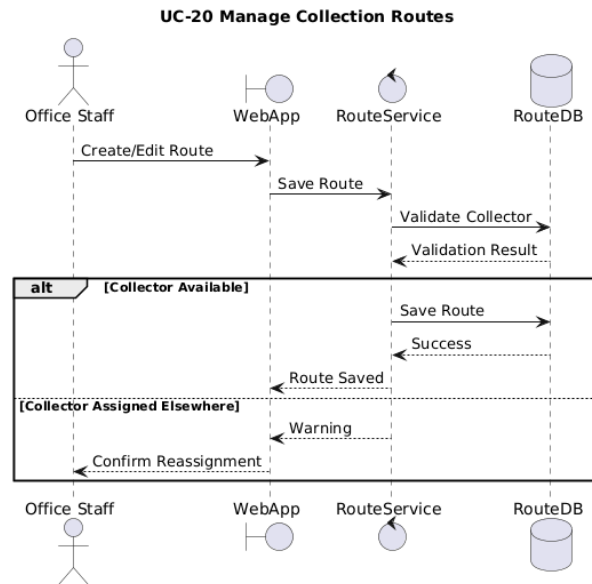
SD-20 — Manage Collection Routes (UC-20)

Figure 25: *SD-20 — Manage Collection Routes (UC-20)*

The Owner or Office Staff creates or edits a route, sets its name and code, and assigns a responsible Collector (FR-14); farmers are assigned or re-assigned to routes, with the full assignment history retained (FR-15). The Route Service enforces the governing business rules: a farmer may belong to only one active route at a time (BR-28), a route must exist before it can populate the farmer-registration dropdown (BR-29), and re-assigning an already-allocated collector raises a warning prompt. The state change is the creation or amendment of the route topology against which all field collection is organised.

3.4.3 State Machine Diagrams

The system contains three entities whose behaviour is governed by an explicit lifecycle. Modelling these as state machines makes the permissible transitions — and equally importantly the forbidden ones — unambiguous for implementation and testing.

3.4.3.1 Farmer State Machine

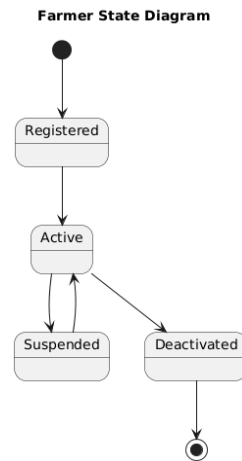


Figure 27: Farmer State Diagram — Registered, Active, Suspended, Deactivated lifecycle

A Farmer enters the Registered state upon creation (FR-07). From Registered it transitions to Active, the normal operating state in which the farmer appears in route collection lists and participates in payment runs. From Active, two transitions are possible: to Suspended, a reversible state (shown by the bidirectional transition) used for temporary holds, from which the farmer may be reactivated; and to Deactivated, the terminal soft-deleted state reached when a farmer ceases supply. The transition into Deactivated is the soft-delete of FR-12: the record and its history are retained, but the farmer no longer appears in active collection lists.

3.4.3.2 Payment Run State Machine

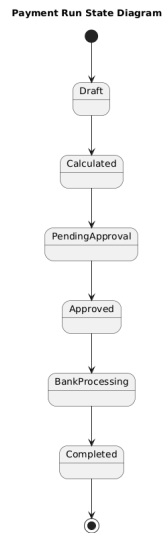


Figure 28: Payment Run State Diagram — Draft through Completed (irreversible linear lifecycle)

The PaymentRun lifecycle is strictly linear and irreversible — appropriate for a financial artefact. A run begins in Draft, transitions to Calculated when the gross/net computation completes (FR-37, FR-38), and to PendingApproval when the preview is submitted for the Owner's decision (FR-39). Owner approval transitions it to Approved, at which point all constituent records are locked and immutable (FR-41) — this is the point of no return. From Approved the run enters BankProcessing as disbursement is executed (FR-43), and finally reaches Completed once transfers are confirmed, after which SMS notifications are emitted (FR-47). The absence of any backward transition from Approved onward is the structural enforcement of payment immutability; correction of an error in an approved run is handled not by mutation but by a compensating run.

3.4.3.3 Collection Record State Machine

Collection Record State Diagram

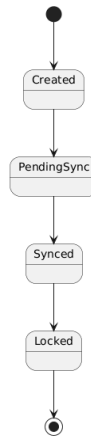


Figure 29: Collection Record State Diagram — Created, PendingSync, Synced, Locked

A CollectionRecord is Created the moment a weight is entered on the device, and immediately enters PendingSync — it is durable locally before it is ever transmitted (FR-20, NFR-17). On successful synchronisation it transitions to Synced, at which point central authority is established (FR-26/FR-27). Once the record has contributed to an approved payment run, it transitions to Locked and becomes immutable, mirroring the immutability of the payment that consumed it (FR-41). This lifecycle is the per-record complement to the Payment Run lifecycle: together they guarantee that the chain from a field weight entry to a disbursed payment is, once approved, tamper-evident end to end.

3.4.4 Database Design

3.4.4.1 Central (Cloud / Office) Entity-Relationship Model

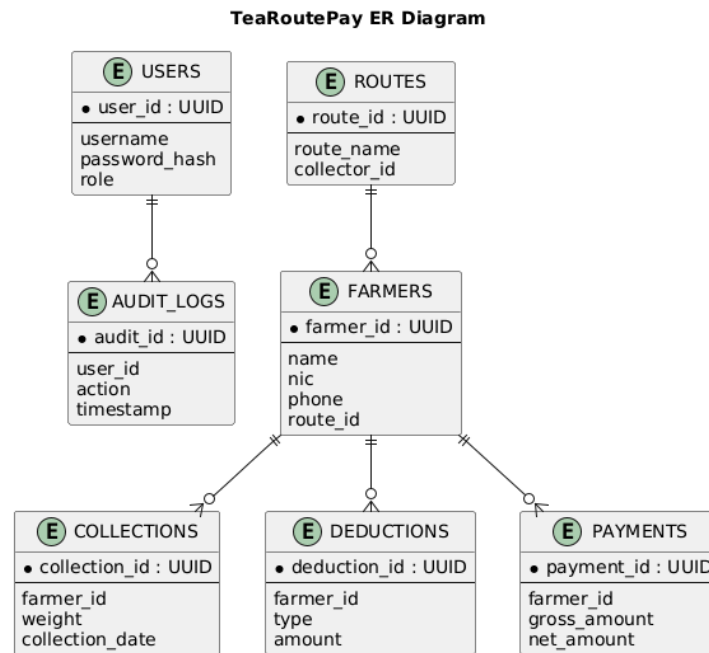


Figure 30: TeaRoutePay Central Entity-Relationship Diagram — *USERS, ROUTES, FARMERS, COLLECTIONS, DEDUCTIONS, PAYMENTS, AUDIT_LOGS*

The central entity-relationship model defines the authoritative relational schema held in the PostgreSQL tier. Seven entities are modelled. **USERS** holds principals and their roles. **ROUTES** holds the collection-route master data and its assigned collector. **FARMERS** is the central entity, carrying a foreign key to **ROUTES** and acting as the parent of the three transactional entities. **COLLECTIONS**, **DEDUCTIONS**, and **PAYMENTS** each hold a `farmer_id` foreign key and represent, respectively, the field weight records, the scheduled liabilities, and the computed payment outcomes. **AUDIT_LOGS** references **USERS** and captures the immutable action history.

Every primary key is a UUID, which is essential to the offline-first design: because identifiers are generated on the device at record-creation time and are globally unique, records created offline on multiple devices can be merged into the central store without key collision, which is the structural precondition for idempotent, conflict-aware synchronisation (NFR-19).

3.4.4.2 Mobile Local (SQLite) Schema

The collector device holds a deliberately reduced, denormalised subset of the central schema, optimised for offline read performance and synchronisation bookkeeping. It carries only what the

field workflow requires, plus the synchronisation metadata that the cloud schema does not need. Three design intentions are made explicit. First, weights and monetary amounts are stored as integers (`weight_grams`, `amount_cents`), never as floats, honouring the fixed-point financial-accuracy constraint at the very point of capture (FR-37, NFR-20). Second, every transactional row carries a `sync_status` and a `device_timestamp`; the former drives the Collection Record state machine, and the latter is the key on which last-write-wins conflict resolution operates (FR-28). Third, an explicit `SYNC_QUEUE` with a bounded `retry_count` is the on-device realisation of the retry-with-back-off requirement (FR-29, NFR-21). The entire local store is encrypted at rest via SQLCipher (NFR-07).

3.4.4.3 Data Dictionary

The data dictionary below specifies the principal persistent attributes of the central schema, their types, constraints, and the requirements they serve. Monetary fields use a fixed-point integer representation (cents); identifiers are UUIDs to support offline-safe key generation.

Entity	Attribute	Type	Constraints	Notes / Requirements
USERS	<code>user_id</code>	UUID	PK, not null	Principal identifier
USERS	<code>username</code>	VARCHAR(64)	unique, not null	BR-27 uniqueness (FR-05)
USERS	<code>password_hash</code>	VARCHAR(255)	not null	Salted hash; never plaintext (NFR-07)
USERS	<code>role</code>	VARCHAR(20)	not null, enum	Owner/Office Staff/Collector/SysAdmin (FR-02)
USERS	<code>status</code>	VARCHAR(16)	not null	Active / Locked (FR-03)
ROUTES	<code>route_id</code>	UUID	PK, not null	Route identifier
ROUTES	<code>route_code</code>	VARCHAR(20)	unique, not null	(FR-14)
ROUTES	<code>collector_id</code>	UUID	FK → USERS, not null	Assigned collector (FR-14)
FARMERS	<code>farmer_id</code>	UUID	PK, not null	Encoded in QR card (FR-09)
FARMERS	<code>nic</code>	VARCHAR(12)	unique, not null	Unique incl. deactivated (FR-08)

Entity	Attribute	Type	Constraints	Notes / Requirements
FARMERS	bank_account	VARCHAR(34)	not null	Disbursement target (FR-43)
FARMERS	route_id	UUID	FK → ROUTES	One active route (BR-28, FR-15)
FARMERS	preferred_language	VARCHAR(8)	not null	si / ta / en (FR-49)
FARMERS	status	VARCHAR(16)	not null	Registered/Active/Suspended/Deactivated (FR-12)
COLLECTIONS	collection_id	UUID	PK, not null	Device-generated (FR-20)
COLLECTIONS	weight_grams	BIGINT	not null, ≥ 0	Fixed-point integer (FR-37, NFR-20)
COLLECTIONS	sync_status	VARCHAR(16)	not null	Created/PendingSync/Synced/Locked
COLLECTIONS	device_timestamp	TIMESTAMP	not null	Conflict key (FR-28)
DEDUCTIONS	type	VARCHAR(20)	not null	Loan/Fertilizer/Custom (FR-32–35)
DEDUCTIONS	total_amount_cents	BIGINT	not null, ≥ 0	Fixed-point (NFR-20)
DEDUCTIONS	installment_cents	BIGINT	≥ 0	Loan amortisation (FR-33)
DEDUCTIONS	balance_cents	BIGINT	≥ 0	Remaining liability (FR-33)
DEDUCTIONS	start_month	CHAR(7)	YYYY-MM	(FR-32)
PAYMENTS	gross_amount_cents	BIGINT	not null, ≥ 0	kg × rate (FR-37)
PAYMENTS	deduction_amount_cents	BIGINT	not null, ≥ 0	Σ deductions (FR-38)
PAYMENTS	net_amount_cents	BIGINT	not null	Gross – deductions (FR-38, FR-36)
PAYMENTS	status	VARCHAR(20)	not null	Locked on approval (FR-41)
PAYMENTS	transfer_status	VARCHAR(12)	nullable	Success/Pending/Failed (FR-44)
AUDIT_LOGS	audit_id	UUID	PK, not null	(NFR-10)
AUDIT_LOGS	user_id	UUID	FK → USERS, not null	Actor attribution (NFR-10)
AUDIT_LOGS	action	VARCHAR(255)	not null	Append-only (FR-06, FR-42, FR-45)
AUDIT_LOGS	timestamp	TIMESTAMP	not null	(NFR-10)

3.4.4.4 Architectural Data-Integrity Rules

Beyond conventional relational constraints, the design enforces four cross-cutting data-integrity rules essential to the financial trustworthiness of the system.

- **1. Maker-Checker Separation.** No single principal may both compute and authorise a disbursement. The payment calculation is performed by Office Staff (the maker, SD-13), but the approval that locks the run and releases funds is reserved to the Owner (the checker, SD-15). This separation of duties is enforced by the role-based access control matrix (Section 7) and structurally prevents both fraud and unilateral error from reaching the bank (FR-40, FR-41, FR-02).
- **2. Synchronisation Integrity.** Reconciliation from device to centre is idempotent and conflict-aware. Because every record carries a globally unique UUID generated at creation, a record transmitted more than once is recognised and never duplicated (NFR-19); and where two devices submit competing values for the same logical record, the last-write-wins policy keyed on device_timestamp deterministically selects the winner and logs the conflict for Owner review (FR-28).
- **3. Immutable Financial History.** Once a payment run is approved, its payment records, and the collection records that fed them, transition to a Locked state and become immutable; no update or delete is permitted by any role (FR-41). The audit log is strictly append-only — entries may be written and read but never altered or removed — providing a tamper-evident record of every action affecting farmer profiles, deductions, and payments (NFR-10, FR-06, FR-42, FR-45).
- **4. Loan Amortisation Integrity.** A loan is recorded once with a total amount and a monthly installment (FR-32). During each monthly calculation, the Payment Service applies exactly one installment, decrementing the loan's balance_cents, and ceases to apply the deduction automatically once the balance reaches zero (FR-33). The invariant maintained is that the sum of installments applied across all runs never exceeds the original total, and the balance is monotonically non-increasing — preventing both over-recovery from the farmer and indefinite deduction.

3.4.4.5 Requirements Traceability Matrix (Functional)

The following matrix consolidates the realisation of every functional requirement by the design artefacts of this section, providing forward traceability from requirement to design.

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-01	User login	Authentication Service; User	SD-01, SD-08
FR-02	Role-based access control	Authentication Service	SD-01, SD-08, SD-19
FR-03	Account lockout	Authentication Service	SD-08
FR-04	Session timeout	Auth Service / Gateway	SD-08
FR-05	User-account management	Authentication Service; User	SD-19
FR-06	Auth audit logging	Audit Service; AuditLog	SD-19
FR-07	Farmer registration	Farmer Service; Farmer	SD-09
FR-08	Unique NIC validation	Farmer Service	SD-09
FR-09	QR-code generation	Farmer Service	SD-09, SD-03
FR-10	QR-card printing	Farmer Service / Reporting	SD-09
FR-11	Profile update + audit	Farmer Service; AuditLog	SD-10
FR-12	Farmer deactivation (soft)	Farmer Service; Farmer SM	SD-10
FR-13	Farmer search / filter	Farmer Service	SD-10
FR-14	Route creation	Route Service; Route	SD-20
FR-15	Farmer-to-route assignment	Route Service	SD-20
FR-16	Route schedule	Route Service	SD-20
FR-17	Route selection at session start	Mobile App / Route cache	SD-02
FR-18	QR-code scanning	Mobile App; Collection Service	SD-03
FR-19	Manual farmer fallback	Mobile App	SD-03
FR-20	Offline weight entry + local store	Collection Service; CollectionRecord	SD-04

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-21	Pre-sync weight correction	Collection Service	SD-04
FR-22	Duplicate-entry prevention	Collection Service	SD-04
FR-23	Session summary view	Mobile App	SD-04
FR-24	Connectivity-status indicator	Mobile App	SD-07
FR-25	Dual / selectable sync mode	Sync Service	SD-07
FR-26	Local LAN sync	Sync Service	SD-07
FR-27	Cloud sync	Sync Service	SD-07
FR-28	Sync conflict resolution	Sync Service	SD-07
FR-29	Sync retry with back-off	Sync Service	SD-07
FR-30	Monthly tea-rate setting	Payment Service	SD-12
FR-31	Tea-rate history (immutable past)	Payment Service	SD-12
FR-32	Loan deduction recording	Deduction Service; Deduction	SD-11
FR-33	Automatic loan installment	Deduction / Payment Service	SD-11, SD-13
FR-34	Fertilizer deduction recording	Deduction Service	SD-05, SD-11
FR-35	Custom deduction types	Deduction Service	SD-11
FR-36	Negative-payment alert	Payment Service	SD-13
FR-37	Gross-payment calculation	Payment Service; PaymentRun	SD-13
FR-38	Net-payment calculation	Payment Service; PaymentRun	SD-13
FR-39	Payment preview	Payment Service	SD-14
FR-40	Owner payment approval	Payment Service	SD-15

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-41	Record immutability on approval	Payment Service; State Machines	SD-15
FR-42	Approval audit record	Audit Service	SD-15
FR-43	Bank-transfer submission	Bank Integration Service	SD-16
FR-44	Transfer-status retrieval	Bank Integration Service	SD-16
FR-45	Transfer audit log	Audit Service	SD-16
FR-46	Manual transfer re-submission	Bank Integration Service	SD-16
FR-47	Farmer SMS on success	SMS Service	SD-17
FR-48	SMS only after confirmation	SMS Service	SD-17
FR-49	Multi-language SMS	SMS Service	SD-17
FR-50	Daily collection report	Reporting Service	SD-18
FR-51	Monthly payment report	Reporting Service	SD-18
FR-52	Report export (PDF/CSV)	Reporting Service	SD-18
FR-53	Farmer payment receipt	Reporting Service	SD-18

3.5. USER INTERFACE DESIGN

3.5.1 UX Decisions for Low-Technical-Proficiency Users

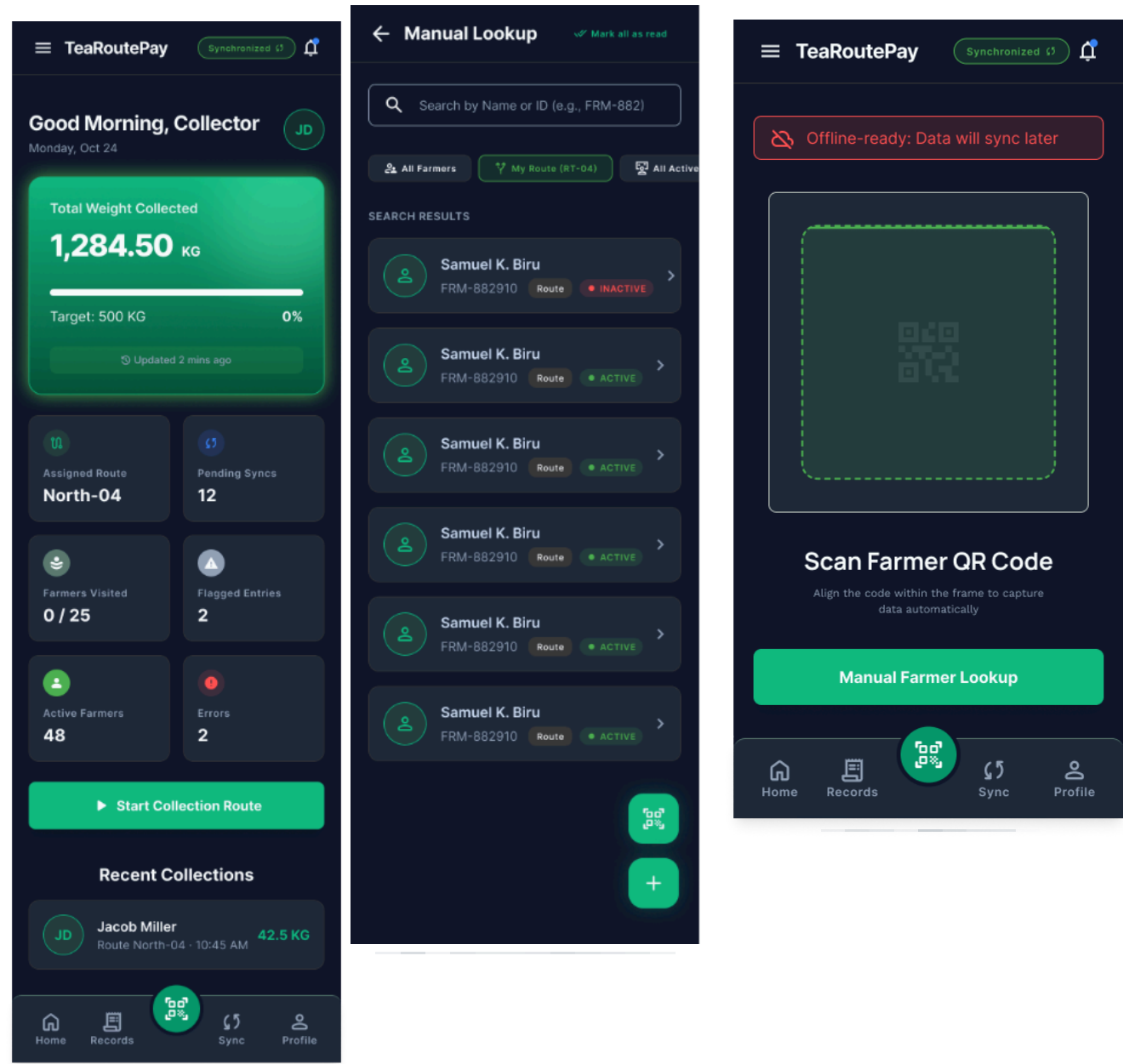
The user-experience design is governed by the persona analysis of the SRS, which characterises the primary field user (the Collector, Suresh, 28, lorry driver) as low in technical proficiency and the primary office user (the Owner, Nimal, 52) as medium. The design optimises the field client for speed, error-resistance, and minimal typing, and optimises the office client for familiar, tabular, dashboard-driven control. The following decisions implement the usability requirements NFR-12 through NFR-16.

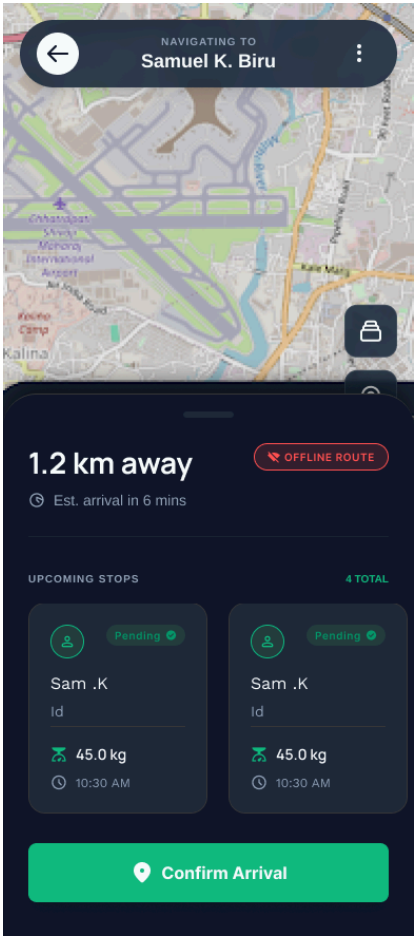
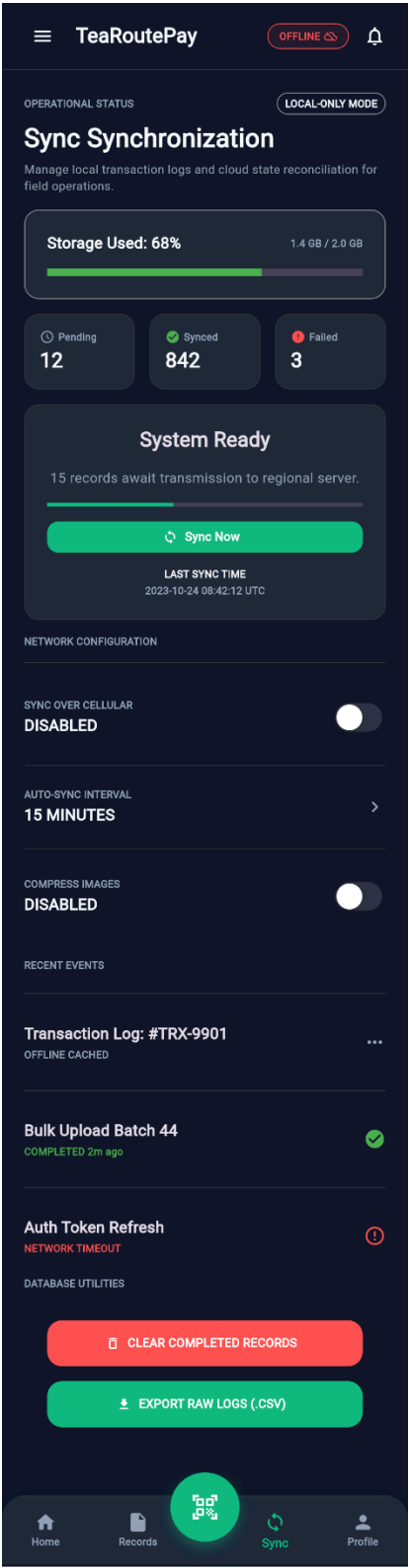
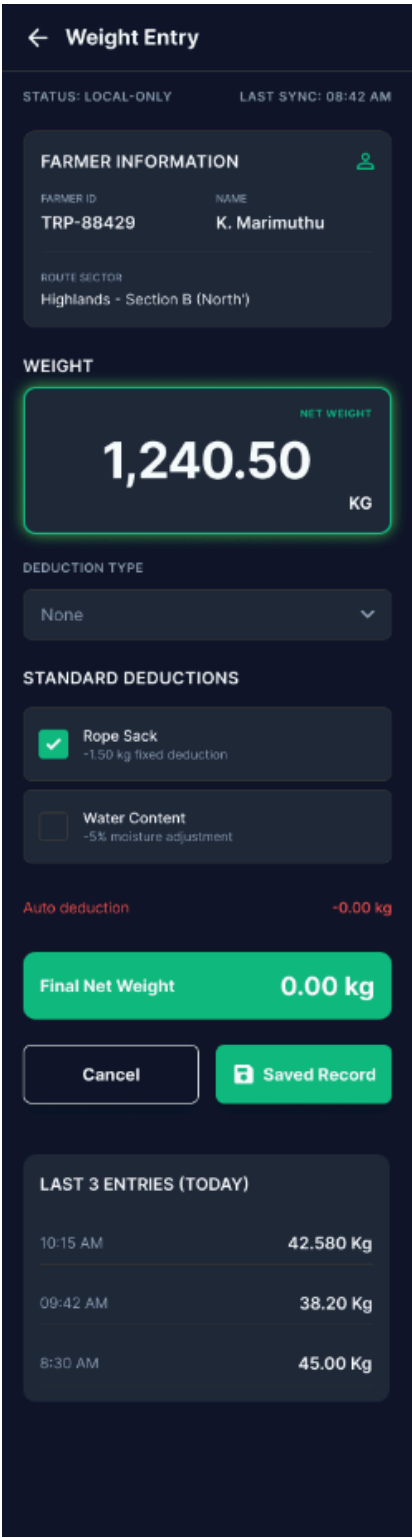
- **Large touch targets.** Every interactive element on the mobile client meets a minimum target of 48x48 dp, sized for one-handed use by a standing collector, often gloved and in bright outdoor light (NFR-12). Primary actions — Scan, Save, Sync — are rendered as oversized, high-contrast buttons.
- **Minimal-typing flows.** The core collection task is reduced to a three-step flow — scan, weigh, save — in which QR scanning replaces manual farmer identification entirely (FR-18) and numeric weight entry is the only free input. Field deductions and advances use quick-select menus rather than free text. This directly serves the 30-minute training target of NFR-13.
- **Offline indicators.** A persistent connectivity indicator displays the current state — Offline, LAN Connected, or Internet Connected — and a visible pending-sync counter shows how many records remain unsynced (FR-24). This makes the system's offline behaviour legible to the user, who must trust that an unsynced record is nonetheless safe.
- **Plain-language, corrective error messages.** All error messages are written in plain language with explicit corrective guidance rather than codes or technical jargon (NFR-15).
- **Localisation.** Both clients support Sinhala, Tamil, and English, with the language selectable per user; SMS content is additionally localised to each farmer's preferred language (NFR-14, FR-49).
- **Familiar office conventions.** The desktop/web client follows standard conventions — keyboard navigation, tab order, tabular data entry, and a single-click approval flow — so that no skill beyond ordinary desktop usage is required (NFR-16).

The principal mobile screens are: Login, Route Selection, QR Scan, Weight Entry, Offline Sync status, Advance Payment, and Session Summary. The principal office screens are: Dashboard,

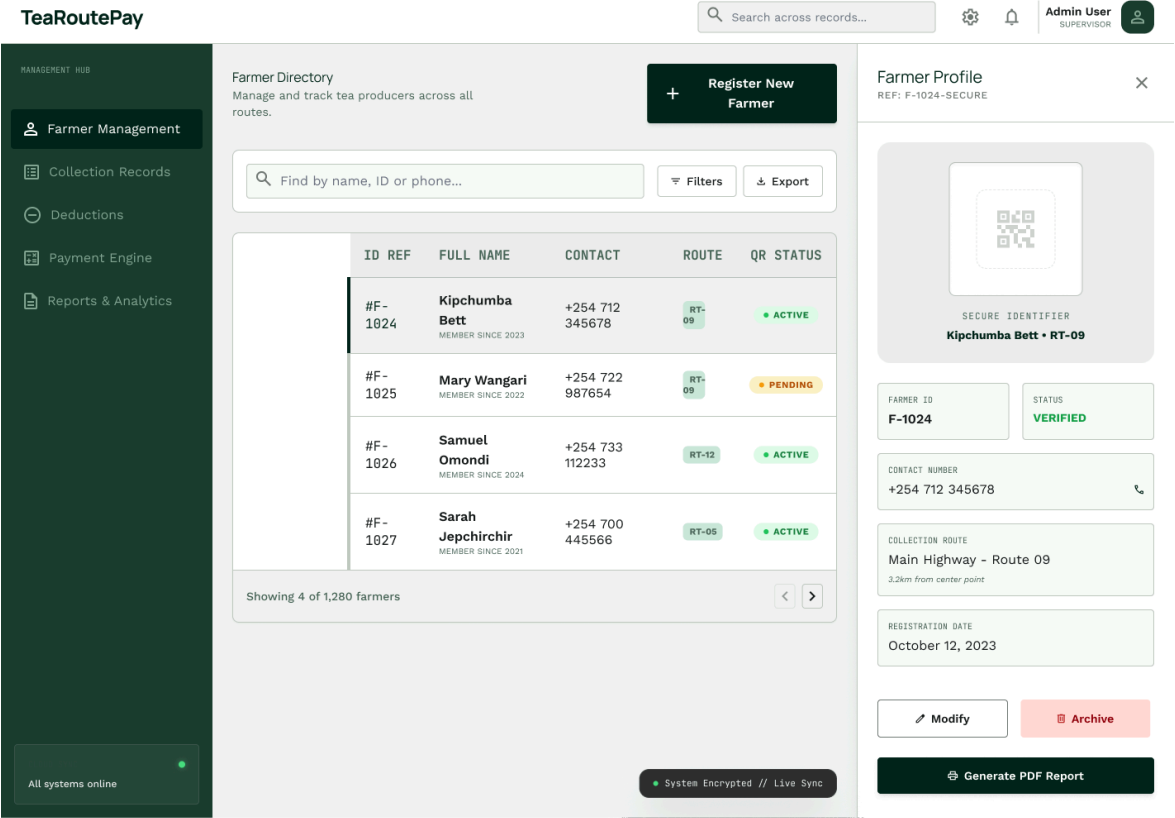
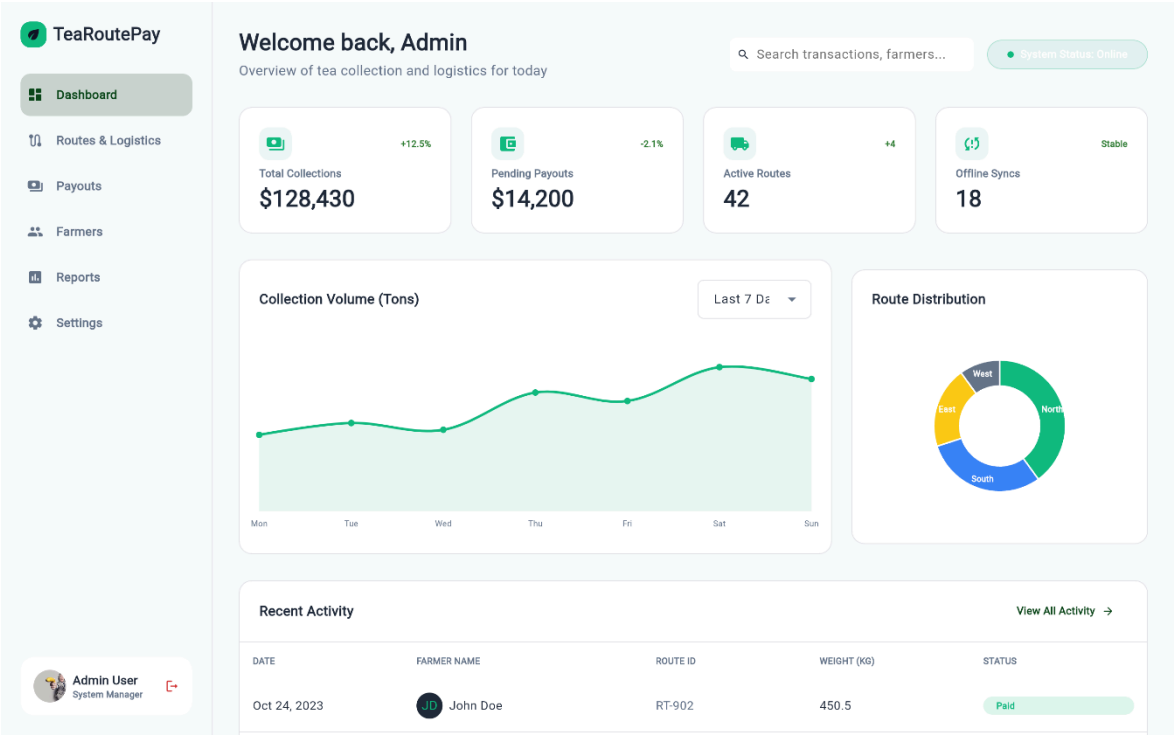
Farmer Management, Route Management, Deductions, Tea-Rate Setup, Payment Calculation, Approval Screen, Reports, and User Management.

3.5.1.1 Mobile Application Wireframes





3.5.1.2 Office Desktop Application Wireframes



TeaRoutePay

Search transactions...

Business Hub

ESTATE MANAGER

Farmers Management

Collection Records

Deduction Management

Payment Calculation

Receipts & Reports

Deduction Management

Manage and apply various farmer-level financial deductions for the current cycle.

REF: DED-2023-10

SEARCH FARMER

Enter ID or Name...

FARMER NAME

J. Doe - TRP0442

CYCLE EARNINGS

14,250.00 KES

PRIMARY FIELD

Block B, Plot 12

OUTSTANDING BALANCE

2,400.00 KES

Loan

INSTALLMENT AMOUNT

1500

REASON / LOAN ID

L-992-B

Fertilizer

UNITS (50KG BAGS)

2

UNIT PRICE

4500

Personal Use

WEIGHT (KG)

5

PROCESSING FEE

250

DEDUCTION CALCULATION SUMMARY

AUTOSAVED 2 MINS AGO

CATEGORY	DESCRIPTION	SUBTOTAL
Loan Repayment	ID: L-992-B (3 of 12)	1,500.00 KES
Agri-Inputs	2x Fertilizer Bags (Special Rate)	9,000.00 KES
Personal Supply	5kg Tea + Factory Processing	250.00 KES
TOTAL DEDUCTIONS		10,750.00 KES

CANCEL

UPDATE

SAVE & CONTINUE

Cloud Synced

Tea Route Pay | Page 114

TeaRoutePay

Search records...

Farmer Management

Collection Records

Deduction Management

Payment Calculation

Receipts & Reports

Payment Calculation

Calculate and verify farmer payouts for selected period.

Generate Payment

Confirm Payment

Select Month

October 2023

Payment Batch ID

PAY-2023-10-042

LIVE SYNC ACTIVE

Farmers (42)

FRM-001
Samuel Kirui

FRM-002
Esther Mwangi

FRM-003
John Mutua

FRM-004
Grace Wanjiku

FRM-005
Kiprotich Sang

Collection Totals

Date	Type	Net Weight (kg)	Rate (KES)	Subtotal
Oct 01, 2023	BLACK TEA	12.50	45.00	562.50
Oct 04, 2023	BLACK TEA	15.20	45.00	684.00
GROSS TOTAL		27.70 kg		1,246.50

Deduction Summary

Fertilizer Credit

- 150.00

Transport Levy

- 45.50

SACCO Contribution

- 50.00

TOTAL DEDUCTIONS

245.50

FINAL PAYOUT AMOUNT

KES 1,001.00

NET CALCULATED

Payment Status Log

Viewing real-time operations

[14:20:05] INFO Calculation initialized for FRM-001 (Samuel Kirui)

[14:20:08] INFO Deductions verified against SACCO database records.

[14:21:12] WARN Payment above 50,000 threshold requires Manager Authorization for Batch PAY-2023-10-042.

[14:22:45] SUCCESS Calculations finalized for current session. Awaiting Batch Confirmation.

System Status

● SYNCING

Local logs updating...

Tea Route Pay | Page 115

TeaRoutePay

Search system health...

?

⚙️

MAIN MENU

Farmer Management

Collection Records

Deduction Management

Sync Status

Receipts & Reports

System / Sync Health

Sync Status Dashboard

FORCE FULL RESYNC

Global Synchronization

Cross-module integrity check in progress

75%
COMPLETE

Data Integrity Check

100%

Cloud Handshake

82%

Database Writeback

45%

RUNNING

SYNCING

Farmer DB

Master registry and land coordinates

Records12,402

UPDATED

Collection Records

Daily weight logs and grade audits

Last entry14:22 PM

UPDATED

Deduction Tables

Loan repayments and supply costs

Sync statusStable

System Event Log

FILTER

EXPORT

TIMESTAMP	MODULE	EVENT DETAILS	STATUS	ACTION
16:45:02.11	Farmer DB	Partial delta sync initiated from regional node #04	SUCCESS	
16:42:15.89	Collection	Failed to resolve dependency for entity: COLL_0492	RETRY	
16:38:44.05	Deduction	Validation pass completed for Monthly Cycle B	SUCCESS	
16:35:10.22	Auth Engine	System integrity check: Signature match confirmed	PENDING	

CLOUD CONNECTION

000

Last sync: 2 mins ago

Data integrity is our harvest.

Serving 14,000+ small-scale farmers across the Central Province.

Tea Route Pay | Page 116

3.5.2 Navigation Flow

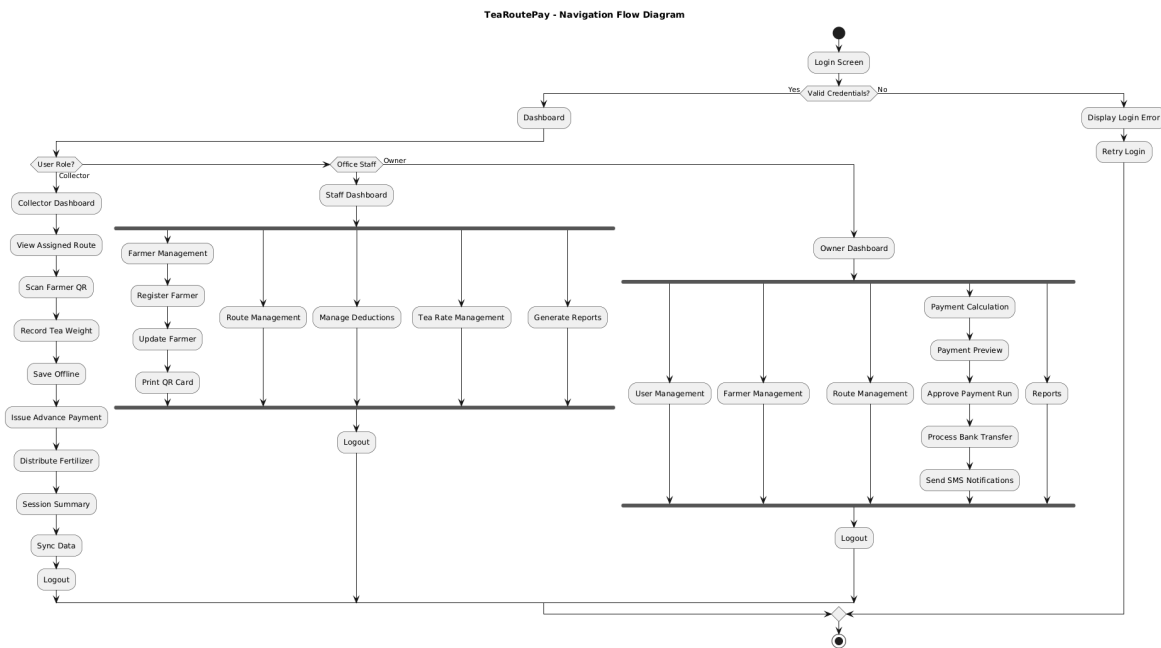


Figure 31: TeaRoutePay Navigation Flow Diagram — role-branched journeys for Collector, Office Staff, and Owner

The navigation model branches at authentication on the authenticated principal's role, producing three distinct journeys from a single login surface. After the Login Screen, a valid-credentials check routes a failure to Display Login Error and Retry Login, and a success to the Dashboard, where the role discriminator selects the journey.

The Collector journey is a linear, top-to-bottom field workflow optimised for repetition: Collector Dashboard → View Assigned Route → Scan Farmer QR → Record Tea Weight → Save Offline → Issue Advance Payment → Distribute Fertilizer → Session Summary → Sync Data → Logout. The linearity is deliberate — it mirrors the physical sequence of a route stop and minimises navigational decisions for the low-proficiency user. The Save Offline step sits in the critical path precisely because persistence must precede, and is independent of, the later Sync Data step.

The Office Staff journey fans out from a Staff Dashboard to the parallel administrative capabilities — Farmer Management (with its Register Farmer, Update Farmer, and Print QR Card sub-flow), Route Management, Manage Deductions, Tea Rate Management, and Generate Reports — before

converging on Logout. The parallel structure reflects that these are independent tasks selected on demand rather than a fixed sequence.

The Owner journey extends the staff capabilities with the privileged financial path: from the Owner Dashboard, the Owner reaches Payment Calculation → Payment Preview → Approve Payment Run → Process Bank Transfer → Send SMS Notifications, alongside User Management, Farmer Management, Route Management, and Reports. The placement of approval and disbursement exclusively within the Owner journey is the navigational expression of the maker-checker control: the approval screen is simply unreachable for a non-Owner principal.

4.5.3 UI Resources and Prototyping

The complete UI design system, including interactive components, assets, and low-fidelity prototypes, is maintained in the project's design workspace. The panel and supervisors may access the live Figma file via the link below:

- **TeaRoutePay Figma Design Workspace:**

<https://www.figma.com/design/t7JCaM3YTcm3Az4QTIPWwW/TeaRoutePay>

Note: The live workspace contains the most current version of the mobile and desktop wireframes; individual screen exports referenced as figures in this section were captured from this workspace at the time of document publication.

3.6. INTERFACE DESIGN

3.6.1 External Interfaces

3.6.1.1 Bank Transfer API

The Bank Integration Service disburses approved payments through the external Bank Transfer API. Communication is over HTTPS (TLS 1.3), authenticated with a bearer credential retrieved from the secrets manager and never embedded in source (NFR-09). A representative disbursement request and response are detailed below.

Element	Value	Notes
Method	POST	
Path	/api/transfers	(FR-43)
Header: Authorization	Bearer <token>	Retrieved from secrets manager (NFR-09)
Header: Content-Type	application/json	
Header: Idempotency-Key	UUID	Prevents duplicate disbursement on retry (NFR-19, NFR-21)
Body: paymentRunId	UUID	Identifies the approved run
Body: farmerId	UUID	Payee identifier
Body: accountNumber	VARCHAR	Farmer bank account (FR-43)
Body: amountCents	BIGINT	Integer cents — no floating point (FR-37, NFR-20)
Body: currency	"LKR"	Sri Lankan Rupee
Body: reference	"TRP-YYYY-MM-UUID"	Traceable payment reference
Response: transferId	"BNK-NNNNN"	Bank's transfer identifier (FR-44)
Response: status	SUCCESS / PENDING / FAILED	Mapped to internal taxonomy (FR-44)
Response: processedAt	ISO 8601 timestamp	Written to audit log (FR-45)

The Idempotency-Key is essential: should the request be retried after a network timeout, the bank must treat the repeat as the same logical transfer and not disburse twice — the external counterpart to the system's internal idempotency guarantee (NFR-19, NFR-21). Amounts are transmitted as

integer cents, never as decimals, consistent with the fixed-point financial-accuracy constraint. A Failed or timed-out transfer is retried with exponential back-off up to three attempts, after which it is surfaced for manual re-submission (FR-46).

3.6.1.2 SMS Gateway API

After a transfer is confirmed successful — and only then (FR-48) — the SMS Service dispatches a localised confirmation through the SMS Gateway API.

Element	Value	Notes
Method	POST	
Path	/api/send	(FR-47)
Header: Authorization	Bearer <token>	Secrets manager (NFR-09)
Body: to	"+9477XXXXXXX"	Farmer's registered phone (FR-07)
Body: language	"si" / "ta" / "en"	Farmer's preferred language (FR-49)
Body: templateId	"PAYMENT_CONFIRRM"	Localised template
Body: params.amountLKR	String (formatted)	Net payment amount
Body: params.month	"YYYY-MM"	Payment month
Response: messageId	"SMS-NNNNN"	Delivery tracking reference
Response: status	QUEUED / SENT / FAILED	Tracked for reporting

3.6.2 Internal Interfaces

Internal communication between the clients and the backend traverses the Spring Cloud API Gateway over HTTPS, carrying the bearer JWT issued at login. The principal internal endpoints, grouped by owning service, are as follows.

Service	Method & Path	Purpose	Requirement
Farmer	POST /farmers	Register farmer	FR-07
Farmer	GET /farmers/{id}	Retrieve profile	FR-13
Farmer	PUT /farmers/{id}	Update profile	FR-11
Collection	POST /collections	Ingest a collection record	FR-20
Collection	GET /collections/sync	Pull/confirm sync set	FR-26, FR-27
Payment	POST /payments/calculate	Run monthly calculation	FR-37, FR-38
Payment	POST /payments/approve	Approve and lock a run	FR-40, FR-41
Deduction	POST /deductions	Record loan/fertilizer/custom deduction	FR-32, FR-34
Route	POST /routes	Create collection route	FR-14
Auth	POST /auth/login	Issue JWT	FR-01
Reporting	GET /reports/daily/{date}	Daily collection report	FR-50
Reporting	GET /reports/payment/{month}	Monthly payment report	FR-51

3.6.2.1 Synchronisation Payload

The mobile-to-backend synchronisation payload is the most design-significant internal interface, because it must carry the data and the metadata required for idempotent, conflict-aware reconciliation. The collector device submits a batch of pending records to POST /collections/sync, each bearing its device-generated UUID and device timestamp.

Payload Field	Type	Description	Requirement
deviceId	String	Identifies the submitting device	FR-28
syncMode	Enum	LOCAL / FIRST_CLOUD_BACKUP / LOCAL / CLOUD	FR-25
records[].collectionId	UUID	Device-generated record identity	NFR-19
records[].farmerId	UUID	Farmer being recorded	FR-18
records[].weightGrams	Integer	Fixed-point weight (no floats)	FR-37, NFR-20
records[].collectionDate	ISO date	Calendar date of collection	FR-20
records[].deviceTimestamp	ISO 8601	Device clock timestamp for conflict resolution	FR-28
response.accepted	UUID[]	Records accepted; client advances to Synced	NFR-19
response.duplicatesIgnored	UUID[]	Records already present; idempotency confirmed	NFR-19
response.conflicts[].resolution	String	LAST_WRITE_WINS for contested records	FR-28

The response is the explicit realisation of synchronisation integrity: `duplicatesIgnored` confirms idempotency (a UUID already present is acknowledged, not re-inserted, NFR-19), and `conflicts` reports each contested record together with the deterministic resolution applied and logged for Owner review (FR-28). The client uses the `accepted` list to advance the corresponding local records from `PendingSync` to `Synced`.

3.7. SECURITY DESIGN

3.7.1 Authentication and Authorisation

Authentication is realised through the OAuth2 bearer-token model with JSON Web Tokens. On successful credential validation, the Authentication Service issues a JWT signed with RS256, an asymmetric algorithm that allows every downstream service to verify a token using the public key without ever holding the private signing key (FR-01, NFR-08). Tokens expire within the configured session timeout (FR-04), and the gateway rejects expired or malformed tokens at the perimeter. Account lockout after five failed attempts (FR-03) and audit logging of every authentication event (FR-06) complete the authentication design.

Authorisation is enforced through Role-Based Access Control over the four roles defined by FR-02, each with a distinct, non-overlapping permission set. The matrix below specifies the authoritative capability assignment; it is the structural enforcement of the maker-checker rule — note that Approve Payment Run is granted to the Owner alone.

Capability	Owner	Office Staff	Collector	Sys Admin
Field collection (scan, weigh, save)	—	—	Yes	—
Field deduction / advance	—	—	Yes	—
Synchronise collection data	—	Yes	Yes	—
Register / update farmer	Yes	Yes	Yes (field)	—
Deactivate farmer	Yes	Yes	—	—
Manage routes	Yes	Yes	—	—
Record deductions	Yes	Yes	—	—
Set monthly tea rate	Yes	Yes	—	—
Run payment calculation (maker)	Yes	Yes	—	—
Approve payment run (checker)	Yes	—	—	—
Process / re-submit bank transfer	Yes	Yes	—	—
Generate reports	Yes	Yes	—	—
Manage user accounts and roles	Yes	—	—	Yes
Configure system parameters	Yes	—	—	Yes

3.7.2 Data Protection

Data is protected both at rest and in transit, addressing NFR-06 and NFR-07.

- **At rest (server).** Farmer personal and financial data in the central PostgreSQL tier is encrypted using AES-256. Database credentials and all API keys are held in a secrets manager (AWS Secrets Manager or HashiCorp Vault) and are never committed to source control (NFR-09).
- **At rest (device).** The mobile SQLite store is encrypted with SQLCipher, which transparently applies AES-256 to the entire local database file. This is the critical control for the offline tier: a lost or stolen device does not expose collection or farmer data, because the at-rest data on the device is encrypted to the same standard as the server (NFR-07).
- **In transit.** All network communication — client-to-gateway, inter-service, and outbound to the Bank and SMS gateways — uses TLS 1.3 (the SRS mandates TLS 1.2 or higher; TLS 1.3 is selected as the stronger conforming choice) (NFR-06).
- **Input validation.** All inputs are validated server-side against DTO constraints; persistence uses parameterised queries exclusively, eliminating SQL-injection vectors, and output encoding prevents cross-site scripting on the web client.

3.7.3 Auditability

The audit design provides the immutable, attributable record required for financial compliance and dispute resolution (NFR-10). The Audit Service consumes events from the RabbitMQ bus and writes them to an append-only store: entries may be created and read but never updated or deleted by any role, through any interface. Every entry records the acting user, the action, the affected entity, the before/after values where applicable, and a timestamp — making every change to a farmer profile, a deduction, or a payment record traceable by user, date, and time. Because approved payment and collection records are themselves immutable (Section 4.4.4) and the audit log is append-only, the financial history is tamper-evident end to end: a discrepancy can always be attributed, and no actor can silently rewrite the past (FR-06, FR-42, FR-45).

3.7.4 OWASP Top 10 Mitigations

The design's defences are mapped to the OWASP Top 10 (2021) threat categories below.

OWASP Threat	TeaRoutePay Mitigation	Supporting Requirement
A01 Broken Access Control	RBAC matrix enforced at gateway and service level; maker-checker separation on payment approval	FR-02, FR-40, FR-41
A02 Cryptographic Failures	AES-256 at rest (PostgreSQL server and SQLCipher on device); TLS 1.3 in transit; RS256 JWT signing	NFR-06, NFR-07, NFR-08
A03 Injection	Parameterised queries throughout; server-side DTO validation; output encoding on web client	NFR-18
A04 Insecure Design	Threat-modelled architecture; immutable financial records; offline-first failure-safe handling	FR-41, NFR-17
A05 Security Misconfiguration	Hardened, minimal container images; externalised and reviewed configuration; no secrets in code	NFR-25, NFR-27
A06 Vulnerable and Outdated Components	Automated dependency scanning in CI/CD pipeline; library version pins reviewed at each build	NFR-25
A07 Identification and Authentication Failures	JWT with short expiry; RS256 signing; account lockout after 5 failures; configurable session timeout	FR-03, FR-04, NFR-08
A08 Software and Data Integrity Failures	Append-only immutable audit log; cryptographic locking of all approved payment records	NFR-10, NFR-11
A09 Security Logging and Monitoring Failures	Centralised Audit Service; structured JSON logs across all services; alerting on payment-service downtime	NFR-10, NFR-29
A10 Server-Side Request Forgery	Egress network filtering; strict allow-list of external endpoints (Bank API, SMS Gateway only)	NFR-06

3.8. NON-FUNCTIONAL DESIGN DECISIONS

This section consolidates the mapping of every non-functional requirement of the SRS (NFR-01 to NFR-29) to the specific architectural or detailed-design mechanism that satisfies it, providing the backward traceability that completes the requirements baseline.

NFR	Requirement (abridged)	Design Solution	Where Realised
NFR-01	QR scan and save weight < 1 s	Read/write served entirely from on-device SQLite; no network in the field path	§2.1, §3.5, SD-04
NFR-02	Payment report (500 farmers) < 5 s	Indexed queries; pre-computed payment results read by Reporting Service	§3.6, SD-14, SD-18
NFR-03	Full-day sync (500 records) < 30 s	Batched synchronisation payload over LAN; idempotent bulk ingest	§3.1, SD-07, §6.2.1
NFR-04	Payment calc (1,000 farmers) < 10 s	Aggregation queries; deterministic fixed-point engine in Payment Service	SD-13, §4.4.4
NFR-05	≥ 10 concurrent desktop sessions	Stateless services behind load-balanced gateway; Kubernetes horizontal scaling	§3.5
NFR-06	TLS 1.2+ for all communications	TLS 1.3 on all client, inter-service, and external links	§7.2
NFR-07	AES-256 encryption at rest	AES-256 on server; SQLCipher (AES-256) on device local store	§7.2
NFR-08	JWT RS256, expiring	RS256-signed JWT issued by Auth Service; expiry enforced at gateway	§7.1, SD-01
NFR-09	Secrets in secrets manager	Secrets manager for DB, Bank, and SMS credentials; none in source control	§6.1, §7.2
NFR-10	Immutable audit log	Append-only Audit Service fed by RabbitMQ event bus	§3.2, §7.3
NFR-11	Locked approved records	State-machine lock on approval; cryptographic record locking	§4.3.2, §7.4
NFR-12	Touch targets ≥ 48x48 dp	Oversized high-contrast controls on mobile client	§5.1

NFR	Requirement (abridged)	Design Solution	Where Realised
NFR -13	≤ 30 min collector training	Three-step scan→weigh→save flow; quick-select menus for deductions	§5.1, SD-04
NFR -14	Sinhala/Tamil/English UI	Per-user localisation on both clients and per-farmer language for SMS	§5.1
NFR -15	Plain-language error messages	Corrective, jargon-free error messaging on all screens	§5.1
NFR -16	Standard desktop conventions	Keyboard nav, tab order, tabular entry, standard shortcut keys on web client	§5.1
NFR -17	Zero field data loss	Unconditional local write before sync; durable transactional SQLite store	§2.1, §4.3.3, SD-04
NFR -18	ACID transactions	PostgreSQL with Spring @Transactional boundaries on all write paths	§3.6
NFR -19	Idempotent sync	UUID primary keys generated on device; duplicate detection on central ingest	§4.4.4, §6.2.1
NFR -20	Deterministic calculation	Fixed-point integer arithmetic; pure deterministic engine in Payment Service	§4.1, §4.4.3, SD-13
NFR -21	Retry with back-off (max 3)	Bounded retry queue on device sync; exponential back-off on external API calls	§4.4.2, SD-16
NFR -22	$\geq 2,000$ farmer records without degradation	Indexed PostgreSQL; database-per-service partitioning; horizontal scaling	§3.2, §3.5
NFR -23	≥ 5 concurrent collector syncs	Stateless, containerised ingest endpoints behind load balancer	§3.1, SD-07
NFR -24	Horizontal scaling without code change	Stateless Spring Boot services on Kubernetes with declarative HPA	§3.5

NFR	Requirement (abridged)	Design Solution	Where Realised
NFR -25	Documented coding standards and CI/CD	Containerised builds; GitHub Actions CI/CD; peer-reviewed PRs	§3.6
NFR -26	Schema migrations with rollback	Versioned migration framework (Flyway) per service schema	§3.2
NFR -27	Configurable params via admin UI	Externalised configuration for rates, timeouts, and SMS templates	§3.1, §7.4
NFR -28	$\geq 90\%$ unit-test coverage (payment logic)	Pure, deterministic calculation engine designed for isolated unit testing	§4.1, SD-13
NFR -29	Structured JSON logs	JSON-structured application logs across all services; compatible with Grafana/ELK	§7.4

APPENDIX A — ARCHITECTURAL DECISION RECORDS

Architectural Decision Records (ADRs) document the significant design decisions, the context that made them necessary, the alternatives considered, and the consequences accepted. They are maintained in the repository under documents/architecture/decisions/ and are reproduced here for completeness.

ADR-001: Adopt a Microservices Architecture

Field	Content
Status	Accepted
Context	The backend must scale the payment workload independently, isolate faults, and integrate an offline synchronisation subsystem with different availability characteristics from the transactional core.
Decision	Decompose the backend into independently deployable Spring Boot services, one per bounded capability, fronted by a Spring Cloud API Gateway.
Rationale	Supports independent scaling, fault isolation, and maintainability. Each service can be deployed, tested, and modified without disturbing the others. Addresses NFR-22 to NFR-26.
Alternatives	Monolithic (rejected — tight coupling impedes independent scaling and offline-sync integration); Layered/n-tier (rejected — does not accommodate the distributed field/office/cloud topology with differing availability profiles).
Consequences	Greater operational complexity (orchestration, observability, network latency between services) is accepted in exchange for independent scalability, fault isolation, and maintainability.

ADR-002: Use PostgreSQL as the Central Database

Field	Content
Status	Accepted
Context	The system processes financial transactions requiring strict correctness; payment records, once approved, must be cryptographically reliable and ACID-safe.
Decision	Use PostgreSQL with a schema-per-service ownership model. Each microservice owns its schema; no service may query another's schema directly.
Rationale	Full ACID compliance is the correctness foundation for monetary processing (NFR-18). Per-service schemas preserve architectural isolation while keeping one database platform to operate, back up, and migrate.
Alternatives	MySQL/MariaDB (rejected — weaker MVCC semantics); MongoDB (rejected — document model does not enforce the relational integrity required for payment runs and deductions).
Consequences	Strong transactional guarantees; standardised migration tooling (Flyway); single database platform for infrastructure operations. Join queries spanning service boundaries require API calls rather than SQL JOINS.

ADR-003: Use RabbitMQ for Asynchronous Event-Driven Communication

Field	Content
Status	Accepted
Context	External integrations (Bank API, SMS Gateway) and the Audit Service must not block or destabilise the core payment transaction; they must survive transient failures independently.
Decision	Publish financial events to RabbitMQ topics; consume them in the Bank Integration, SMS, and Audit services.
Rationale	Decoupling absorbs external latency and transient failure through consumer-side retry and back-off. The Audit Service consumes the same event stream, providing an immutable, ordered record of all financial events (NFR-10, NFR-21).
Alternatives	Synchronous HTTP calls (rejected — a slow bank API would block payment completion); Apache Kafka (considered — offers stronger ordering guarantees but higher operational complexity; RabbitMQ is sufficient for this scale).

Field	Content
Consequences	Eventual consistency for side-effects (bank transfer, SMS) is accepted. Reliability and auditability of those side-effects are strengthened by the durable queue.

ADR-004: Offline-First Mobile Architecture with Encrypted SQLite

Field	Content
Status	Accepted
Context	Field collection occurs where connectivity is intermittent or absent, yet no field datum may be lost; synchronisation must be reliable and conflict-aware when connectivity resumes.
Decision	Treat the on-device SQLite store (encrypted via SQLCipher) as the primary write path. Treat synchronisation as a deferred, idempotent, conflict-aware, bounded-retry concern.
Rationale	Realises FR-20, NFR-17 (zero loss), NFR-01 (sub-second interaction), and NFR-07 (encryption at rest on device). UUID-keyed records enable offline-safe identity generation and idempotent reconciliation (NFR-19).
Alternatives	Online-only mobile (rejected — fundamentally incompatible with rural operating environment); service worker / local-first web (rejected — camera access and native performance required for QR scanning).
Consequences	Synchronisation complexity (UUID keys, last-write-wins conflict resolution, bounded retry queue, sync-status lifecycle on every record) is accepted as the necessary cost of field operational autonomy.

— End of Document —

Chapter 4: Conclusion

4.1 Summary of Objectives

This report has presented a comprehensive analysis and design for the Tea Collection and Payment System. The following objectives set in Chapter 1 have been addressed:

- A full stakeholder analysis and requirements elicitation process was conducted, producing a structured SRS conforming to IEEE requirements documentation standards.
- A scalable, offline-first architecture was designed, supporting both LAN and cloud synchronisation modes to accommodate the client's connectivity constraints.
- Detailed UML design artefacts — including class diagrams, sequence diagrams, and ER diagrams — were produced and committed to the project repository.
- UI wireframes were developed for both the mobile collector application and the office desktop application.
- A security design addressing authentication, authorisation, data encryption, and OWASP Top 10 considerations was specified.
- The project was managed using GitHub Projects (Kanban board), with four sprint milestones, professional issue descriptions, and adherence to the Conventional Commits specification.

4.2 Limitations

The following limitations are acknowledged:

- The design phase has not yet been validated through a prototype evaluation with end users; usability issues may emerge during development.
- Exact bank transfer file formats depend on the client's bank, which was not confirmed at the time of writing.
- Performance benchmarks are estimates; load testing will be required during the development phase.

4.3 Future Work

The following enhancements are recommended for consideration in the development phase and beyond:

- Implementation and unit testing of all designed components in SENG 34213.
- Integration testing of the LAN and cloud synchronisation pipelines.

- User acceptance testing with tea collectors and office staff.
- Potential expansion to support multiple collection routes and multi-factory operations.
- Exploration of a web-based dashboard for real-time collection monitoring by the owner.

References

- SENG 31242 System Design Project Course Guideline & Industrial Standards
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- IEEE 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. The structure and content of this SDS follow the design-viewpoint model established by this standard.
- GitHub Repository:
<https://github.com/seng31242-tea-route-pay-2026/tea-collection-payment-system>
- REST API Design Guidelines — the conventions governing the internal and external interface specifications in Section 6.
- OWASP Top 10 (2021) — the threat taxonomy used as the basis for the security-design mitigations in Section 7.
- REST API Design Guidelines — the conventions governing the internal and external interface specifications in Section 6.